

Python avancé



Qualité logicielle, TDD, design patterns et CI/CD

Frédéric Gaurat

Formateur et développeur

Horaires

- 9h00
- pause
- 12h30
- 13h30
- pause
- 17h00

Ce que cette formation n'est pas

Ce n'est **pas** un « Python perfectionnement » généraliste.

Nous ne parlerons pas de :

- métaclasses, descripteurs et magie du langage pour eux-mêmes ;
- `asyncio`, multiprocessing, optimisation de performance ;
- Django, Flask, FastAPI ;
- data science et notebooks.

Ces sujets sont utiles. Ils ne sont pas ce qui rend une librairie interne fiable et durable.

Ce que cette formation est

Du **développement logiciel professionnel en Python**, avec un seul fil directeur :

Rendre des bibliothèques Python internes plus robustes,
plus testables et plus faciles à faire évoluer.

Le problème que nous traitons pendant 5 jours :

« Je dois modifier ce code, et je ne sais pas ce que je vais casser. »

La logique de la formation : guidée par les tests

Les tests ne sont pas un contrôle qualité de fin de chaîne.

Ce sont, dans l'ordre :

1. un **outil d'analyse** — écrire le test force à définir le comportement attendu ;
2. un **outil de conception** — un code difficile à tester est un code mal découpé ;
3. un **filet de sécurité** — ils autorisent le refactoring ;
4. un **contrat** — ils documentent ce que la librairie promet à ses consommateurs.

La vérification des régressions n'arrive qu'en cinquième position.

À la fin des 5 jours, vous saurez

Concevoir

- Découper un code pour le rendre testable
- Réduire le couplage et les dépendances circulaires
- Appliquer SOLID sans sur-architecturer
- Choisir un design pattern quand il résout un vrai problème

Tester et industrialiser

- Écrire des tests pytest utiles (et savoir lesquels ne pas écrire)
- Utiliser fixtures, paramétrisation, doubles de test
- Distinguer mock, stub, fake et monkeypatch
- Refactoriser du code existant sans régression
- Automatiser tout cela dans GitLab CI/CD

Déroulé des 5 jours

Jour	Thème	Question centrale
1	Qualité, POO testable, SOLID	<i>Pourquoi ce code est-il si dur à tester ?</i>
2	pytest, stratégie de test, TDD	<i>Que faut-il tester, et comment ?</i>
3	Mocking et design patterns	<i>Comment tester ce qui dépend de l'extérieur ?</i>
4	Refactoring et code legacy	<i>Comment améliorer sans casser ?</i>
5	GitLab CI/CD et qualité automatisée	<i>Comment rendre tout cela systématique ?</i>

Chaque journée : environ **la moitié du temps en pratique**, le reste en apport et discussion.

Prérequis et outillage

Ce qui est supposé acquis : fonctions, classes, modules, packages, environnements virtuels, usage courant de Git.

Outillage de la formation :

Outil	Rôle
Python 3.12	Le langage
<code>uv</code>	Environnement et dépendances
<code>pytest</code> + <code>pytest-cov</code>	Tests et couverture
<code>ruff</code>	Lint et formatage
<code>mypy</code>	Typage statique
GitLab CI/CD	Automatisation

Tout ce qui est montré avec `uv` reste faisable avec `python -m venv` + `pip`.

Conventions du support

Trois types de slides reviennent régulièrement :

- ! **Piège** — l'erreur classique sur le sujet qui vient d'être présenté.
- 🎯 **En pratique** — la règle de décision utilisable dès lundi prochain.
- À **retenir** — trois points maximum en clôture de chaque section.

Les exemples de code sont volontairement courts et souvent présentés **avant / après**. Le mauvais exemple n'est jamais laissé sans sa correction.

Comment nous allons travailler

- **19 travaux pratiques indépendants.** Chacun a son propre contexte : pas de projet fil rouge à suivre de bout en bout.
- Chaque TP contient un `README.md`, un dossier `depart/` et un dossier `corrige/` que vous pouvez consulter.
- Les questions sont bienvenues à tout moment, en particulier
« **chez nous, on a exactement ce problème** » — ce sont les meilleurs moments de la formation.

Un conseil pour ces 5 jours : gardez en tête **un** code de votre équipe qui vous pose problème. Nous y reviendrons.

1. Qualité logicielle appliquée aux librairies Python

Pourquoi une librairie n'est pas un gros script

Script, notebook, module, librairie

	Durée de vie	Lecteurs	Consommateurs	Coût d'une erreur
Script	heures	1	0	on relance
Notebook	jours	1-2	0	on réexécute
Module	mois	l'équipe	1 projet	on corrige
Librairie	années	inconnus	N projets	N corrections

La différence n'est pas technique. C'est le **nombre de personnes et de codes qui dépendent de vos choix**, et pendant combien de temps.

Ce que ça change concrètement

Dans un script, une décision est réversible : vous êtes le seul concerné.

Dans une librairie, chaque élément public devient une **promesse** :

- le nom d'une fonction ;
- l'ordre et le nom de ses paramètres ;
- le type de ce qu'elle retourne ;
- ce qu'elle fait quand l'entrée est invalide ;
- l'unité dans laquelle elle exprime son résultat.

Renommer un paramètre dans un script prend 10 secondes.

Dans une librairie utilisée par 6 projets, c'est une négociation.

Les problèmes classiques des librairies internes

1. Code difficile à comprendre
2. Effets de bord non annoncés
3. Fonctions ou méthodes trop longues
4. Dépendances implicites
5. Erreurs silencieuses
6. Absence de contrat clair
7. Modifications qui cassent d'autres codes

Aucun de ces problèmes n'est un problème de connaissance de Python.
Ce sont tous des problèmes de **conception** — et ils sont tous détectables.

Effet de bord non annoncé

Ce que fait la fonction

```
def valider(mesures: list[dict]) -> bool:
    for m in mesures:
        if m["valeur"] < 0:
            m["valeur"] = 0      # ← modifie
            m["corrige"] = True  # ← l'entrée
    return all("corrige" not in m
               for m in mesures)
```

Ce que l'appelant croit

```
if valider(mesures):
    archiver(mesures)
else:
    # mesures est déjà modifié...
    signaler(mesures)
```

Le nom promet une vérification.

Le corps effectue une correction.

Règle : une fonction qui s'appelle `valider` ne modifie rien. Si elle corrige, elle s'appelle `corriger` et **retourne** une nouvelle liste.

Erreur silencieuse

Avant

```
def charger_seuil(chemin):  
    try:  
        with open(chemin) as f:  
            return json.load(f)["seuil"]  
    except Exception:  
        return 50 # valeur « par défaut »
```

Fichier absent, JSON invalide, clé manquante, disque plein : le même 50.

Le premier code produit des résultats faux **sans jamais prévenir**.

Après

```
class ConfigurationInvalide(Exception):  
    """Configuration illisible ou incomplète."""  
  
def charger_seuil(chemin: Path) -> float:  
    try:  
        données = json.loads(chemin.read_text())  
    except OSError as e:  
        raise ConfigurationInvalide(chemin) from e  
    if "seuil" not in données:  
        raise ConfigurationInvalide(  
            f"{chemin} : clé 'seuil' absente")  
    return float(données["seuil"])
```

Le coût réel d'une erreur silencieuse

Un bug qui lève une exception coûte **une heure** : la trace pointe la ligne.

Un bug silencieux coûte **des semaines** :

1. les résultats sont légèrement faux pendant six mois ;
2. quelqu'un remarque une incohérence dans un rapport ;
3. on remonte la chaîne de traitement, projet par projet ;
4. on découvre la valeur par défaut ;
5. il faut recalculer et republier ce qui a été diffusé.

 **En pratique** : un `except Exception: pass` ou un `return None` en cas d'erreur est presque toujours une dette qui sera payée par quelqu'un d'autre.

Absence de contrat clair

```
def calculer(donnees, mode=1, flag=False, seuil=None):  
    ...
```

Questions sans réponse : que contient `donnees` ? que valent les modes 2 et 3 ?
que se passe-t-il si `seuil` reste `None` ? l'appel modifie-t-il `donnees` ?

Un contrat, c'est la réunion de quatre éléments :

Élément	Répond à
Signature typée	Qu'est-ce qui entre et qui sort ?
Docstring	Que fait la fonction, dans quelles unités ?
Exceptions déclarées	Que se passe-t-il quand ça se passe mal ?
Tests	Qu'est-ce qui est réellement garanti ?

Les trois premiers peuvent mentir. Le quatrième, non.

Ce que l'on cherche à sécuriser

Ce qu'on sécurise	Exemple concret	Type de test
Comportement métier	une règle de gestion, un seuil de validité	unitaire
Format de données	colonnes attendues, unités, encodage	unitaire + schéma
Calculs	une formule, une agrégation, un arrondi	unitaire paramétré
Compatibilité	l'API publique du package ne change pas	non-régression
Sources externes	fichier, base, API distante	intégration

Ces cinq lignes sont l'ordre de priorité que nous utiliserons au Jour 2 pour décider **quoi tester en premier**.

À retenir

1. Une librairie se distingue d'un script par le nombre de codes qui dépendent d'elle et par la durée pendant laquelle ils en dépendent.
2. Les problèmes classiques (effets de bord, erreurs silencieuses, absence de contrat) sont des problèmes de conception, pas de langage.
3. Une erreur silencieuse coûte toujours plus cher qu'une exception : échouer bruyamment est un service rendu à l'appelant.

2. Introduction à une démarche guidée par les tests

Le test comme outil d'analyse, pas comme corvée finale

Pourquoi parler des tests dès la conception

Le test est le **premier consommateur** de votre code.

Il l'utilise exactement comme le fera un autre projet : de l'extérieur, sans connaître les détails internes, en fournissant des entrées et en observant des sorties.

Conséquence directe :

Si votre code est pénible à tester, il sera pénible à utiliser.

La difficulté à écrire le test n'est pas un problème de test.

C'est un **retour d'information sur la conception**.

C'est pour cela que nous ne traitons pas les tests en fin de formation.

Tester après coup vs concevoir testable

Tester après coup

Le code existe, il faut « le couvrir ».

- On subit la structure existante
- On mocke beaucoup pour contourner
- Les tests deviennent fragiles
- On teste ce qui est facile, pas ce qui est risqué
- La couverture monte, la confiance non

Concevoir testable

On se demande, avant d'écrire : *comment saurai-je que c'est juste ?*

- Les dépendances deviennent des paramètres
- Le calcul se sépare des I/O
- Les fonctions rétrécissent naturellement
- Les tests sont courts et stables

La deuxième colonne ne demande pas plus de temps. Elle demande de se poser la question **avant** plutôt qu'après.

Le TDD n'est pas une technique de test

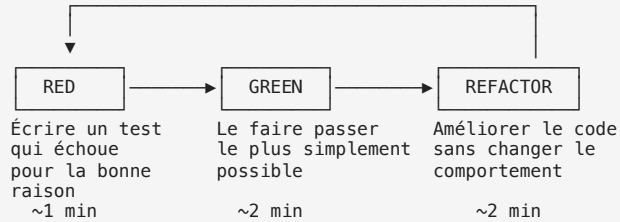
C'est une technique d'**analyse et de conception** qui produit des tests comme sous-produit.

Écrire le test d'abord vous force à répondre, avant d'écrire une ligne d'implémentation :

- Comment cette fonction s'appelle-t-elle ?
- Que reçoit-elle exactement, et sous quelle forme ?
- Que retourne-t-elle ? Dans quelle unité ?
- Que fait-elle quand l'entrée est invalide ?

Ce sont les questions qu'on ne se pose habituellement **qu'après** avoir écrit le code — c'est-à-dire trop tard pour changer d'avis sans coût.

Le cycle Red / Green / Refactor



L'unité de temps est la **minute**, pas la journée. Un cycle qui dure une demi-journée n'est pas un cycle TDD : c'est un développement classique avec un test écrit en premier.

Ce que chaque phase apporte réellement

Phase	Ce qu'on croit y faire	Ce qu'on y fait vraiment
Red	écrire un test	définir le comportement attendu et l'interface
Green	écrire du code	prouver que le test sait détecter l'absence de code
Refactor	faire joli	concevoir, sous protection du test qui vient de passer

La phase la plus souvent sautée est **Refactor**. C'est pourtant la seule où la conception s'améliore : sans elle, le TDD ne produit que des tests.

⚠ **Piège** : un test qui passe *du premier coup* n'a rien prouvé. Vous ignorez s'il est capable d'échouer. Voir le rouge fait partie du test.

Un cycle complet en 3 minutes

Red — le test décrit l'intention

```
def test_moyenne_horaire_simple():
    mesures = [10.0, 20.0, 30.0]
    assert moyenne_horaire(mesures) == 20.0
```

`ImportError` → rouge, pour la bonne raison.

Green — le minimum

```
def moyenne_horaire(mesures):
    return sum(mesures) / len(mesures)
```

Le cas `[]` a été trouvé **en écrivant le test**, pas en production.

Red — le cas limite découvert en écrivant

```
def test_moyenne_horaire_sans_mesure():
    with pytest.raises(SérieVide):
        moyenne_horaire([])
```

Green + Refactor

```
def moyenne_horaire(mesures: Sequence[float]) -> float:
    """Moyenne arithmétique d'une série non vide."""
    if not mesures:
        raise SérieVide("série de mesures vide")
    return sum(mesures) / len(mesures)
```

Où le TDD est clairement pertinent

Dans une librairie métier, le TDD fonctionne très bien sur :

- **les règles de gestion** — seuils, validations, conditions d'éligibilité ;
- **les transformations de données** — un format d'entrée vers un format de sortie ;
- **le parsing** — lire un fichier, une chaîne, un identifiant ;
- **les calculs à résultat connu** — une formule dont on peut écrire un cas de référence à la main ;
- **la correction de bugs** — le bug fournit le test rouge, gratuitement ;
- **la conception d'une API publique** — le test est le premier utilisateur.

Point commun : **on sait énoncer le résultat attendu avant de coder.**

Où une approche plus souple s'impose

Quand on ne connaît pas le résultat attendu à l'avance :

Situation	Pourquoi le TDD strict coince	Démarche adaptée
Exploration, prototype	On cherche encore le problème	Coder puis jeter, tester ce qui survit
Calcul numérique complexe	Le résultat exact est inconnu	Tester les propriétés : bornes, monotonie, symétrie, conservation
Portage d'un code existant	La référence, c'est l'ancien code	Tests de caractérisation (Jour 4)
Visualisation, rendu	Le résultat est visuel	Test de non-plantage + relecture humaine
Intégration d'un outil tiers	Le comportement réel est incertain	Un test d'intégration d'abord, pour apprendre

 **En pratique** : TDD strict sur le cœur métier, approche souple sur les bords. Personne ne fait du TDD à 100 %, et ce n'est pas le but.

À retenir

1. Le test est le premier consommateur de votre code : une difficulté à tester est un signal de conception, pas une contrainte de test.
2. Le TDD est une démarche d'analyse. Son produit principal est une meilleure interface ; les tests ne sont qu'un bénéfice secondaire.
3. Red / Green / **Refactor** — la troisième phase est celle qu'on saute et c'est la seule où la conception progresse.

3. Rappels POO Python orientés testabilité

Seulement les notions qui changent quelque chose aux tests

Une classe = une responsabilité = un axe de changement

La question n'est pas « combien de méthodes ? » mais :

Pour quelles raisons différentes ce fichier devra-t-il être modifié ?

Trois raisons de changer

```
class TraitementMesures:
    def lire_csv(self, chemin): ...
    def valider(self, lignes): ...
    def calculer_indice(self, lignes): ...
    def ecrire_rapport(self, chemin): ...
    def envoyer_mail(self, dest): ...
```

Le format change → on touche la classe.

La formule change → on touche la classe.

Le serveur SMTP change → on touche la classe.

Une raison chacun

```
class LecteurMesuresCSV: ... # format
class ValideurMesures: ... # règles
class CalculIndice: ... # formule
class RapportMarkdown: ... # rendu
class Notificateur: ... # envoi
```

`CalculIndice` se teste sans fichier,
sans réseau, en trois lignes.

Agrégation ou composition : qui crée quoi ?

Composition — l'objet crée sa dépendance

```
class Rapport:
    def __init__(self, chemin_db: str):
        self.db = Connexion(chemin_db) # ← ici
```

Pour tester `Rapport`, il faut une base.

La dépendance est **soudée**.

C'est l'idée la plus rentable de la journée, et elle tient en une ligne :

Ne construisez pas ce que vous pouvez recevoir.

Nous la reverrons sous son nom savant au chapitre 6 : *injection de dépendance*.

Agrégation — la dépendance est fournie

```
class Rapport:
    def __init__(self, source: SourceMesures):
        self.source = source # ← reçue
```

Pour tester, on fournit un objet en mémoire. La dépendance est **remplaçable**.

Le couplage, mesuré concrètement

Le couplage entre A et B, c'est le nombre de choses que A doit savoir sur B pour fonctionner.

Niveau	A connaît de B...	Testabilité
Fort	son type concret, qu'il instancie lui-même	× il faut un vrai B
Moyen	son type concret, mais B est fourni	~ il faut un vrai B, mais on choisit lequel
Faible	une seule méthode , quel que soit le type	✓ trois lignes suffisent

```
# Couplage minimal : Rapport n'exige qu'une chose de sa source
class SourceMesures(Protocol):
    def lire(self, jour: date) -> list[Mesure]: ...
```

Un test fournit alors n'importe quel objet ayant une méthode `lire`.

Loi de Déméter : ne parlez qu'à vos voisins

Chaîne de points = couplage caché

```
ville = commande.client.adresse.ville.nom
```

Cette ligne dépend de **quatre** classes.

Toute modification de l'une des quatre casse l'appelant.

Un test doit construire quatre objets imbriqués pour vérifier une ligne d'adresse.

Demander, ne pas fouiller

```
ville = commande.ville_de_livraison()
```

Une seule dépendance : `Commande` .

Le test fournit un objet avec une méthode, et c'est tout.

 **Piège** : cette règle vise les **objets**, pas les structures de données.

`config["db"]["host"]` OU `df.loc[...].mean()` ne posent pas de problème.

Encapsulation pragmatique en Python

Python n'a pas de `private`. Il a une **convention**, et elle suffit.

À ne pas faire — du Java en Python

```
class Mesure:
    def __init__(self, valeur):
        self.__valeur = valeur

    def get_valeur(self):
        return self.__valeur

    def set_valeur(self, v):
        self.__valeur = v
```

Trois fois plus de code, zéro garantie supplémentaire.

Pour les tests : ce qui est préfixé `_` ne se teste pas directement.

Si vous en éprouvez le besoin, c'est que ce code mérite sa propre classe publique.

Ce qui se fait

```
class Mesure:
    def __init__(self, valeur: float):
        self.valeur = valeur      # public, assumé
        self._cache = None       # interne

    @property
    def valeur_fahrenheit(self) -> float:
        return self.valeur * 9 / 5 + 32
```

`_` = « ceci peut changer sans préavis ».

`@property` = quand un calcul apparaît derrière un attribut public.

Duck typing : l'interface implicite

En Python, un objet est acceptable s'il a les bonnes méthodes. Point.

```
def exporter(destination, lignes: list[str]) -> None:
    for ligne in lignes:
        destination.write(ligne + "\n")
```

`destination` peut être un fichier, un `io.StringIO`, un socket, un objet maison.

La fonction ne demande rien d'autre qu'un `.write()`.

Le test devient trivial

```
def test_exporter_ajoute_les_sauts_de_ligne():
    tampon = io.StringIO()
    exporter(tampon, ["a", "b"])
    assert tampon.getvalue() == "a\nb\n"
```

Aucun fichier temporaire.

Aucun mock.

Aucune configuration.

C'est le duck typing qui rend cela possible, gratuitement.

ABC ou Protocol : lequel choisir ?

ABC — héritage explicite

```
class SourceMesures(ABC):
    @abstractmethod
    def lire(self, jour: date) -> list[Mesure]: ...

class SourceCSV(SourceMesures): # doit hériter
    def lire(self, jour): ...
```

- Impose l'héritage
- Erreur à l'instanciation si incomplet
- Permet du code partagé dans la base

Protocol — conformité structurelle

```
class SourceMesures(Protocol):
    def lire(self, jour: date) -> list[Mesure]: ...

class SourceCSV:
    def lire(self, jour): ... # n'hérite de rien
```

- Aucun héritage nécessaire
- Vérifié par `mypy`, pas à l'exécution
- Fonctionne sur du code que vous ne possédez pas

 **En pratique** : `Protocol` par défaut dans une librairie — il documente le contrat sans imposer de contrainte à vos consommateurs. `ABC` quand vous avez réellement du comportement commun à partager (voir *Template Method*, Jour 3).

Quand une abstraction n'apporte rien

Une interface avec **une seule implémentation** et **aucun besoin de test double** est du code mort déguisé en architecture.

Sur-design

```
class CalculeurInterface(ABC):  
    @abstractmethod  
    def calculer(self, x: float) -> float: ...  
  
class CalculeurStandard(CalculeurInterface):  
    def calculer(self, x): return x * 1.2
```

Suffisant

```
def calculer(x: float) -> float:  
    return x * 1.2
```

Testable, lisible, remplaçable
le jour où un second cas apparaîtra.

⚠ **Piège** : « on pourrait en avoir besoin plus tard » n'est pas une raison.

La deuxième implémentation est le bon moment pour créer l'abstraction — pas la première.

Objets de valeur : `dataclass(frozen=True)`

Beaucoup de bugs viennent de dictionnaires trimballés entre couches, dont on ne sait plus quelles clés sont garanties.

Avant

```
t = {"titre": "Déployer", "prio": 2}
# ailleurs, 400 lignes plus loin
t["priorite"] # KeyError
t["prio"] = 7 # personne ne vérifie
```

Après

```
@dataclass(frozen=True)
class Tache:
    titre: str
    priorite: int = 2

    def __post_init__(self):
        if not 1 <= self.priorite <= 3:
            raise PrioriteInvalide(self.priorite)
```

Bénéfices immédiats : `mypy` détecte les fautes de frappe, l'objet est immuable (donc sans effet de bord), la validation est faite **une seule fois**, et `==` fonctionne — ce qui rend les assertions de test lisibles.

Exceptions métier : dire ce qui s'est passé

Avant

```
if seuil is None:  
    return None  
if not chemin.exists():  
    return None  
if not lignes:  
    return None
```

L'appelant reçoit `None` et ne sait pas
laquelle des trois situations s'est produite.

Une **racine unique par librairie** permet à vos consommateurs d'écrire

`except ErreurMesures:` pour tout attraper, ou de cibler un cas précis —
sans jamais avoir à faire `except Exception:`.

Après — une hiérarchie

```
class ErreurMesures(Exception):  
    """Racine des erreurs de la librairie."""  
  
class FichierMesuresIntrouvable(ErreurMesures): ...  
class SérieVide(ErreurMesures): ...  
class SeuilNonConfiguré(ErreurMesures): ...
```

Ne perdez jamais la cause : `raise ... from`

Sans `from` — la trace originale disparaît

```
try:
    données = json.loads(texte)
except ValueError:
    raise ConfigurationInvalide(chemin)
```

Avec `from` — la cause est conservée

```
try:
    données = json.loads(texte)
except ValueError as e:
    raise ConfigurationInvalide(chemin) from e
```

La trace affiche alors :

```
... ValueError: Expecting ',' delimiter: line 4
```

```
The above exception was the direct cause of...
```

Et côté test, l'exception métier se vérifie précisément :

```
def test_configuration_illisible():
    with pytest.raises(ConfigurationInvalide, match="seuil"):
        charger_seuil(chemin_sans_seuil)
```

À retenir

1. Ne construisez pas ce que vous pouvez recevoir. Une dépendance instanciée dans un `__init__` est une dépendance non testable.
2. `Protocol` par défaut, `ABC` seulement quand il y a du comportement commun à partager. Une interface à une seule implémentation ne sert à rien.
3. Une exception métier explicite, dérivée d'une racine propre à la librairie, remplace avantageusement tout `return None`.

4. Introduction aux design patterns

Reconnaître un problème, pas réciter un catalogue

Un pattern est un nom donné à une solution récurrente

Les patterns n'ont pas été inventés : ils ont été **observés**. Des équipes différentes, sur des projets sans rapport, résolvaient les mêmes problèmes de la même façon. Quelqu'un a fini par leur donner des noms.

Un pattern comporte donc toujours trois parties :

Partie	Question
Le problème	Quelle douleur ressent-on ?
La solution	Quelle structure la soulage ?
Les conséquences	Qu'est-ce que ça coûte ?

On enseigne presque toujours la deuxième. C'est la **première** qui est utile au quotidien, et la **troisième** qui évite les catastrophes.

Connaître un pattern ≠ reconnaître le problème

Deux façons opposées d'utiliser un catalogue de patterns :

Partir de la solution ✗

« On va mettre une *Factory* ici, ça fera propre. »

Résultat classique : une classe

`FactoryProvider` qui fabrique un objet unique, dans un seul cas d'usage, appelée depuis un seul endroit.

Complexité ajoutée : réelle.

Problème résolu : aucun.

Partir du problème ✓

« Cette classe teste le type de fichier à cinq endroits différents, et il faut en ajouter un sixième. »

→ le problème est un **choix d'implémentation dispersé**.

→ le pattern qui y répond s'appelle *Factory*. On lui donne son nom après, pas avant.

En Python, la solution est souvent plus simple qu'en Java

Beaucoup de patterns du catalogue classique compensent des limitations que Python n'a pas : les fonctions sont des objets, les classes aussi, et un `dict` fait office de table de dispatch.

Strategy « à la Java »

```
class Strategie(ABC):
    @abstractmethod
    def calculer(self, x): ...

class Lineaire(Strategie):
    def calculer(self, x): return x

class Quadratique(Strategie):
    def calculer(self, x): return x ** 2
```

Strategy en Python

```
def lineaire(x): return x
def quadratique(x): return x ** 2

STRATEGIES = {
    "lineaire": lineaire,
    "quadratique": quadratique,
}
```

Même découplage, même testabilité,
un tiers du code.

 **En pratique** : passez à la version « classes » quand la stratégie a besoin d'un état, de plusieurs méthodes, ou d'une configuration propre.

Les problèmes récurrents dans une librairie Python

Cinq situations que vous rencontrerez dans vos propres codes :

1. **La création d'un objet est complexe** — il faut lire une configuration, choisir un type, brancher trois dépendances... et ce code est dupliqué partout.
2. **Le code dépend d'une source externe** — base, API, fichier — et rien ne peut être testé sans elle.
3. **L'algorithme dépend du contexte** — une cascade de `if` sur un mode, un format d'export, un type de client.
4. **Le code est trop couplé pour être testé** — la classe construit elle-même tout ce dont elle a besoin.
5. **Le workflow métier est difficile à faire évoluer** — les mêmes six étapes se répètent dans huit fonctions, avec des variantes.

Du problème vers le pattern

Le problème que vous vivez	Le nom de la solution
« Je ne peux pas tester sans la base / l'API »	Adapter, Repository
« J'ai une cascade de <code>if</code> sur un mode de calcul »	Strategy
« Ce code de création est dupliqué à six endroits »	Factory
« L'appelant doit connaître cinq classes pour faire une chose »	Facade
« Cette classe construit elle-même ses dépendances »	Dependency Injection
« Les mêmes étapes se répètent avec des variantes »	Template Method

Ce tableau se lit **de gauche à droite**. Jamais dans l'autre sens.

Le catalogue de la formation

Sept patterns, choisis parce qu'ils servent tous la même cause — **réduire le couplage et rendre le code testable** :

Pattern	En une phrase	Vu au
Adapter	Traduire une interface externe en une interface qui vous convient	J3
Strategy	Rendre un algorithme interchangeable	J3
Factory	Centraliser la décision « quel objet créer »	J3
Repository	Cacher totalement l'origine des données	J3
Facade	Offrir une porte d'entrée simple à un sous-système complexe	J3
Dependency Injection	Recevoir ses dépendances au lieu de les créer	J1 → J3
Template Method	Figurer le squelette d'un workflow, laisser varier les étapes	J3

Pour situer le vocabulaire : les trois familles classiques

Le catalogue historique (*Gang of Four*, 1994) range les patterns en trois familles. Nos sept s'y placent ainsi :

Famille	Question posée	Nos patterns
Créationnels	<i>comment l'objet est-il construit ?</i>	Factory, Dependency Injection
Structurels	<i>comment les objets s'assemblent-ils ?</i>	Adapter, Facade, Repository
Comportementaux	<i>comment le comportement varie-t-il ?</i>	Strategy, Template Method

La classification aide à s'y retrouver dans un catalogue. Elle ne dit rien de **quand** utiliser un pattern — c'est le tableau problème → pattern de la slide précédente qui sert à cela.

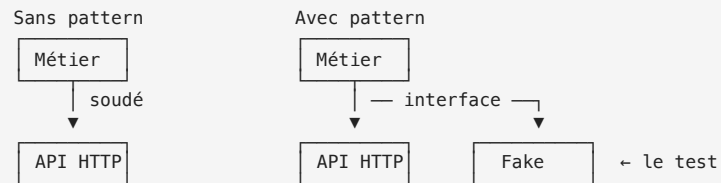
Repository n'est pas dans le catalogue de 1994 : il vient du *Domain-Driven Design* (2003). Il est structurel par nature, et c'est le plus utilisé des sept dans les librairies qui accèdent à des données.

Ce que les patterns ont en commun

Regardez la colonne du milieu du tableau précédent. Tous font la même chose :

Ils introduisent un **point de substitution** là où le code était soudé.

Or un point de substitution, c'est exactement ce dont un test a besoin :
un endroit où glisser un objet simple, prévisible, en mémoire.



C'est pour cette raison que les patterns arrivent au Jour 3, entre le mocking et le refactoring : ce sont les **alternatives à un mock compliqué**.

À retenir

1. Un pattern se reconnaît par son **problème**, jamais par sa structure.
Si vous ne savez pas nommer la douleur, n'appliquez pas le remède.
2. En Python, la première solution à essayer est une fonction, un `dict` ou un paramètre. Le pattern « à classes » vient quand cela ne suffit plus.
3. Tous les patterns de cette formation créent un **point de substitution** — c'est ce qui les rend utiles aux tests.

5. Identifier le code difficile à tester

Six symptômes, et ce qu'ils révèlent

L'heuristique de base

Avant de lire du code, posez-vous une seule question :

De quoi ai-je besoin pour vérifier que cette fonction est juste ?

Réponse	Diagnostic
« Trois valeurs et un <code>assert</code> »	Le code est bien découpé
« Un fichier temporaire »	Acceptable, mais l'I/O n'est pas isolée
« Une base de données »	Le métier est mélangé à la persistance
« Le réseau et un jeton d'API »	La frontière du système n'est pas matérialisée
« Tout l'environnement de production »	Il n'y a pas de composant, juste un programme

Aucune de ces réponses ne parle de tests. Elles parlent toutes de **conception**.

Symptôme 1 — La méthode longue

Une méthode de 300 lignes contient typiquement 12 décisions imbriquées, donc plusieurs milliers de chemins d'exécution possibles.

Conséquences mesurables :

- On ne peut pas la tester : on ne peut atteindre qu'une poignée de chemins.
- On ne peut pas la lire : elle ne tient pas dans un écran ni dans une tête.
- On ne peut pas la modifier : personne ne sait ce que touche un changement.
- Elle est **toujours** en train de faire plusieurs choses — sinon elle serait courte.

 **En pratique** : le seuil d'alerte utile n'est pas un nombre de lignes, c'est le moment où vous devez **faire défiler** pour comprendre le début.

Nous la démonterons méthodiquement au Jour 4.

Symptôme 2 — Calcul et I/O mélangés

Avant — intestable sans fichiers

```
def traiter(chemin_in, chemin_out):  
    with open(chemin_in) as f:  
        lignes = list(csv.DictReader(f))  
        total = 0.0  
        for l in lignes:  
            v = float(l["valeur"])  
            if v > 0:  
                total += v * 1.15  
        with open(chemin_out, "w") as f:  
            f.write(f"{total:.2f}\n")
```

Tester la formule `* 1.15` exige
d'écrire un CSV et de relire un fichier.

`corriger` se teste en une ligne. Et c'est elle qui porte le risque métier.

Après — le cœur est une fonction pure

```
def corriger(valeurs: list[float]) -> float:  
    return sum(v * 1.15 for v in valeurs if v > 0)  
  
def lire(chemin: Path) -> list[float]:  
    with chemin.open() as f:  
        return [float(l["valeur"])  
                for l in csv.DictReader(f)]  
  
def traiter(chemin_in: Path, chemin_out: Path) -> None:  
    total = corriger(lire(chemin_in))  
    chemin_out.write_text(f"{total:.2f}\n")
```

Symptôme 3 — Accès direct à une ressource externe

```
class Avancement:
    def calculer(self, projet: str) -> float:
        r = requests.get(f"https://api.example.org/projets/{projet}")
        taches = r.json()["taches"]
        return sum(t["terminee"] for t in taches) / len(taches)
```

Trois problèmes en quatre lignes :

Problème	Conséquence sur le test
L'URL est en dur	Impossible de pointer ailleurs
<code>requests</code> est appelé directement	Il faut mocker au niveau du module
Le parsing et le calcul sont mélangés	On ne peut pas tester la moyenne seule

Ce code **peut** se tester avec des mocks. Il sera plus simple de le tester après l'avoir découpé (Jour 3, *Adapter*).

Symptôme 4 — L'objet qui construit ses dépendances

Avant

```
class GénérateurRapport:
    def __init__(self):
        self.db = psycopg2.connect(DSN)
        self.cache = Redis(host="prod-01")
        self.mailer = SMTP("smtp.interne")
```

Instancier cet objet dans un test ouvre trois connexions réseau.

Il n'existe **aucun moyen** de le tester sans infrastructure : le constructeur ne laisse aucune prise.

 **Piège** : ce symptôme est le plus fréquent et le plus coûteux, parce qu'il est contagieux — un objet non injectable rend non testable tout ce qui l'utilise.

Après

```
class GénérateurRapport:
    def __init__(self, source: SourceMesures,
                 notif: Notificateur):
        self.source = source
        self.notif = notif
```

Le test fournit deux objets de dix lignes.

Bonus : le constructeur **documente** les dépendances réelles de la classe, qui étaient jusque-là invisibles.

Symptôme 5 — Variables globales et état partagé

```
_CONFIG = {}          # rempli au premier appel, quelque part

def calculer(x):
    if not _CONFIG:
        _CONFIG.update(json.loads(Path("config.json").read_text()))
    return x * _CONFIG["facteur"]
```

Les tests deviennent **dépendants de leur ordre d'exécution** :

- le premier test remplit `_CONFIG` ;
- le deuxième hérite de la valeur du premier ;
- lancé seul, le deuxième échoue ; lancé après le premier, il passe.

C'est le symptôme le plus démoralisant, parce qu'il produit des tests
« qui échouent une fois sur trois » et qu'on finit par ignorer.

Correctif : passer la configuration en paramètre, ou la charger une fois au point d'entrée et la transmettre explicitement.

Symptôme 6 — Dépendances circulaires

```
# mesures.py
from rapports import formater      # ← rapports importe mesures

# rapports.py
from mesures import Mesure        # ← mesures importe rapports
```

Symptômes typiques : `ImportError: cannot import name ...`, imports placés au milieu d'une fonction « pour que ça marche », ordre d'import qui compte.

Ce que cela signifie en réalité :

Deux modules qui s'importent mutuellement **sont un seul module** qui n'a pas été découpé au bon endroit.

Impossible de tester l'un sans charger l'autre — donc impossible de tester une partie du système isolément. Traité en détail au Jour 4.

Grille de diagnostic

À utiliser sur votre propre code, cette semaine :

Question	Si oui →
Dois-je faire défiler pour lire la fonction ?	Découper (J4)
La fonction ouvre-t-elle un fichier et calcule-t-elle ?	Extraire une fonction pure
Le constructeur ouvre-t-il une connexion ?	Injecter la dépendance
Y a-t-il un état modifiable au niveau du module ?	Passer en paramètre
Deux modules s'importent-ils mutuellement ?	Extraire un troisième module
Le nom contient-il « et », « manager », « utils », « process » ?	Chercher les responsabilités cachées

 **En pratique** : trois « oui » sur la même fonction suffisent à en faire une bonne candidate pour le premier refactoring de votre équipe.

À retenir

1. « De quoi ai-je besoin pour tester ceci ? » est un diagnostic de conception déguisé en question de test.
2. Le symptôme le plus rentable à corriger est l'objet qui construit ses propres dépendances : il contamine tout ce qui l'utilise.
3. Ces symptômes ne sont pas des fautes de goût. Chacun a un coût mesurable en temps de correction et en bugs silencieux.

6. Principes SOLID utiles en Python

Cinq principes, une seule intention

Cinq principes, un objectif commun

	Principe	Ce qu'il demande
S	Single Responsibility	Une seule raison de changer par module
O	Open/Closed	Étendre sans modifier l'existant
L	Liskov Substitution	Un sous-type doit pouvoir remplacer son type
I	Interface Segregation	Ne pas imposer ce dont on n'a pas besoin
D	Dependency Inversion	Dépendre d'abstractions, pas de détails

L'intention unique derrière les cinq :

Faire en sorte qu'un changement reste local.

Les tests sont le meilleur détecteur de violation : quand un changement casse des tests éloignés du code modifié, un de ces principes a été enfreint.

S — Une seule raison de changer

Le principe le plus mal cité. Il ne dit pas « une classe ne fait qu'une chose », il dit :

Un module doit avoir **une seule source de demandes de modification**.

```
class RapportMensuel:
    def calculer_totaux(self): ...    # ← demandé par le métier
    def formater_html(self): ...    # ← demandé par la communication
    def envoyer(self): ...          # ← demandé par l'exploitation
```

Trois interlocuteurs différents modifient le même fichier. Chacun risque de casser le travail des deux autres — et les tests de tout le monde.

Le découpage suit les **interlocuteurs**, pas les verbes.

S — Jusqu'où découper ?

Découpage utile

```
class CalculTotaux: ...  
class RenduHTML: ...  
class Expéditeur: ...
```

Trois fichiers, trois responsables,
trois suites de tests indépendantes.

Découpage nuisible

```
class AdditionneurDeLignes: ...  
class ArrondisseurDeTotaux: ...  
class FormateurDeNombre: ...  
class ValideurDeFormat: ...
```

Douze fichiers pour cinquante lignes.
Comprendre le calcul exige d'ouvrir
quatre fichiers.

 **En pratique** : découpez quand vous avez **une raison observée** —
deux interlocuteurs, deux rythmes de changement, ou un besoin de test isolé.
Pas quand une méthode « fait deux choses » sur le papier.

O — Ouvert à l'extension, fermé à la modification

Le symptôme d'une violation est facile à repérer : **ajouter un cas oblige à rouvrir une fonction existante**, et donc à risquer de casser les cas déjà testés.

```
def exporter(données, format):  
    if format == "csv":  
        ...  
    elif format == "json":  
        ...  
    elif format == "xml":      # ← ajouté le mois dernier  
        ...  
    elif format == "parquet":  # ← à ajouter cette semaine  
        ...
```

À chaque ajout : la fonction grossit, tous ses tests sont rejoués, et un `elif` mal placé peut casser l'export CSV utilisé par quatre projets.

O — En Python, un `dict` suffit souvent

Registre explicite

```
EXPORTEURS = {
    "csv": exporter_csv,
    "json": exporter_json,
    "xml": exporter_xml,
}

def exporter(données, format: str) -> str:
    try:
        return EXPORTEURS[format](données)
    except KeyError:
        raise FormatInconnu(format) from None
```

Ajouter Parquet = écrire une fonction

- une ligne dans le dictionnaire.
`exporter` n'est jamais rouverte.

Ce que ça change pour les tests

```
@pytest.mark.parametrize("format", EXPORTEURS)
def test_tous_les_formats(format, données):
    assert exporter(données, format)
```

Le test couvre automatiquement les formats futurs.

Chaque exporteur se teste seul, sans passer par `exporter`.

L — Un sous-type ne doit pas piéger l'appelant

Liskov, en pratique : une sous-classe ne doit pas être plus exigeante ni moins généreuse que sa classe de base.

```
class SourceMesures:
    def lire(self, jour: date) -> list[Mesure]:
        """Retourne les mesures du jour, éventuellement vide."""

class SourceAPI(SourceMesures):
    def lire(self, jour: date) -> list[Mesure]:
        if jour > date.today():
            raise ValueError("date future")    # ← nouvelle exigence
        ...
```

Tout code écrit contre `SourceMesures` fonctionne — jusqu'au jour où on lui passe une `SourceAPI`.

 **En pratique** : le test qui protège de cela est un **test de contrat**, rejoué sur chaque implémentation. Nous en écrirons un au Jour 3.

I — Ne pas imposer ce dont on n'a pas besoin

Interface trop grosse

```
class SourceDonnées(Protocol):  
    def lire(self, jour): ...  
    def écrire(self, mesures): ...  
    def supprimer(self, id): ...  
    def statistiques(self): ...
```

Un calcul en lecture seule doit fournir **quatre** méthodes dans son fake, dont trois qu'il n'appellera jamais.

C'est le principe SOLID le plus directement rentable pour les tests : il divise la taille des doubles de test par le nombre de méthodes inutiles.

Interfaces étroites

```
class Lecteur(Protocol):  
    def lire(self, jour: date) -> list[Mesure]: ...  
  
class Écrivain(Protocol):  
    def écrire(self, mesures: list[Mesure]) -> None: ...
```

Le fake du test fait deux lignes.

Et la signature de la fonction **documente** qu'elle ne modifie rien.

D — L'abstraction, c'est souvent juste un paramètre

Dépendance vers un détail

```
class Analyse:
    def __init__(self):
        self.source = ClientAPIProd() # détail
```

`Analyse` (haut niveau, métier) dépend
d'un client HTTP (bas niveau, technique).

Dépendance inversée

```
class Analyse:
    def __init__(self, source: Lecteur):
        self.source = source # abstraction
```

`Analyse` ne dépend plus que d'un
contrat. C'est l'appelant qui choisit
l'implémentation.

En Python, l'inversion de dépendance ne demande **ni conteneur, ni framework, ni fichier XML**. Elle demande de déplacer une instantiation du constructeur vers l'appelant.

```
analyse = Analyse(source=ClientAPIProd()) # production
analyse = Analyse(source=SourceEnMémoire(mesures_de_test)) # test
```

Les limites de SOLID en Python

SOLID a été formulé pour des langages à typage statique et héritage obligatoire. Python en absorbe une partie gratuitement :

Principe	Ce que Python offre déjà
O	Fonctions de première classe, <code>dict</code> de dispatch
L	Duck typing : pas de hiérarchie imposée

Les dérives à éviter :

- une interface par classe, systématiquement ;
- des classes d'une seule méthode partout ;
- une usine à fabriquer les usines ;
- l'application de SOLID à un script de 40 lignes.

 **Piège** : le sur-design est aussi coûteux que le sous-design, et bien plus difficile à défaire — parce qu'il a l'air sérieux.

À retenir

1. Les cinq principes servent un seul objectif : **qu'un changement reste local**. Vos tests vous diront quand il ne l'est pas.
2. Les deux qui rapportent le plus, tout de suite : **DIP** (recevoir ses dépendances) et **ISP** (interfaces étroites). Les deux réduisent immédiatement l'effort de test.
3. Appliquez un principe quand vous avez **observé** le problème qu'il résout — jamais par anticipation.

Travaux pratiques du Jour 1

Trois exercices indépendants — 2 h

Mise en place

```
cd TPs
uv sync # une seule fois
uv run pytest j1-02-decoupage-responsabilites/corrige
```

Chaque TP suit la même structure :

README.md	contexte, objectif, étapes, critères de réussite
depart/	le code sur lequel vous travaillez
corrige/	une solution possible — après avoir cherché

Les trois TP sont **indépendants** : chacun a son propre petit programme.

Le programme

TP	Ce que vous allez faire	Durée
j1-01	Auditer un script todo-list de 180 lignes que personne n'ose modifier. Aucun code : un diagnostic, sept symptômes à localiser, un découpage à proposer.	45 min
j1-02	Extraire des fonctions pures d'un <i>hello world</i> qui mêle heure, langue, configuration et <code>print</code> . Isoler les I/O, écrire les premiers tests — sans changer un seul message.	45 min
j1-03	Remplacer neuf comportements silencieux d'une calculatrice (<code>10 / 0</code> → <code>0.0</code>) par des exceptions métier explicites, sans jamais perdre la cause d'origine.	30 min

 **Consigne commune** : à chaque étape, posez-vous la question du chapitre 5 — *de quoi aurais-je besoin pour tester ceci ?* C'est elle qui guide le découpage.

Le TP j1-01 se fait de préférence **en binôme** : le diagnostic gagne beaucoup à être discuté à voix haute.

1. Le rôle des tests dans une librairie partagée

Ce qu'un test protège, et de quoi

Pourquoi tester une librairie interne

Une application a un utilisateur qui se plaint. Une librairie a **N projets qui ne se plaignent pas** — parce qu'ils ne savent pas encore que le résultat a changé.

	Application	Librairie
Qui détecte la régression ?	l'utilisateur, vite	un autre projet, des mois plus tard
Coût de la correction	un correctif	un correctif × N mises à jour
Qui connaît le comportement attendu ?	l'équipe	personne — sauf les tests

Dans une librairie, la suite de tests est le seul endroit où le comportement attendu est écrit **de manière exécutable**. La documentation, elle, vieillit en silence.

Trois natures de défaut, trois coûts

	Bug visible	Bug silencieux	Régression
Symptôme	exception, plantage	résultat faux, plausible	ce qui marchait ne marche plus
Découverte	immédiate	tardive, par hasard	lors d'une mise à jour
Coût	faible	très élevé	élevé, et politique
Détecté par	à peu près n'importe quoi	un test de calcul ciblé	une suite de tests existante

Les deux dernières colonnes justifient à elles seules l'effort de test dans une librairie. La première n'a jamais eu besoin de tests pour être trouvée.

 **En pratique** : ne dimensionnez pas votre effort de test sur les bugs visibles. Ils se signalent tout seuls.

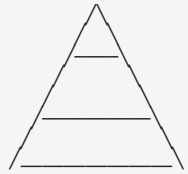
Unitaire, intégration, fonctionnel

	Périmètre	Dépendances réelles	Durée	Ce qu'il prouve
Unitaire	une fonction, une classe	aucune	µs	<i>le calcul est juste</i>
Intégration	un composant + une frontière	une (base, fichier, API)	ms-s	<i>le branchement fonctionne</i>
Fonctionnel	la chaîne complète	toutes	s-min	<i>l'ensemble produit le bon résultat</i>

Les trois sont utiles et **ne se remplacent pas** :

- 200 tests unitaires ne garantissent pas que votre requête SQL est correcte ;
- un test fonctionnel vert ne dit pas *laquelle* de vos 40 formules est juste.

La pyramide, et ce qu'elle veut vraiment dire



fonctionnels	quelques-uns	lents, fragiles diagnostic faible
intégration	quelques dizaines	
unitaires	des centaines	rapides, stables diagnostic précis

La forme n'est pas un dogme esthétique. Elle découle d'un arbitrage :

Plus un test est haut, plus il **couvre** de code — et moins il **localise** la panne. Un test fonctionnel rouge dit « quelque chose est cassé ».

Un test unitaire rouge dit « la ligne 34 de `calculer_indice` ».

Sur du code scientifique, la base de la pyramide est souvent la seule partie rentable à construire en premier.

Tester le comportement, pas l'implémentation

Test lié à l'implémentation ✗

```
def test_calcul():  
    c = Calculateur()  
    c._preparer()  
    assert c._cache == {}  
    c._appliquer_coefficient(2)  
    assert c._resultat_intermediaire == 2
```

Renommer `_cache` casse le test.

Le comportement, lui, n'a pas changé.

Test lié au comportement ✓

```
def test_calcul():  
    assert Calculateur().calculer(2) == 4
```

Toute réécriture interne est libre,
tant que `calculer(2)` vaut 4.

C'est **exactement** ce dont vous avez
besoin pour refactoriser au Jour 4.

⚠ **Piège** : un test qui casse à chaque refactoring, alors que rien de visible n'a changé, ne protège rien. Il **empêche** l'amélioration du code.

Le test comme contrat exécutable

Chaque test énonce une clause du contrat de votre librairie :

```
def test_indice_est_nul_en_absence_de_mesure():  
    """Sans mesure exploitable, l'indice est indéterminé – pas zéro."""  
    assert calculer_indice([]) is None
```

Ce test dit trois choses à vos consommateurs :

1. cette situation **est prévue** (elle n'est pas un oubli) ;
2. voici ce qui est retourné (`None`, pas `0`, pas une exception) ;
3. si quelqu'un change cela un jour, **il le fera sciemment** — le test rougira.

Le nom du test fait partie du contrat. `test_cas_3` ne documente rien.

`test_indice_est_nul_en_absence_de_mesure` remplace un paragraphe.

Ce qu'il faut tester au-delà du cas nominal

Le cas qui marche est le plus facile à écrire, et le moins utile : c'est celui que vous avez déjà vérifié à la main.

Catégorie	Exemples concrets
Bornes	0, 1, la valeur du seuil, le seuil $\pm$ 1, la dernière heure du jour
Vide	liste vide, chaîne vide, fichier de 0 octet, <code>None</code>
Volume	un seul élément, deux, beaucoup
Invalides	type inattendu, valeur négative, date future, encodage exotique
Erreurs attendues	l'exception métier est-elle bien levée, avec le bon message ?

Sur les slides du Jour 1, quatre des six bugs silencieux du TP j1-01 étaient dans cette colonne de droite. Aucun n'était dans le cas nominal.

Un test qui échoue doit vous dire quoi faire

Comparez deux échecs pour le même bug :

Diagnostic faible

```
FAILED test_pipeline
AssertionError: assert False
```

Il faut ouvrir le test, lire 40 lignes,
et lancer un débogueur.

Diagnostic immédiat

```
FAILED test_conversion_mg_vers_ug
assert 12.4 == 12400.0
+ where 12.4 = normaliser(12.4, 'mg/m3')
```

Le nom donne la règle métier fautive.
L'assertion donne l'entrée et l'écart.

Trois pratiques suffisent : **un test = un comportement**, un nom qui décrit ce comportement, et une assertion sur une valeur — pas sur un booléen déjà calculé.

À retenir

1. Dans une librairie, la suite de tests est le seul endroit où le comportement attendu est écrit de façon exécutable — et donc le seul qui ne mente pas.
2. Testez le **comportement** observable. Un test qui connaît les attributs privés bloque le refactoring au lieu de le protéger.
3. L'effort se concentre sur les bugs silencieux et les régressions : bornes, cas vides, entrées invalides, erreurs attendues.

2. Prioriser ce qui doit être testé

Un budget de test est fini — il faut le dépenser

On ne teste pas tout : on choisit

Personne n'a le temps de tester exhaustivement une librairie existante. La question réaliste n'est donc pas « *faut-il tester ?* » mais :

Où placer les 20 premiers tests pour éliminer 80 % du risque ?

Deux critères suffisent à décider :

Critère	Question
Risque	Si c'est faux, combien de temps avant qu'on s'en aperçoive ?
Volatilité	À quelle fréquence ce code est-il modifié ?

Un code risqué **et** souvent modifié est votre priorité absolue.

Un code stable et sans conséquence peut attendre indéfiniment.

La grille de décision

	Rarement modifié	Souvent modifié
Conséquence forte	Tester — un jeu de cas de référence	Tester en premier — et densément
Conséquence faible	Laisser tel quel	Tester au fil de l'eau, quand on y touche

Exemples typiques dans une librairie métier :

- *fort + volatile* → règles de gestion, grilles tarifaires, formules d'indice ;
- *fort + stable* → conversions d'unités, parsing d'un format figé ;
- *faible + volatile* → messages de log, mise en forme d'un affichage ;
- *faible + stable* → accesseurs, constantes, code de confort.

Ce qui mérite un test en priorité

Cible	Pourquoi
Fonctions critiques métier	Ce sont elles qui produisent les chiffres publiés
Calculs sensibles	Une erreur y est silencieuse par nature
Transformations de données	Un décalage de colonne ne lève aucune exception
Formats d'entrée / sortie	C'est le contrat avec les autres projets
Règles de validation	Elles décident ce qui entre dans le système
Bugs déjà rencontrés	Ils sont déjà revenus une fois ; ils reviendront
Code utilisé par plusieurs packages	Une régression y coûte N fois

 **En pratique** : commencez par la dernière ligne du tableau. Le code partagé par plusieurs projets est celui dont la régression est la plus chère, et c'est souvent celui que personne n'ose modifier.

Le bug déjà rencontré : le test le plus rentable

Un bug qui a été corrigé une fois a déjà prouvé trois choses :

1. le cas se produit **réellement** en production ;
2. le code ne le gèrait pas — donc rien ne l'y empêche de revenir ;
3. quelqu'un a déjà passé du temps à le diagnostiquer.

Le test correspondant coûte cinq minutes, puisque le cas reproductible est déjà identifié. C'est le meilleur rapport effort / valeur de toute la suite.

Une politique d'équipe simple, applicable dès demain :

aucune correction de bug n'est fusionnée sans le test qui échouait avant.

Nous verrons comment le formuler au chapitre 6 de cette journée.

Ce qu'il est peu rentable de tester

À éviter	Pourquoi
Accesseurs simples	<code>return self._x</code> — tester le langage, pas votre code
Détails internes instables	Le test cassera au premier refactoring légitime
Duplication de tests	Trois tests pour la même règle : trois à maintenir, un seul signal
Tests couplés à l'implémentation	Ils bloquent le refactoring (chapitre 1)
Le code des bibliothèques tierces	<code>pandas</code> et <code>numpy</code> ont leurs propres suites
Le rendu exact d'un log	Change souvent, ne casse jamais rien de fonctionnel

⚠ **Piège** : un test inutile n'est pas neutre. Il coûte du temps d'exécution, du temps de maintenance, et il **dilue le signal** — dans une suite de 800 tests dont 300 sont sans valeur, plus personne ne lit les échecs.

La couverture est un indicateur, pas un objectif

La couverture mesure **quelles lignes ont été exécutées**. Elle ne mesure ni la pertinence des assertions, ni les cas absents.

```
def diviser(a, b):  
    return a / b  
  
def test_diviser():  
    diviser(10, 2)           # 100 % de couverture... et aucune assertion
```

Ce test couvre la fonction intégralement et ne vérifie rien — ni le résultat, ni le comportement pour `b = 0`.

Ce que la couverture sait faire, en revanche, et qui est très utile :

Repérer le code que vous n'avez jamais exécuté. Une fonction métier à 0 % est une information exploitable. La différence entre 82 % et 85 %, non.

Nous la brancherons dans la pipeline au Jour 5, avec un seuil — mais un seuil choisi comme garde-fou, pas comme trophée.

À retenir

1. Priorisez sur deux axes : la **conséquence** d'une erreur et la **fréquence** de modification. Le croisement des deux donne l'ordre de travail.
2. Le test le plus rentable de tous est celui qui reproduit un bug déjà rencontré : le cas est connu, l'écriture prend cinq minutes.
3. Un test sans valeur n'est pas gratuit : il dilue le signal. Mieux vaut 60 tests qu'on lit que 600 qu'on ignore.

3. Organisation d'un projet de tests

Où poser les fichiers, et pourquoi ça compte

Structure recommandée d'un package Python

```
mesures/
├── pyproject.toml
├── src/
│   └── mesures/
│       ├── __init__.py
│       ├── modele.py        # objets métier
│       ├── calculs.py       # fonctions pures
│       ├── lecture.py       # I/O
│       └── sources/
│           ├── __init__.py
│           └── api.py
└── tests/
    ├── conftest.py          # fixtures partagées
    ├── donnees/             # jeux de test versionnés
    │   └── releve_minimal.csv
    ├── unitaires/
    │   ├── test_calculs.py
    │   └── test_modele.py
    └── integration/
        └── test_api.py
```

Pourquoi `src/` — ce n'est pas cosmétique

Sans `src/`, le package est **au même niveau** que le dossier de tests. Python ajoute le répertoire courant à `sys.path`, donc `import mesures` trouve le code source local, **même s'il n'est pas installé**.

Conséquence : vous testez vos fichiers, pas votre package.

Ce qui casse en silence sans <code>src/</code>	Détecté avec <code>src/</code>
Un fichier oublié dans le paquet distribué	✓ à l'import
Un <code>package_data</code> manquant (fichiers de données)	✓ à l'exécution
Un sous-package sans <code>__init__.py</code>	✓ à l'import

Avec `src/`, la seule façon d'importer `mesures` est de l'installer :

```
uv pip install -e .      # ou : pip install -e .
```

Vos tests s'exécutent alors contre le package tel que vos consommateurs le recevront.

Le dossier `tests/` en miroir du package

La règle la plus économique : **un fichier de test par module**, au même emplacement relatif.

Module	Test
<code>src/mesures/calculs.py</code>	<code>tests/unitaires/test_calculs.py</code>
<code>src/mesures/sources/api.py</code>	<code>tests/integration/test_api.py</code>

Vous savez ainsi immédiatement :

- où ajouter un test quand vous modifiez un module ;
- quel module n'a **aucun** test — le fichier correspondant est absent.

 **Piège** : ne mettez pas de `__init__.py` dans `tests/`. Le dossier de tests n'est pas un package importable, et pytest s'en passe très bien.

`conftest.py` : le fichier que pytest découvre tout seul

`conftest.py` contient les fixtures et la configuration **partagées par tous les tests du dossier et de ses sous-dossiers**. Vous ne l'importez jamais.

```
tests/
├── conftest.py          # fixtures pour tous les tests
├── unitaires/
│   └── conftest.py      # fixtures propres aux tests unitaires
└── integration/
    └── conftest.py      # ex. : une base temporaire
```

```
# tests/conftest.py
import pytest
from pathlib import Path

@pytest.fixture
def dossier_donnees() -> Path:
    """Racine des jeux de données de test."""
    return Path(__file__).parent / "donnees"
```

Toute fonction de test peut désormais recevoir `dossier_donnees` en paramètre.

Conventions de nommage

pytest découvre les tests par convention. Il n'y a rien à déclarer.

Élément	Convention	Exemple
Fichier	<code>test_*.py</code>	<code>test_calculs.py</code>
Fonction	<code>test_*</code>	<code>test_moyenne_sur_serie_vide</code>
Classe (optionnelle)	<code>Test*</code> , sans <code>__init__</code>	<code>TestGrilleTarifaire</code>

Pour les **noms de tests**, une seule règle utile :

Le nom décrit le **comportement attendu**, pas l'appel technique.

✗ `test_calculer_indice_1`

✗ `test_calculer_indice_ko`

✗ `test_bug_4231`

✓ `test_indice_plafonne_a_100`

✓ `test_indice_none_sans_mesure`

✓ `test_seuil_inclusif_a_la_borne`

Quand un test rougit en CI, son nom est souvent la seule chose que le relecteur lira.

Arrange / Act / Assert

Trois blocs, dans cet ordre, séparés par une ligne vide. C'est tout.

```
def test_avancement_complet_vaut_100():  
    # Arrange – préparer les données  
    taches = [Tache("A", terminee=True), Tache("B", terminee=True)]  
  
    # Act – une seule action, celle qu'on teste  
    avancement = calculer_avancement(taches)  
  
    # Assert – vérifier le résultat observable  
    assert avancement == 100.0
```

Bénéfices concrets :

- si le bloc **Act** contient plus d'une ligne, le test vérifie plusieurs choses ;
- si le bloc **Arrange** fait 30 lignes, le code testé a trop de dépendances ;
- le lecteur trouve l'entrée et la sortie attendue sans lire le corps.

Les commentaires sont facultatifs — la structure en trois paragraphes suffit.

Séparer les tests unitaires des tests d'intégration

Deux moyens, complémentaires : les **dossiers** et les **marqueurs**.

```
# pyproject.toml
[tool.pytest.ini_options]
testpaths = ["tests"]
markers = [
    "integration: nécessite une ressource externe (base, réseau, fichier volumineux)",
    "lent: dépasse une seconde",
]
```

```
@pytest.mark.integration
def test_lecture_depuis_la_base(base_temporaire):
    ...
```

L'intérêt apparaît au quotidien :

```
pytest -m "not integration"  # boucle de développement : quelques secondes
pytest                      # avant de pousser : tout
```

C'est ce découpage que nous exploiterons au Jour 5 pour tenir des pipelines rapides.

Les données de test

Type de donnée	Où la mettre
Deux ou trois valeurs	en dur dans le test — c'est le plus lisible
Un objet métier récurrent	une fixture dans <code>conftest.py</code>
Un fichier d'entrée réaliste	<code>tests/donnees/</code> , versionné , réduit au minimum
Un fichier produit par le test	<code>tmp_path</code> — jamais le dossier du projet

 **En pratique** : un fichier de test de 200 Mo extrait de la production est presque toujours une erreur. Réduisez-le à 5 lignes qui contiennent le cas intéressant, et documentez en commentaire ce qu'elles représentent.

Un jeu de données de test doit tenir dans un dépôt Git **et** dans la tête du relecteur.

À retenir

1. `src/` + installation en mode éditable : vos tests s'exécutent contre le package tel qu'il sera livré, pas contre vos fichiers locaux.
2. `tests/` en miroir du package, sans `__init__.py`, avec un `conftest.py` pour les fixtures partagées.
3. Arrange / Act / Assert. Si l'un des trois blocs devient long, c'est le **code testé** qui a un problème, pas le test.

4. pytest en pratique

Les fonctionnalités qui servent tous les jours

Le premier test

Aucune classe à hériter, aucune méthode `setUp`, aucun `assertEqual`.

```
# tests/unitaires/test_calculs.py
from mesures.calculs import moyenne

def test_moyenne_de_trois_valeurs():
    assert moyenne([10.0, 20.0, 30.0]) == 20.0
```

```
$ pytest
tests/unitaires/test_calculs.py .
1 passed in 0.01s [100%]
```

Une fonction, un `assert`. pytest découvre le reste par convention :
fichiers `test_*.py`, fonctions `test_*`.

L'introspection des assertions

Le simple `assert` de Python suffit, parce que pytest **réécrit** l'expression pour vous montrer les valeurs intermédiaires en cas d'échec.

```
def test_moyenne_de_trois_valeurs():
>     assert moyenne([10.0, 20.0, 30.0]) == 20.0
E       assert 21.5 == 20.0
E       +   where 21.5 = moyenne([10.0, 20.0, 30.0])
```

Sur des structures, la comparaison est détaillée automatiquement :

```
E       AssertionError: assert {'haute': 12, 'basse': 8} == {'haute': 12, 'basse': 9}
E         Common items: {'haute': 12}
E         Differing items: {'basse': 8} != {'basse': 9}
```

 **Piège :** `assert resultat_est_valide` où la variable est un booléen déjà calculé n'affiche que `assert False`. Asserterez sur des **valeurs**.

Lancer pytest : les options qui servent

Commande	Usage
<code>pytest</code>	tout
<code>pytest tests/unitaires</code>	un dossier
<code>pytest tests/test_calculs.py::test_moyenne</code>	un test précis
<code>pytest -k "moyenne and not vide"</code>	par motif sur le nom
<code>pytest -m "not integration"</code>	par marqueur
<code>pytest -x</code>	s'arrêter au premier échec
<code>pytest --lf</code>	rejouer uniquement les derniers échecs
<code>pytest --ff</code>	rejouer les échecs d'abord , puis le reste
<code>pytest -q / -v</code>	sortie compacte / détaillée

 **En pratique** : la boucle de travail réelle est

`pytest --lf -x -q` — on ne rejoue que ce qui était rouge, et on s'arrête au premier.

Comparer des flottants : `pytest.approx`

`0.1 + 0.2 == 0.3` est faux en virgule flottante. Sur du code scientifique, c'est la première source de tests faussement rouges.

Fragile

```
assert moyenne([0.1, 0.2]) == 0.15000000000000002
```

Correct

```
assert moyenne([0.1, 0.2]) == pytest.approx(0.15)
```

Tolérance explicite

```
# tolérance relative de 1 %
assert x == pytest.approx(12.4, rel=1e-2)

# tolérance absolue
assert x == pytest.approx(0.0, abs=1e-9)
```

Sur des structures

```
assert resultats == pytest.approx([1.0, 2.0])
assert stats == pytest.approx({"moy": 1.0})
```

`approx` accepte listes, tuples, dictionnaires et tableaux `numpy`.

Tester les exceptions

```
def test_serie_vide_est_refusee():  
    with pytest.raises(SerieVide):  
        moyenne([])
```

Le test échoue si **aucune** exception n'est levée, ou si c'est une autre.

Vérifier aussi le **message**, avec `match` (une expression régulière) :

```
def test_le_message_nomme_le_parametre_fautif():  
    with pytest.raises(ParametreInvalide, match=r"taille_fenetre = 511"):  
        valider({"taille_fenetre": 511})
```

Inspecter l'exception elle-même :

```
def test_la_cause_est_conservee():  
    with pytest.raises(ParametresIllisibles) as info:  
        charger(chemin_casse)  
    assert isinstance(info.value.__cause__, json.JSONDecodeError)
```


pytest.raises — les erreurs classiques

Bloc trop large ✗

```
with pytest.raises(SerieVide):
    donnees = preparer() # peut lever
    moyenne(donnees)    # ce qu'on teste
```

Si `preparer()` lève `SerieVide`, le test passe pour la mauvaise raison.

Ciblé ✓

```
donnees = preparer()
with pytest.raises(SerieVide):
    moyenne(donnees)
```

Trop générique ✗

```
with pytest.raises(Exception):
    moyenne([])
```

Un `TypeError` accidentel ferait passer le test.

Précis ✓

```
with pytest.raises(SerieVide, match="vide"):
    moyenne([])
```

⚠ **Piège :** `pytest.raises(ValueError)` attrape aussi ses sous-classes.

Pour vos exceptions métier, c'est un avantage — à condition que la hiérarchie soit voulue.

Les fixtures : préparer sans dupliquer

Une fixture est une fonction décorée dont le **nom** devient un paramètre de test.
pytest la résout automatiquement.

```
@pytest.fixture
def grille() -> GrilleTarifaire:
    """Grille de référence utilisée par la plupart des tests."""
    return GrilleTarifaire(prix_hp=0.25, prix_hc=0.20, abonnement=12.60)

def test_facture_heures_pleines(grille):
    assert grille.montant(hp=100, hc=0) == pytest.approx(25.0)

def test_facture_heures_creuses(grille):
    assert grille.montant(hp=0, hc=100) == pytest.approx(20.0)
```

Chaque test reçoit une instance **fraîche** : pas d'état partagé entre tests,
donc pas de dépendance à l'ordre d'exécution.

Fixtures : nettoyage avec `yield`

Ce qui est écrit après le `yield` s'exécute à la fin du test, même s'il échoue.

```
@pytest.fixture
def base_temporaire(tmp_path):
    connexion = sqlite3.connect(tmp_path / "test.db")
    connexion.execute("CREATE TABLE mesures (id INTEGER, valeur REAL)")
    yield connexion          # ← le test s'exécute ici
    connexion.close()        # ← toujours exécuté
```

C'est l'équivalent de `setUp` / `tearDown`, en une seule fonction lisible.

 **En pratique** : pour un fichier ou un dossier temporaire, n'écrivez pas de fixture — `tmp_path` fait déjà le travail, et le nettoyage aussi.

Pourquoi le nettoyage s'exécute même en cas d'échec

La fixture `yield` est un **gestionnaire de contexte** : c'est le mécanisme de `with`, que vous utilisez déjà pour les fichiers.

Ce que fait `with`

```
with open(chemin) as f:    # __enter__
    traiter(f)             # __exit__ - même si
                          # traiter() a levé
```

Le fichier est fermé quoi qu'il arrive :
retour normal, exception, `return`.

Le vôtre, en trois lignes

```
from contextlib import contextmanager
@contextmanager
def connexion_temporaire(dsn):
    conn = connect(dsn)
    try:
        yield conn          # ← le bloc with
    finally:
        conn.close()        # ← toujours exécuté
```

 **En pratique** : une fixture `yield` et un `@contextmanager` s'écrivent pareil. Écrivez le second dans votre librairie — vos consommateurs l'utiliseront avec `with`, et vos tests l'utiliseront comme fixture. Une seule implémentation.

Fixtures : les portées (**scope**)

Portée	Créée	Usage typique
<code>function</code> (défaut)	à chaque test	tout ce qui est modifiable
<code>class</code>	une fois par classe	rare
<code>module</code>	une fois par fichier	une connexion lente en lecture seule
<code>session</code>	une fois pour tout	un serveur de test, une image Docker

```
@pytest.fixture(scope="session")
def schema_reference() -> dict:
    """Chargé une seule fois : lecture seule, coûteux à construire."""
    return json.loads(Path("tests/donnees/schema.json").read_text())
```

⚠ **Piège** : une fixture de portée large qui retourne un objet **mutable** réintroduit exactement le problème d'état partagé que les fixtures évitent.
Portée large ⇒ objet en lecture seule.

Fixtures partagées et composées

Une fixture placée dans `conftest.py` est disponible dans tout le dossier, sans import.

```
# tests/conftest.py
@pytest.fixture
def dossier_donnees() -> Path:
    return Path(__file__).parent / "donnees"

@pytest.fixture
def releve_minimal(dossier_donnees) -> list[Releve]:
    """Trois lignes couvrant une heure pleine, une creuse et une borne."""
    return lire_releve(dossier_donnees / "releve_minimal.csv")
```

Une fixture peut en demander d'autres : pytest construit le graphe et n'instancie que ce qui est nécessaire au test en cours.

```
pytest --fixtures      # liste toutes les fixtures visibles, avec leur docstring
```

La paramétrisation

Le même test, sur plusieurs jeux de données. C'est le mécanisme le plus rentable de pytest sur du code de calcul.

Sans ✗

```
def test_hc_22h():  
    assert est_heure_creuse(22)  
  
def test_hc_23h():  
    assert est_heure_creuse(23)  
  
def test_hc_3h():  
    assert est_heure_creuse(3)
```

Avec ✓

```
@pytest.mark.parametrize(  
    "heure", [22, 23, 0, 3, 5]  
)  
def test_heures_creuses(heure):  
    assert est_heure_creuse(heure)
```

Chaque valeur produit un **test distinct** : cinq lignes dans le rapport, cinq échecs possibles indépendants. Ce n'est pas une boucle — une boucle s'arrête au premier échec et cache les suivants.

Paramétrisation : plusieurs arguments

```
@pytest.mark.parametrize(
    ("hp", "hc", "attendu"),
    [
        (100.0, 0.0, 25.16),
        (0.0, 100.0, 20.32),
        (0.0, 0.0, 0.00),      # cas limite : aucune consommation
        (0.5, 0.5, 0.22),      # arrondi au centime
    ],
)
def test_montant_de_consommation(hp, hc, attendu):
    assert montant(hp, hc) == pytest.approx(attendu, abs=0.005)
```

Le tableau se lit comme une **spécification** : chaque ligne est un cas métier, avec son résultat de référence.

 **En pratique** : c'est le format idéal pour faire relire une règle de calcul par un expert métier qui ne lit pas Python. Il lit le tableau.

Paramétrisation : noms de cas lisibles

Par défaut, pytest génère `test_montant[100.0-0.0-25.16]`. Sur des données complexes, cela devient illisible.

```
@pytest.mark.parametrize(
    ("entree", "attendu"),
    [
        pytest.param("12,4", 12.4, id="virgule_decimale"),
        pytest.param("12.4", 12.4, id="point_decimal"),
        pytest.param(" 12.4 ", 12.4, id="espaces_autour"),
        pytest.param("1 2 4", None, id="espaces_internes_refuses"),
    ],
)
def test_parser_valeur(entree, attendu):
    ...
```

```
tests/test_parsing.py::test_parser_valeur[virgule_decimale] PASSED
tests/test_parsing.py::test_parser_valeur[espaces_internes_refuses] FAILED
```

L'identifiant apparaît dans le rapport CI : il doit dire **quel cas métier** a cassé.

Fichiers temporaires : `tmp_path`

`tmp_path` est une fixture native qui fournit un `Path` vers un dossier vide, unique au test, supprimé ensuite.

```
def test_ecriture_du_rapport(tmp_path):
    destination = tmp_path / "rapport.csv"

    ecrire_rapport(destination, [Mesure(8, 12.4)])

    lignes = destination.read_text().splitlines()
    assert lignes[0] == "heure,valeur"
    assert lignes[1] == "8,12.4"
```

Fixture	Contenu
<code>tmp_path</code>	un <code>Path</code> vers un dossier vide, par test
<code>tmp_path_factory</code>	une fabrique, pour une fixture de portée <code>session</code>

 **Piège** : n'écrivez **jamais** dans le dossier du projet depuis un test.
Les tests deviennent dépendants de l'ordre, et polluent le dépôt Git.

Marqueurs

Trois usages, du plus ponctuel au plus structurant :

```
@pytest.mark.skip(reason="fonctionnalité retirée en v2")
def test_ancien_format(): ...

@pytest.mark.skipif(sys.platform == "win32", reason="chemins POSIX")
def test_permissions(): ...

@pytest.mark.xfail(reason="bug #4231, corrigé en v1.4", strict=True)
def test_arrondi_a_la_borne(): ...
```

`xfail` avec `strict=True` est particulièrement utile : le test **doit** échouer.

S'il se met à passer, la suite rougit — et vous savez que le bug est corrigé.

```
@pytest.mark.integration          # marqueur maison, déclaré dans pyproject.toml
def test_lecture_reelle(): ...
```

Déclarez toujours vos marqueurs maison dans `[tool.pytest.ini_options]`,
sinon une faute de frappe passe inaperçue.

Bonnes pratiques d'écriture

Règle	Pourquoi
Un test = un comportement	Un échec désigne une cause, pas trois
Pas de logique dans un test	Un <code>if</code> dans un test est un bug potentiel non testé
Pas de dépendance entre tests	Chaque test doit passer seul et dans le désordre
Nommer par le comportement	Le nom est ce qu'on lit en CI
Une assertion principale	Plusieurs <code>assert</code> liés au même comportement : oui. Trois comportements : non.
Des données minimales	Trois lignes qui montrent le cas, pas 200 extraites de la production

Vérification utile de temps en temps :

```
pytest --lf -x -q          # la boucle de travail : rejouer ce qui était rouge
pytest tests/unitaires -q  # la suite unitaire doit tenir en secondes
```

Si votre suite unitaire dépasse **10 secondes**, elle contient des tests d'intégration mal étiquetés.

À retenir

1. `assert` simple, `pytest.approx` pour les flottants, `pytest.raises` avec `match` pour les erreurs. Ces trois-là couvrent 80 % des besoins.
2. Les **fixtures** éliminent la duplication de préparation ; la **paramétrisation** transforme un tableau de cas métier en tests indépendants.
3. `tmp_path` pour tout ce qui touche au disque. Un test qui écrit dans le projet est un test qui cassera les autres.

5. Premiers cycles TDD avec pytest

Le cycle, appliqué pas à pas sur une fonction métier

Le cas de travail

Une règle de gestion réelle, énoncée par le métier :

« Le tarif horaire d'une intervention est de 65 €. Au-delà de 8 heures dans la même journée, les heures supplémentaires sont majorées de 25 %. À partir de la 11^e heure, la majoration passe à 50 %. »

Ce que nous **ne** faisons pas : ouvrir un fichier et commencer par

```
def calculer_tarif(...):
```

Ce que nous faisons : écrire le premier test, et laisser les questions arriver.

Combien de temps faut-il pour se rendre compte, en codant directement, que l'énoncé ne dit rien de ce qui se passe à **exactement** 8 heures ?

Red — décrire le comportement le plus simple

```
# tests/test_tarification.py
from facturation.tarifs import cout_intervention

def test_journee_standard():
    assert cout_intervention(heures=6) == 390.0
```

Trois décisions viennent d'être prises, et elles sont **les bonnes questions** :

Décision	Alternative écartée
Le nom : <code>cout_intervention</code>	<code>calculer</code> , <code>tarif</code> , <code>process</code>
Le paramètre : <code>heures</code> , nommé	un positionnel, un objet <code>Intervention</code>
Le retour : un <code>float</code> en euros	un dict, un objet <code>Facture</code>

Nous venons de concevoir une interface. Aucune implémentation n'existe encore.

Red — voir l'échec, et vérifier sa raison

```
$ pytest tests/test_tarification.py
ImportError: cannot import name 'cout_intervention' from 'facturation.tarifs'
```

C'est un rouge **valide** : il dit exactement ce qui manque.

```
# facturation/tarifs.py
def cout_intervention(heures: float) -> float:
    raise NotImplementedError
```

```
$ pytest
E   NotImplementedError
```

Encore mieux : le test atteint désormais la fonction. Nous savons que le test **sait échouer**.

 **Piège** : sauter cette étape est l'erreur la plus fréquente. Un test jamais vu rouge peut ne rien vérifier du tout — et vous ne le saurez jamais.

Green — le minimum qui fait passer

```
TAUX_HORAIRE = 65.0  
  
def cout_intervention(heures: float) -> float:  
    return heures * TAUX_HORAIRE
```

```
$ pytest  
1 passed in 0.01s
```

C'est volontairement incomplet : les majorations ne sont pas gérées. Et c'est **correct à ce stade** — aucun test ne les demande encore.

 **En pratique** : écrire ici la règle complète des majorations, c'est écrire du code que rien ne vérifie. C'est précisément ce que le TDD cherche à éviter : du code dont personne ne sait s'il est juste.

Red — le deuxième cas fait apparaître la règle

```
def test_heures_supplementaires_majorees_de_25_pourcent():  
    # 8 h à 65 € + 2 h à 81,25 €  
    assert cout_intervention(heures=10) == 682.5
```

```
$ pytest  
E   assert 650.0 == 682.5
```

Rouge, pour la bonne raison. On implémente :

```
SEUIL_MAJORATION_25 = 8  
  
def cout_intervention(heures: float) -> float:  
    normales = min(heures, SEUIL_MAJORATION_25)  
    sup = max(0.0, heures - SEUIL_MAJORATION_25)  
    return normales * TAUX_HORAIRE + sup * TAUX_HORAIRE * 1.25
```

Red — la question que le métier n'avait pas tranchée

```
def test_a_exactement_huit_heures_aucune_majoration():  
    """Le seuil est exclusif : la 8e heure n'est pas majorée."""  
    assert cout_intervention(heures=8) == 520.0
```

Cette question **n'était pas dans l'énoncé**. Elle est apparue en écrivant les tests, avant que le code parte en production.

Trois cas de ce type sur cette seule règle :

Question	Réponse à obtenir du métier
À exactement 8 h, majoration ?	Non — le seuil est exclusif
À exactement 11 h, quel taux ?	25 %, la 11 ^e heure n'est pas encore à 50 %
Heures négatives ou nulles ?	Erreur métier explicite

C'est cela, « le TDD comme outil d'analyse ». Le bénéfice principal n'est pas le test : c'est la **conversation** qu'il déclenche.

Refactor — la phase qu'on saute

Après le troisième palier, la fonction accumule les `min` et les `max` :

Avant

```
def cout_intervention(heures):  
    n = min(heures, 8)  
    s25 = max(0.0, min(heures, 11) - 8)  
    s50 = max(0.0, heures - 11)  
    return (n * 65.0 + s25 * 65.0 * 1.25  
            + s50 * 65.0 * 1.5)
```

Après

```
PALIER = [(8, 1.00), (11, 1.25), (float("inf"), 1.50)]  
  
def cout_intervention(heures: float) -> float:  
    total, precedent = 0.0, 0  
    for borne, coef in PALIER:  
        tranche = max(0.0, min(heures, borne) - precedent)  
        total += tranche * TAUX_HORAIRE * coef  
        precedent = borne  
    return total
```

Les tests ne changent pas d'une ligne — c'est ce qui prouve que le comportement est préservé. Et un quatrième palier ne coûte plus qu'une entrée dans `PALIER`.

Ajouter les cas limites, un par un

Une fois la règle nominale en place, le cycle continue sur les bords :

```
@pytest.mark.parametrize(
    ("heures", "attendu"),
    [
        (0, 0.0),      # aucune intervention
        (0.5, 32.5),    # demi-heure
        (8, 520.0),     # borne du premier palier
        (11, 763.75),   # borne du second palier
        (24, 2031.25),  # journée maximale
    ],
)
def test_paliers(heures, attendu):
    assert cout_intervention(heures) == pytest.approx(attendu)

def test_duree_negative_refusee():
    with pytest.raises(DureeInvalide):
        cout_intervention(heures=-1)
```

Chaque ligne ajoutée a suivi le même cycle : rouge, vert, refactor éventuel.

Ce que le TDD a produit, au-delà des tests

Livrable	Comment il est apparu
Une interface nommée et typée	Écrite dans le premier test, avant tout code
Trois ambiguïtés métier levées	Découvertes en écrivant les cas limites
Une table de paliers extensible	Sortie de la phase Refactor
Une exception métier explicite	Exigée par un test de cas invalide
8 tests qui documentent la règle	Sous-produit

La dernière ligne est celle dont on parle toujours. Les quatre premières sont celles qui font gagner du temps.

TDD strict ou pragmatique : la règle de décision

Question	Si oui → TDD strict	Si non → approche souple
Sais-je énoncer le résultat attendu avant de coder ?	✓	écrire un prototype, puis tester ce qui survit
Le comportement est-il une règle métier ?	✓	tester les propriétés (bornes, monotonie)
Le code existe-t-il déjà ?	—	tests de caractérisation d'abord (Jour 4)
Suis-je en train d'explorer une bibliothèque ?	—	un test d'intégration pour apprendre

Une variante intermédiaire, très utile en pratique :

« **Test-first sur l'interface, code-first sur l'algorithme** » — écrivez le test qui fixe la signature et le cas nominal, puis implémentez librement, et revenez ajouter les cas limites en TDD.

À retenir

1. Le premier test conçoit l'**interface**. C'est le moment le moins cher pour changer d'avis sur un nom, un paramètre ou un type de retour.
2. Voir le rouge n'est pas une formalité : c'est la seule preuve que votre test sait détecter l'absence du comportement.
3. Les cas limites écrits en TDD révèlent les **ambiguïtés du besoin** avant la production. C'est le bénéfice principal, et il n'est pas technique.

6. Tests de non-régression

Transformer un incident en garantie permanente

Le bug est une spécification gratuite

Quand un bug remonte du terrain, vous avez déjà en main tout ce qui coûte cher à produire :

- un **cas d'entrée réel** qui déclenche le problème ;
- un **résultat observé**, faux ;
- un **résultat attendu**, validé par quelqu'un.

C'est exactement le contenu d'un test. Il ne reste qu'à l'écrire — et le transformer en garantie permanente au lieu d'une correction ponctuelle.

 **Piège** : corriger sans écrire le test, c'est accepter que le même bug revienne au prochain refactoring — sans que personne ne s'en aperçoive.

La séquence, dans l'ordre

- | | |
|-----------------------|--|
| 1. Reproduire
↓ | Écrire le test qui échoue AVANT de corriger.
S'il passe du premier coup, vous n'avez pas reproduit le bug – vous en avez inventé un autre. |
| 2. Voir le rouge
↓ | La sortie du test doit correspondre à ce que le terrain a observé. |
| 3. Corriger
↓ | Le minimum pour faire passer le test. |
| 4. Voir le vert
↓ | La correction traite bien ce cas-là. |
| 5. Élargir | Le bug était-il un cas particulier d'une famille ? Ajouter les cas voisins. |

L'ordre compte : écrire le test **après** la correction ne prouve rien, puisque vous n'aurez jamais vu ce test échouer.

Un exemple complet

Le support signale : « *les tâches dont le titre contient une apostrophe n'apparaissent pas dans l'export* ».

```
def test_titre_avec_apostrophe_est_conserve():
    """Bug #4231 – les titres type « Relire l'annexe » disparaissaient de l'export."""
    tache = Tache(id=1, titre="Relire l'annexe")

    lignes = exporter([tache])

    assert "Relire l'annexe" in lignes[0]
```

Trois éléments à ne pas négliger :

Élément	Rôle
Le numéro de ticket en docstring	relie le test à l'historique de l'incident
Le cas réel (Relire l'annexe)	pas "a'b" — le vrai cas est plus parlant
Le nom du test	décrit le comportement, pas le bug

Élargir : le bug appartient-il à une famille ?

Un bug isolé est rare. Une apostrophe qui casse le formatage suggère une famille entière de caractères mal gérés.

```
@pytest.mark.parametrize(
    "titre",
    [
        "Relire l'annexe",      # apostrophe – le bug signalé
        "Mettre-à-jour",      # tiret
        "Écrire le bilan",     # accent en initiale
        'Lire "Python avancé"', # guillemets
        "Bilan Q3",           # double espace
    ],
)
def test_titres_speciaux_sont_conservees(titre):
    assert titre in exporter([Tache(id=1, titre=titre)])[0]
```

 **En pratique** : cinq minutes de plus, et vous corrigez quatre bugs qui n'ont pas encore été signalés. C'est le meilleur retour sur investissement de toute la journée.

Tests de caractérisation : capturer l'existant

Sur du code legacy, vous n'avez pas de spécification. Vous avez un **comportement observé**, et il faut le figer avant d'y toucher.

```
def test_caracterisation_calcul_indice_v1():
    """Fige le comportement actuel – NON validé par le métier.

    Objectif : détecter tout changement pendant le refactoring.
    À revoir une fois la règle réellement spécifiée.
    """
    assert calculer_indice(RELEVANCE_REFERENCE) == pytest.approx(72.4)
```

La différence avec un test classique est **d'intention** :

	Test unitaire	Test de caractérisation
Affirme	ce qui doit être	ce qui est
Source	la spécification	l'exécution du code actuel
Si le code est faux	le test est rouge	le test fige le bug
Durée de vie	permanente	jusqu'à spécification du besoin

Écrire un test de caractérisation quand on ignore la réponse

La technique est délibérément mécanique :

```
def test_caracterisation():  
    assert calculer_indice(RELEVE_REFERENCE) == 0    # valeur arbitraire
```

```
E    assert 72.4 == 0
```

Le message d'erreur **vous donne** la valeur actuelle. Vous la recopiez dans le test, qui devient vert. Le filet de sécurité est posé.

 **Piège** : ne présentez jamais un test de caractérisation comme une validation métier. Il documente un comportement, y compris ses bugs. La docstring doit le dire explicitement, sinon quelqu'un « corrigera » un jour un test qui protégeait en réalité un bug connu.

Nous approfondirons cette technique au Jour 4, sur du code de 300 lignes.

Construire un filet avant de refactoriser

Avant de toucher à un composant existant, la question n'est pas
« *ai-je assez de tests ?* » mais :

Quels changements de comportement dois-je être certain de détecter ?

Niveau du filet	Ce qu'il détecte	Coût
Un test de bout en bout sur un jeu réel	tout écart global	faible
+ des tests de caractérisation par fonction	l'endroit de l'écart	moyen
+ des tests unitaires sur les règles métier	l'écart et sa cause	élevé

En pratique, on construit **de haut en bas** : d'abord un test global qui rougit si quoi que ce soit change, puis on descend en précision dans les zones qu'on s'apprête à modifier.

À retenir

1. Un bug corrigé sans test est un bug qui reviendra. Écrivez le test **avant** la correction, pour l'avoir vu rouge.
2. Élargissez systématiquement le cas signalé à sa famille : c'est là que se cachent les bugs pas encore remontés.
3. Un test de caractérisation fige le comportement **actuel**, bugs compris. Sa docstring doit le dire, sous peine d'induire en erreur le prochain lecteur.

Travaux pratiques du Jour 2

Un peu de pytest, puis les design patterns par le problème

Mise en place

```
cd TPs_reels/tp2
python3.12 -m venv .venv          # un Python 3.10 ou plus, nommé explicitement
source .venv/bin/activate        # Windows : .venv\Scripts\activate
python -m pip install pytest
cd 01-premiers-tests-geometrie/depart
python -m pytest -v
```

Chaque TP suit la même structure :

README.md	contexte, étapes, critères de réussite, discussion
depart/	le code sur lequel vous travaillez, avec son <code>main.py</code>
corrige/	une solution possible, commentée aux endroits de décision

Les TP sont **indépendants** : chacun a son propre petit programme, repris de ce que nous avons codé hier.

La trame des TP patterns

Les TP 03 à 09 suivent tous le même chemin, dans le même ordre :

Étape	Ce qui se passe
1. Un code qui marche	<code>python main.py</code> tourne. Rien ne semble cassé.
2. Le test qui fait mal	on essaie de l'écrire, et c'est impossible, ou faux, ou trop long.
3. Le problème nommé	couplage, responsabilité, duplication, état global.
4. Le pattern	la réponse à ce problème précis, pas à un autre.
5. Le test devient simple	quelques lignes, sans base, sans réseau, sans fichier.
6. Quand c'est de trop	le cas où le pattern coûterait plus qu'il ne rapporte.

Le pattern n'est jamais présenté avant le problème. Si vous ne ressentez pas la douleur de l'étape 2, l'étape 4 n'a pas de raison d'être.

Le programme

TP	Pattern	Le test qui fait mal	Durée
01	(pytest)	Rectangle, Cercle et Carre d'hier : paramétrisation, <code>approx</code> , le piège de <code>raises(Exception)</code> , le constructeur qui ne valide rien.	45 min
03	Dependency Injection	<code>TodoService</code> crée son <code>TodoDAO("todos.db")</code> : le test pollue le disque.	45 min
04	Repository	le faux DAO ne fait pas comme le vrai : tests verts, doublon en base.	60 min
06	Strategy	<code>lister(tri="titre")</code> en cascade de <code>if</code> : un tri de plus rouvre la classe.	45 min
07	Factory	<code>if type_ == "cercle"</code> recopié partout : le triangle inconnu disparaît en silence.	45 min
09	Template Method	rapport texte et HTML dupliqués : bug corrigé d'un seul côté.	45 min

En réserve : **02** fixtures et `tmp_path` , **05** Adapter avec un premier `mock.patch` , **08** Facade avec un journal espion.

Deux consignes pour la journée

🎯 **Voyez le rouge.** À chaque TP, écrivez le test, lancez-le, regardez-le échouer, puis seulement modifiez le code. Un test qui n'a jamais échoué ne prouve rien.

🎯 **Lancez** `python main.py` **avant de toucher au code.** Le programme de départ marche. C'est le test qui révèle le défaut de conception, pas le programme.

Et à la fin de chaque TP, la question du chapitre 5 :

Combien de tests étaient impossibles avant ce pattern, et sont possibles maintenant ? Si la réponse est zéro, le pattern était de trop.

Les corrigés sont dans `corrige/`, à consulter **après** avoir cherché.

Les décisions non évidentes y sont commentées : zéro accepté ou non, `Carre` égal à `Rectangle` ou non, qui convertit le booléen sqlite.

1. Comprendre les doubles de test

Cinq objets différents, souvent appelés « mock » par erreur

Un double de test remplace une dépendance réelle

Le mot « mock » sert couramment à désigner cinq objets distincts, dont les usages n'ont rien à voir. Les distinguer change la façon dont on écrit ses tests.

Double	Ce qu'il fait	On l'utilise pour
Dummy	rien — il occupe une place	satisfaire une signature
Stub	renvoie une réponse figée	fournir une entrée au test
Fake	une vraie implémentation, simplifiée	remplacer une infrastructure
Mock	enregistre les appels et les vérifie	vérifier qu'un appel a eu lieu
Spy	enregistre les appels, sans exiger	observer sans contraindre

La distinction utile tient en une question :
est-ce que je vérifie un résultat, ou une interaction ?

Dummy, Stub, Fake

Dummy — jamais utilisé

```
def test_creation_sans_notification():
    rapport = Rapport(notificateur=None)
    assert rapport.titre == "Janvier"
```

Stub — une réponse figée

```
class SourceFiguee:
    def lire(self, jour):
        return [Mesure(8, 12.4)]

def test_moyenne():
    analyse = Analyse(SourceFiguee())
    assert analyse.moyenne(JOUR) == 12.4
```

Fake — une vraie implémentation

```
class DepotEnMemoire:
    def __init__(self):
        self._par_id = {}

    def enregistrer(self, mesure):
        self._par_id[mesure.id] = mesure

    def par_id(self, id):
        return self._par_id.get(id)
```

Il fonctionne : on peut écrire, relire, compter. Il ne persiste simplement rien sur disque.

Mock et Spy

Mock — vérifie l'interaction

```
def test_alerte_envoyee():
    notif = Mock()

    rappeler(taches_en_retard, notif)

    notif.envoyer.assert_called_once_with(
        "3 tâches en retard"
    )
```

L'objet du test est l'appel,
pas une valeur de retour.

Un spy écrit à la main est souvent **plus lisible** qu'un `Mock`, et son message d'échec est meilleur : `assert 3 == 1` plutôt que `Expected 'envoyer' to be called once`.

Spy — observe sans exiger

```
class NotificateurEspion:
    def __init__(self):
        self.envoyes = []

    def envoyer(self, message):
        self.envoyes.append(message)

def test_un_seul_rappel_par_personne():
    espion = NotificateurEspion()
    rappeler(taches, espion)
    assert len(espion.envoyes) == 1
```

Vérifier un résultat ou vérifier une interaction

C'est la décision qui structure tout le reste.

Test d'état — préférable

```
def test_moyenne_horaire():  
    assert moyenne([10, 20]) == 15
```

Vérifie **ce que le code produit**.

Insensible à la façon dont il s'y prend.

Survit au refactoring.

Test d'interaction — parfois nécessaire

```
def test_le_cache_evite_un_second_appel():  
    source = Mock()  
    source.lire.return_value = []  
    cache = Cache(source)  
  
    cache.lire(JOUR)  
    cache.lire(JOUR)  
  
    assert source.lire.call_count == 1
```

Vérifie **comment** le code s'y prend.

Casse dès qu'on change la mécanique.

 **En pratique** : testez l'interaction seulement quand l'interaction **est** le comportement — un envoi de mail, une écriture en base, un appel facturé.

Quand un mock est justifié

Situation	Pourquoi le mock est le bon outil
L'effet est hors du système	Envoyer un mail, publier un message, appeler une API payante — il n'y a pas d'état à vérifier chez soi
L'appel doit ne pas avoir lieu	« en mode simulation, rien n'est écrit » se vérifie avec <code>assert_not_called</code>
La dépendance est lente ou instable	Un service distant qui répond en 3 s, ou pas
Il faut simuler une panne	Provoquer un timeout ou un HTTP 500 à la demande
Le nombre d'appels est la règle	Un cache, une limitation de débit, une reprise sur erreur

Dans les cinq cas, ce qu'on veut vérifier **est** l'interaction. Le mock n'est pas un contournement : c'est l'outil adapté.

Quand éviter un mock

Situation	À faire à la place
Tester un calcul pur	Rien — appelez la fonction
Remplacer un objet de valeur	Construisez le vrai objet, il est gratuit
Remplacer un dépôt de données	Écrivez un fake en mémoire , réutilisable
Contourner un constructeur trop lourd	Injectez la dépendance (Jour 1, chapitre 6)
Mocker une classe que vous possédez	Extrayez une interface étroite, et faites-en un fake

⚠ **Piège** : si vous devez mocker trois objets pour tester une fonction, le problème n'est pas le mocking. C'est que la fonction a trois dépendances qu'elle ne devrait pas avoir.

Le danger des tests sur-mockés

```
def test_generer_rapport():
    source = Mock()
    source.lire.return_value = [Mock(valeur=10), Mock(valeur=20)]
    formateur = Mock()
    formateur.formater.return_value = "... "
    ecrivain = Mock()

    generer(source, formateur, ecrivain)

    ecrivain.ecrire.assert_called_once()
```

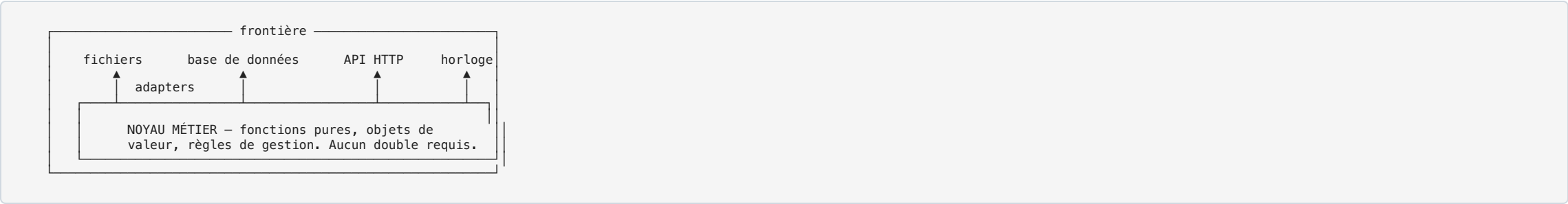
Ce test est vert. Que prouve-t-il ?

- que `generer` appelle `ecrire` — donc que le code appelle ce qu'il appelle ;
- **rien** sur la moyenne, le format produit, ni le contenu écrit ;
- il resterait vert si la moyenne devenait fausse.

Un test entièrement mocké teste **le graphe d'appels**, c'est-à-dire un miroir du code de production. Il se casse au moindre refactoring et ne détecte aucun bug de calcul.

Tester les frontières du système

Représentez votre librairie comme un noyau entouré d'une frontière :



Zone	Comment on la teste
Noyau	tests unitaires directs — aucun double
Adapters	quelques tests d'intégration contre la vraie ressource
Franchissement	fakes ou mocks, selon qu'on vérifie un état ou un appel

Plus le noyau est gros, moins vous aurez besoin de doubles.

L'ordre de préférence

Quand une dépendance doit être remplacée, essayez dans cet ordre :

1. **Rien** — la dépendance est-elle vraiment nécessaire ? Un calcul pur n'en a pas.
2. **L'objet réel** — un objet de valeur, une `dataclass`, une liste : gratuits.
3. **Un fake** — une implémentation simplifiée, écrite une fois, réutilisée partout.
4. **Un stub** — trois lignes, quand une seule réponse figée suffit.
5. **Un mock** — seulement si l'**interaction** est ce qu'on veut vérifier.

 **En pratique** : dans une librairie bien découpée, la majorité des tests se situent aux niveaux 1 et 2. Le niveau 5 reste minoritaire — et c'est un bon indicateur de conception.

À retenir

1. Cinq doubles, deux familles : ceux qui **fournissent une entrée** (dummy, stub, fake) et ceux qui **vérifient une sortie d'interaction** (mock, spy).
2. Préférez systématiquement vérifier un **résultat**. Vérifiez une interaction seulement quand l'interaction est le comportement attendu.
3. Un besoin de mock complexe est un signal de conception : la dépendance est mal isolée, ou la fonction en a trop.

2. Mocking en Python

`unittest.mock`, et la règle qui évite 80 % des erreurs

Mock : un objet qui accepte tout

```
from unittest.mock import Mock

source = Mock()
source.lire(projet="site-web")
source.n_importe_quoi.encore_autre_chose()    # ne lève rien
```

Un `Mock` crée ses attributs et méthodes à la demande, et enregistre tous les appels.

```
source.lire.return_value = [Tache("Déployer")] # ce qu'il renverra
source.lire.assert_called_once_with(projet="site-web")
```

Mock	attributs et méthodes à la demande
MagicMock	idem, plus les méthodes spéciales (<code>__len__</code> , <code>__iter__</code> , <code>__enter__</code> ...)

`MagicMock` est nécessaire dès que le code fait `len(obj)`, `for x in obj`,
ou `with obj:`. En cas de doute, `MagicMock` — c'est le défaut de `patch`.

Valeurs de retour et effets de bord

`return_value` — toujours la même

```
client = Mock()
client.get.return_value = {"statut": "ok"}

client.get("/a") # {'statut': 'ok'}
client.get("/b") # {'statut': 'ok'}
```

`side_effect` = **liste** — une par appel

```
client.get.side_effect = [
    {"page": 1},
    {"page": 2},
    StopIteration,
]
```

`side_effect` = **exception** — simuler une panne

```
client.get.side_effect = TimeoutError(
    "délai dépassé"
)

with pytest.raises(ServiceIndisponible):
    recuperer_taches(client)
```

`side_effect` = **fonction** — réponse calculée

```
def repondre(chemin):
    if chemin.startswith("/projets"):
        return {"taches": []}
    raise KeyError(chemin)

client.get.side_effect = repondre
```

Vérifier les appels

```
notif = Mock()
rappeler(taches_en_retard, notif)
```

Assertion	Vérifie
<code>notif.envoyer.assert_called()</code>	au moins un appel
<code>notif.envoyer.assert_called_once()</code>	exactement un appel
<code>notif.envoyer.assert_called_with("Déployer")</code>	le dernier appel
<code>notif.envoyer.assert_called_once_with("Déployer")</code>	un seul appel, avec ces arguments
<code>notif.envoyer.assert_any_call("Déployer")</code>	au moins un appel avec ces arguments
<code>notif.envoyer.assert_not_called()</code>	aucun appel

Et pour les cas plus fins :

```
assert notif.envoyer.call_count == 2
assert notif.envoyer.call_args.args == ("Déployer",)
assert notif.envoyer.call_args_list == [call("Relire"), call("Déployer")]
```

⚠ Le piège n°1 : un **Mock** nu accepte tout

Python protège les fautes de frappe sur les assertions — `assert_called_once()` lève bien une `AttributeError` depuis la 3.5. Le vrai danger est ailleurs :

```
notif = Mock()
notif.envoyerr("Déployer")          # méthode qui n'existe pas
notif.envoyer("a", "b", "c", 42)    # signature fantaisiste
notif.envoyerr.assert_called_once_with("Déployer") # ✓ vert
```

Ces trois lignes passent. Conséquence concrète : le jour où quelqu'un renomme `envoyer` en `notifier` dans le code de production, **vos tests restent verts**.

Objet	Méthode inexistante	Mauvaise signature
<code>Mock()</code>	acceptée	acceptée
<code>Mock(spec=Notificateur)</code>	<code>AttributeError</code>	acceptée
<code>create_autospec(Notificateur)</code>	<code>AttributeError</code>	<code>TypeError</code>

🎯 **En pratique** : `create_autospec` par défaut. C'est le seul qui vérifie aussi les signatures — donc le seul qui rougit quand l'interface réelle change.

patch : remplacer temporairement

Quand la dépendance n'est pas injectée, il faut la remplacer là où elle vit.

Gestionnaire de contexte

```
def test_appel_reseau():
    with patch("todo.api.requests.get") as get:
        get.return_value.json.return_value = {"v": 1}
        assert recuperer() == 1
```

patch.object — plus sûr

```
def test_appel_reseau():
    with patch.object(api, "recuperer") as r:
        r.return_value = 1
        ...
```

Décorateur

```
@patch("todo.api.requests.get")
def test_appel_reseau(get):
    get.return_value.json.return_value = {"v": 1}
    assert recuperer() == 1
```

Pas de chaîne de caractères : une faute de frappe devient une erreur d'attribut immédiate.

Le décorateur injecte le mock en premier paramètre. Avec plusieurs `@patch`, l'ordre est de bas en haut.

Le piège n°2 : patcher là où c'est utilisé

C'est l'erreur la plus fréquente, et la plus déroutante.

```
# todo/api.py
from requests import get      # ← import direct

def recuperer(url):
    return get(url).json()
```

Ne marche pas ✗

```
patch("requests.get")
```

`todo.api` détient déjà **sa propre référence** vers la fonction. Remplacer l'attribut du module `requests` ne change rien pour lui.

Marche ✓

```
patch("todo.api.get")
```

On remplace la référence **là où elle est consultée**.

La règle : on patche le chemin par lequel le code **cherche** l'objet au moment de l'appel, pas celui où l'objet est défini.

Le corollaire de la règle

Le style d'import détermine le chemin à patcher :

Dans le module testé	À patcher
<code>from requests import get</code>	<code>mon_module.get</code>
<code>import requests puis requests.get(...)</code>	<code>mon_module.requests.get</code> OU <code>requests.get</code>
<code>from .client import Client puis Client()</code>	<code>mon_module.Client</code>

 **En pratique** : préférer `import requests puis requests.get(...)` rend le code **plus facile à patcher**, parce que la résolution se fait à l'appel.

Et surtout : une dépendance **injectée** ne demande aucun patch.

```
def recuperer(url, client=requests): # ← paramètre avec défaut
    return client.get(url).json()

def test_recuperer():
    assert recuperer("/x", client=ClientFactice()) == 1
```

Mocker une méthode, une propriété, un module

Une méthode d'instance

```
with patch.object(Rapport, "charger") as c:  
    c.return_value = [Tache("A"), Tache("B")]  
    Rapport().avancement()
```

Une constante de module

```
with patch("todo.rappels.SEUIL_JOURS", 3):  
    assert doit_rappeler(retard_jours=5)
```

Une propriété

```
from unittest.mock import PropertyMock  
  
with patch.object(  
    Tache, "est_en_retard", new_callable=PropertyMock  
) as en_retard:  
    en_retard.return_value = True  
    ...
```

Un module entier absent

```
import sys  
with patch.dict(sys.modules, {"lib_proprio": Mock()}):  
    import mon_module
```

 **Piège :** `patch.object(Classe, "methode")` remplace la méthode pour **toutes** les instances, pendant toute la durée du bloc. Préférez patcher l'instance quand c'est possible.

Simuler une panne externe

C'est l'usage le plus rentable du mocking : produire à la demande une erreur qu'on ne sait pas provoquer autrement.

```
def test_un_timeout_est_traduit_en_erreur_metier():
    client = create_autospec(ClientHTTP)
    client.get.side_effect = TimeoutError("délai dépassé")

    with pytest.raises(ServiceIndisponible, match="délai"):
        recuperer_taches(client, projet="site-web")

def test_une_reponse_incomplete_est_signalee():
    client = create_autospec(ClientHTTP)
    client.get.return_value = {"projet": "site-web"} # clé « taches » absente

    with pytest.raises(ReponseInvalide):
        recuperer_taches(client, projet="site-web")
```

Ces deux cas se produiront en production. Sans double de test, ils ne seront jamais vérifiés — et c'est précisément là que le code est le plus fragile.

Mocker l'horloge et l'aléatoire

Deux dépendances invisibles, sources de tests instables.

Par patch — fonctionne partout

```
with patch("todo.rapport.datetime") as dt:
    dt.now.return_value = datetime(2026, 1, 5)
    assert titre_rapport() == "Rapport 2026-01-05"
```

Fragile : dépend du style d'import.

Par injection — préférable

```
def titre_rapport(horloge=datetime.now) -> str:
    return f"Rapport {horloge():%Y-%m-%d}"

def test_titre():
    fige = lambda: datetime(2026, 1, 5)
    assert titre_rapport(fige) == "Rapport 2026-01-05"
```

Aucun patch, aucune chaîne de caractères, aucun effet global.

Le même raisonnement vaut pour `random`, `uuid4`, `time.time` et les variables d'environnement : ce sont des **entrées** du système, pas des détails internes.

Récapitulatif : quel outil pour quoi

Besoin	Outil
Une réponse figée	<code>Mock(return_value=...)</code> ou une classe stub de 3 lignes
Des réponses successives	<code>side_effect=[...]</code>
Simuler une panne	<code>side_effect=Exception(...)</code>
Vérifier qu'un envoi a eu lieu	<code>assert_called_once_with</code>
Vérifier qu'un envoi n'a pas eu lieu	<code>assert_not_called</code>
Rester fidèle à l'interface réelle	<code>create_autospec</code>
Remplacer une dépendance non injectée	<code>patch("module_qui_utilise.nom")</code>
Remplacer une dépendance injectée	rien — passez un fake en paramètre

À retenir

1. `create_autospec` par défaut : sans lui, un mock accepte des méthodes et des signatures qui n'existent pas — et vos tests survivent au renommage qu'ils auraient dû détecter.
2. On patche là où l'objet est utilisé, pas là où il est défini. Le style d'import du module testé détermine le chemin.
3. Le meilleur `patch` est celui qu'on n'écrit pas : une dépendance injectée se remplace par un paramètre.

3. Monkeypatch avec pytest

La fixture native, et quand la préférer à `patch`

monkeypatch : modifier, et défaire automatiquement

C'est une fixture native de pytest. Toute modification est **annulée à la fin du test**, y compris s'il échoue.

```
def test_titre_du_rapport(monkeypatch):
    monkeypatch.setattr(rapport, "aujourd'hui", lambda: date(2026, 1, 5))
    assert titre() == "Rapport du 2026-01-05"
```

Méthode	Usage
<code>setattr(objet, "nom", valeur)</code>	remplacer un attribut ou une fonction
<code>delattr(objet, "nom")</code>	simuler l'absence d'un attribut
<code>setitem(dict, clé, valeur)</code>	modifier une entrée de dictionnaire
<code>delitem(dict, clé)</code>	simuler une clé manquante
<code>setenv</code> / <code>delenv</code>	variables d'environnement
<code>chdir(chemin)</code>	changer de répertoire courant
<code>syspath_prepend(chemin)</code>	ajouter un chemin d'import

Variables d'environnement

C'est l'usage le plus fréquent, et celui où `monkeypatch` est nettement le plus pratique que `patch`.

```
def test_configuration_par_defaut(monkeypatch):
    monkeypatch.delenv("MESURES_API_URL", raising=False)
    assert url_api() == "https://api.interne/v1"

def test_url_surchargee(monkeypatch):
    monkeypatch.setenv("MESURES_API_URL", "http://localhost:8000")
    assert url_api() == "http://localhost:8000"

def test_url_invalide_est_refusee(monkeypatch):
    monkeypatch.setenv("MESURES_API_URL", "pas-une-url")
    with pytest.raises(ConfigurationInvalide):
        url_api()
```

`raising=False` évite l'erreur quand la variable n'était pas définie —
indispensable pour que le test passe aussi bien sur votre poste qu'en CI.

Remplacer une fonction ou une méthode

Une fonction de module

```
def test_sans_reseau(monkeypatch):
    def faux_get(url, timeout=None):
        return Reponse({"mesures": [12.4]})

    monkeypatch.setattr(
        "mesures.api.requests.get", faux_get
    )
    assert recuperer("35A") == [12.4]
```

Une méthode de classe

```
def test_lecture_figee(monkeypatch):
    monkeypatch.setattr(
        SourceAPI, "lire",
        lambda self, jour: [Mesure(8, 12.4)],
    )
    assert Analyse(SourceAPI()).moyenne(JOUR) == 12.4
```

 **Piège** : la règle du chapitre 2 s'applique à l'identique —

`monkeypatch.setattr` cible là où l'objet est utilisé. `monkeypatch` ne résout pas ce problème, il le partage.

Simuler l'absence d'une dépendance

Cas réel : votre librairie doit fonctionner même quand un paquet optionnel n'est pas installé.

```
def test_repli_quand_le_paquet_optionnel_est_absent(monkeypatch):
    monkeypatch.setitem(sys.modules, "accélérateur", None)

    resultat = calculer_rapide([1, 2, 3])

    assert resultat == [1, 4, 9]      # même résultat, chemin lent
```

Et pour vérifier qu'une clé de configuration manquante est bien signalée :

```
def test_cle_manquante(monkeypatch):
    monkeypatch.delitem(CONFIG, "seuil", raising=False)
    with pytest.raises(SeuilNonConfigure):
        appliquer_seuil(12.4)
```

Ces deux situations arrivent en production et ne se testent pas autrement.

chdir et syspath_prepend

```
def test_chemin_relatif(monkeypatch, tmp_path):  
    (tmp_path / "config.json").write_text('{"seuil": 10}')
```

monkeypatch.chdir(tmp_path)

```
    assert charger_config_locale()["seuil"] == 10
```

`chdir` est restauré automatiquement — contrairement à un `os.chdir` manuel, qui laisserait tous les tests suivants dans le mauvais répertoire.

 **En pratique :** `monkeypatch.chdir` est la bonne réponse à du code existant qui utilise des chemins relatifs. La bonne réponse à long terme est de faire du chemin un **paramètre** de la fonction.

patch ou monkeypatch ?

	<code>unittest.mock.patch</code>	<code>monkeypatch</code>
Origine	bibliothèque standard	fixture pytest
Portée	bloc <code>with</code> ou décorateur	le test entier
Restauration	fin du bloc	fin du test, automatique
Crée un <code>Mock</code>	oui , par défaut	non — vous fournissez la valeur
Vérifie les appels	oui (<code>assert_called*</code>)	non
Variables d'environnement	<code>patch.dict(os.environ, ...)</code>	<code>setenv</code> / <code>delenv</code>
Vérification d'interface		

La règle de choix, en une ligne :

- Vous voulez **vérifier un appel** → `patch` (avec `create_autospec`).
- Vous voulez **remplacer une valeur** → `monkeypatch`.

Et dans les deux cas : si la dépendance peut être **injectée**, ni l'un ni l'autre.

Les trois niveaux, sur le même besoin

Tester une fonction qui lit une URL dans l'environnement et appelle un service :

1. monkeypatch — code existant

```
def test_recuperer(monkeypatch):
    monkeypatch.setenv("API_URL", "http://x")
    monkeypatch.setattr(
        "mesures.api.requests.get", faux_get
    )
    assert recuperer("35A") == [12.4]
```

2. patch + autospec — vérifier l'appel

```
def test_appel_unique():
    client = create_autospec(ClientHTTP)
    recuperer("35A", client=client)
    client.get.assert_called_once()
```

3. Injection — aucun outil

```
class ClientFactice:
    def get(self, url):
        return {"mesures": [12.4]}

def test_recuperer():
    assert recuperer(
        "35A",
        client=ClientFactice(),
        url_base="http://x",
    ) == [12.4]
```

Lisible, rapide, sans chaîne magique.

C'est le chemin que suit le chapitre suivant : rendre le niveau 3 possible.

À retenir

1. `monkeypatch` est la fixture pytest pour **remplacer une valeur** : variables d'environnement, attributs, entrées de dictionnaire, répertoire courant. Tout est restauré automatiquement.
2. `patch` reste préférable quand vous voulez **vérifier un appel**, avec `create_autospec` pour rester fidèle à l'interface réelle.
3. Les deux souffrent du même défaut : ils dépendent d'un **chemin en chaîne de caractères** que rien ne vérifie. Une dépendance injectée n'en a pas besoin.

4. Tester les accès externes

Bases, fichiers, APIs, flux : la stratégie avant l'outil

Le problème commun

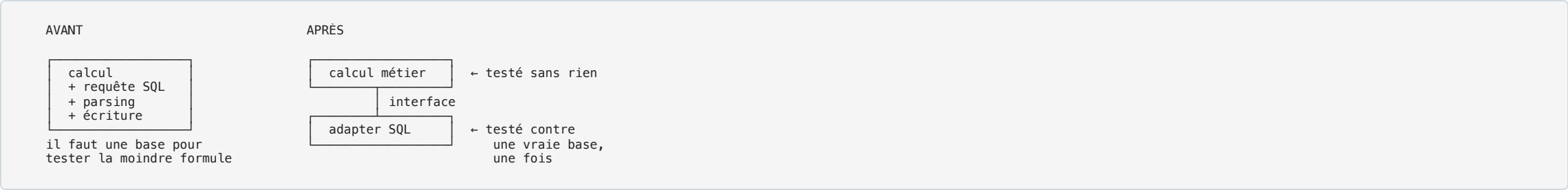
Base de données, fichier, API HTTP, flux temps réel, données distantes : ces dépendances partagent quatre propriétés qui les rendent hostiles aux tests.

Propriété	Conséquence
Lentes	la suite passe de 2 s à 4 min, on cesse de la lancer
Indisponibles	rouge le lundi matin, vert le mardi, sans changement de code
Partagées	deux tests concurrents se marchent dessus
Difficiles à mettre en panne	on ne teste jamais le chemin d'erreur

La bonne question n'est pas « comment mocker ma base ? » mais
« **quelle part de mon code a réellement besoin de la base ?** »

Dans un module bien découpé, la réponse est : quelques dizaines de lignes.

La stratégie : séparer avant de simuler



Ce qu'on teste	Comment	Combien de tests
Le calcul métier	directement, en mémoire	des dizaines
L'adapter	contre la vraie ressource	quelques-uns
Le branchement	un fake en mémoire	quelques-uns

Les sept stratégies disponibles

Stratégie	Principe	Quand
Séparer métier / I/O	extraire une fonction pure	toujours, en premier
Injection de dépendance	recevoir la ressource en paramètre	dès que possible
Adapter	traduire l'API externe en une interface à vous	API tierce, format exotique
Repository	cachez totalement l'origine des données	accès à des données persistées
Service	regrouper l'orchestration d'un cas d'usage derrière une fonction ou une classe	plusieurs adapters à coordonner
Fake en mémoire	une implémentation simplifiée mais réelle	pour tester tout ce qui consomme

Les deux premières ne coûtent rien et suppriment la majorité du besoin de doubles. Les patterns viennent après, quand elles ne suffisent plus.

Cas 1 — Les fichiers

C'est le plus simple, et pourtant celui qu'on mocke le plus inutilement.

Sur-mocké ✗

```
@patch("builtins.open", mock_open(read_data="a,b\n1,2"))
def test_lire():
    assert lire("x.csv") == [{"a": "1", "b": "2"}]
```

Fragile, illisible, et ne teste ni
l'encodage, ni les fins de ligne,
ni les fichiers absents.

Avec un vrai fichier ✓

```
def test_lire(tmp_path):
    chemin = tmp_path / "x.csv"
    chemin.write_text("a,b\n1,2\n", encoding="utf-8")
    assert lire(chemin) == [{"a": "1", "b": "2"}]
```

`tmp_path` est rapide (quelques
millisecondes), isolé, et teste
le **vrai** comportement.

 **En pratique** : ne mockez jamais le système de fichiers. Utilisez
`tmp_path` . Et faites que vos fonctions acceptent un `Path` — ou mieux, un
objet fichier déjà ouvert, ce qui les rend testables avec `io.StringIO` .

Cas 2 — Les bases de données

Trois niveaux, du moins au plus coûteux :

```
# 1. Le métier ne connaît pas la base – c'est ici que sont les règles
def en_retard(taches: list[Tache], jour: date) -> list[str]:
    return sorted(t.titre for t in taches if not t.terminee and t.echeance < jour)
```

```
# 2. Un Repository cache l'origine ; le test fournit un fake en mémoire
class DepotEnMemoire:
    def __init__(self, taches): self._taches = taches
    def du_projet(self, projet): return [t for t in self._taches if t.projet == projet]
```

```
# 3. L'adapter réel est testé contre une vraie base – SQLite en mémoire
@pytest.fixture
def base():
    connexion = sqlite3.connect(":memory:")
    connexion.executescript(SCHEMA)
    yield connexion
    connexion.close()
```

SQLite en mémoire s'ouvre en moins d'une milliseconde. Pour du PostgreSQL, une image jetable en CI joue le même rôle — quelques tests, pas des centaines.

Cas 3 — Les APIs HTTP

Ce qu'il faut éviter

```
def recuperer(projet):  
    r = requests.get(f"{URL}/{projet}")  
    return [t["titre"] for t in r.json()["taches"]]
```

Le parsing, l'URL et le transport sont soudés. Tester le parsing exige de mocker `requests`.

`interpreter` concentre tout ce qui peut mal tourner — clé absente, type inattendu, liste vide — et se teste avec des dictionnaires écrits à la main.

Il ne reste alors qu'à mocker le **transport**, dont le comportement est simple : il répond, ou il échoue.

Séparer transport et interprétation

```
def interpreter(charge: dict) -> list[str]:  
    """Fonction pure – testable sans réseau."""  
    if "taches" not in charge:  
        raise ReponseInvalide(charge)  
    return [t["titre"] for t in charge["taches"]]  
  
def recuperer(projet, client):  
    return interpreter(client.get(f"/{projet}"))
```

Cas 4 — Sources distantes et flux

Pour des données volumineuses issues d'un producteur externe (référentiels publics, flux d'événements, exports d'un autre système) :

Ce qu'on veut vérifier	Comment
Le format attendu est bien géré	un extrait de 5 lignes, versionné dans <code>tests/donnees/</code>
Un format inattendu est détecté	un extrait volontairement corrompu
Le volume ne casse rien	un test marqué <code>lent</code> , exclu de la boucle rapide
Le producteur a changé son format	un test d'intégration réel, joué une fois par jour en CI

⚠ **Piège** : le dernier point est le seul que les doubles ne couvrent **jamais**. Un fake reproduit ce que vous *croyez* que le service renvoie.
Prévoyez au moins un test qui parle au vrai service, isolé dans un stage CI séparé pour ne pas bloquer les développeurs quand le fournisseur est en panne.

Le test de contrat : garder fake et réel alignés

Un fake dérive avec le temps. Un test de contrat écrit **une seule fois** et rejoué sur les deux implémentations empêche cette dérive.

```
class ContratDepot:
    """Tests que toute implémentation de Depot doit satisfaire."""

    def test_relit_ce_qui_a_ete_ecrit(self, depot):
        depot.enregistrer(Tache(1, "Déployer"))
        assert depot.par_id(1) == Tache(1, "Déployer")

    def test_identifiant_inconnu(self, depot):
        with pytest.raises(TacheInconnue):
            depot.par_id(42)

class TestDepotEnMemoire(ContratDepot):
    @pytest.fixture
    def depot(self): return DepotEnMemoire()

class TestDepotSQLite(ContratDepot):
    @pytest.fixture
    def depot(self, base): return DepotSQLite(base)
```

Les mêmes assertions s'exécutent deux fois. Si le fake ment, la comparaison le révèle.

À retenir

1. Avant de choisir un outil de simulation, **déplacez la frontière** :
une fonction pure extraite est une fonction qui n'a plus besoin de double.
2. Ne mockez jamais le système de fichiers — `tmp_path` est plus simple,
plus rapide et plus fidèle.
3. Un fake finit toujours par diverger du réel. Un **test de contrat**
rejoué sur les deux implémentations est la seule protection.

5. Application des design patterns à la testabilité

Sept patterns, une même grille de lecture

La grille de lecture

Pour chacun des sept patterns, quatre questions dans le même ordre :

1. Le problème	quelle douleur concrète ?
2. La transformation	qu'est-ce qui change dans le code ?
3. Le test rendu possible	qu'est-ce qu'on peut vérifier qu'on ne pouvait pas ?
4. Quand s'abstenir	dans quel cas le pattern coûte plus qu'il ne rapporte ?

La quatrième question est celle qu'on ne pose jamais. Elle est pourtant la plus utile : elle vous évitera d'appliquer six patterns à un module de 80 lignes.

Adapter — traduire une interface externe

Problème : votre code métier parle le langage d'une bibliothèque tierce.

Le jour où elle change — ou qu'il faut la tester — tout le code est concerné.

Avant

```
class Convertisseur:
    def convertir(self, montant, de, vers):
        r = requests.get(
            f"{URL}/v3/latest",
            params={"base": de, "apikey": CLE},
            timeout=30,
        )
        rates = r.json()["rates"]
        return round(montant * rates[vers], 2)
```

Le vocabulaire du fournisseur (`rates` ,
`base` , `apikey`) a envahi le métier.

Après

```
class SourceTaux(Protocol):
    def taux(self, base: str) -> dict[str, float]: ...

class AdaptateurApiTaux:
    """Traduit l'API du fournisseur en dict de taux."""
    def __init__(self, client): self._client = client

    def taux(self, base):
        brut = self._client.get("/v3/latest", base=base)
        return {d: float(v) for d, v in brut["rates"].items()}

class Convertisseur:
    def __init__(self, source: SourceTaux):
        self._source = source
```

Adapter — ce que le test devient

Le métier, sans réseau

```
class SourceFiguee:
    def taux(self, base):
        return {"USD": 1.08, "GBP": 0.84}

def test_convertir():
    conv = Convertisseur(SourceFiguee())
    assert conv.convertir(100, "EUR", "USD") == 108.0
```

L'adapter, isolément

```
def test_traduction_du_format_fournisseur():
    client = create_autospec(ClientHTTP)
    client.get.return_value = {
        "base": "EUR", "rates": {"USD": 1.08}
    }

    taux = AdaptateurApiTaux(client).taux("EUR")

    assert taux == {"USD": 1.08}
```

Zéro mock, zéro patch, zéro réseau.

Quand s'abstenir : quand la bibliothèque tierce est déjà stable, simple, et que son vocabulaire est le vôtre. Adapter `json.loads` n'a aucun intérêt.

 **Le signe qu'il faut un Adapter** : le nom d'un fournisseur ou d'un format apparaît dans votre code métier.

Strategy — rendre un algorithme interchangeable

Problème : une cascade de `if` sur un mode de calcul. Chaque nouveau mode rouvre la fonction et met en danger les cas déjà testés.

Avant

```
def exporter(taches, format="texte"):
    if format == "texte":
        ...
    elif format == "csv":
        ...
    elif format == "markdown":
        ...
    else:
        raise ValueError(format)
```

Tester l'export Markdown exige de passer par `exporter`, donc par la cascade.

Après — en Python, un dict de fonctions

```
def en_texte(taches: list[Tache]) -> str: ...
def en_csv(taches: list[Tache]) -> str: ...
def en_markdown(taches: list[Tache]) -> str: ...

FORMATS = {
    "texte": en_texte,
    "csv": en_csv,
    "markdown": en_markdown,
}

def exporter(taches, format="texte"):
    try:
        return FORMATS[format](taches)
    except KeyError:
        raise FormatInconnu(format) from None
```

Strategy — ce que le test devient

```
@pytest.mark.parametrize("nom", FORMATS)          # ← couvre les futurs ajouts
def test_tout_format_contient_les_titres(nom):
    assert "Déployer" in exporter([Tache("Déployer")], nom)

def test_markdown_directement():
    """Chaque stratégie se teste seule, sans passer par le dispatch."""
    assert en_markdown([]).startswith("| Titre |")

def test_strategie_maison():
    """Le code appelant peut injecter sa propre stratégie."""
    assert exporter_avec([Tache("A")], lambda t: str(len(t))) == "1"
```

Quand s'abstenir : deux branches stables qui ne bougeront pas. Un `if` lisible vaut mieux qu'un registre.

Passez aux classes quand la stratégie a besoin d'un **état** (des paramètres de configuration) ou de **plusieurs méthodes**. Sinon, une fonction suffit — et se teste plus facilement.

Factory — centraliser la décision de création

Problème : le code qui décide *quel* objet construire est dupliqué à six endroits. Ajouter un type oblige à retrouver les six.

Avant, dans six modules

```
if cfg["type"] == "csv":
    source = SourceCSV(cfg["chemin"])
elif cfg["type"] == "api":
    source = SourceAPI(cfg["url"], cfg["cle"])
elif cfg["type"] == "sql":
    source = SourceSQL(connect(cfg["dsn"]))
```

Après, à un seul endroit

```
CONSTRUCTEURS = {
    "csv": lambda c: SourceCSV(Path(c["chemin"])),
    "api": lambda c: SourceAPI(c["url"], c["cle"]),
    "sql": lambda c: SourceSQL(connect(c["dsn"])),
}

def creer_source(cfg: dict) -> SourceTaches:
    """Construit la source décrite par la configuration."""
    try:
        return CONSTRUCTEURS[cfg["type"]](cfg)
    except KeyError as e:
        raise TypeDeSourceInconnu(cfg.get("type")) from e
```

Le test devient possible **parce que la décision est isolée** : on vérifie le choix sans construire de vraie connexion.

```
def test_choix_du_type():
    assert isinstance(creer_source({"type": "csv", "chemin": "x"}), SourceCSV)
```

Quand s'abstenir : un seul type, ou une construction d'une ligne.

Factory pour fabriquer une `dataclass` est du bruit.

Repository — cacher l'origine des données

Problème : le SQL est éparpillé dans le métier. Impossible de tester une règle de gestion sans base.

Avant

```
class Rapport:
    def surcharges(self):
        cur = self.conn.cursor()
        cur.execute(
            "SELECT assignee, COUNT(*) FROM taches "
            "WHERE terminee = 0 GROUP BY assignee "
            "HAVING COUNT(*) > ?", (SEUIL,))
        return [r[0] for r in cur.fetchall()]
```

La règle métier (« plus de tâches que le seuil ») est **dans le SQL**.

La règle est remontée dans du Python testable. Le dépôt ne fait plus que **fournir des données**.

Après

```
class DepotTaches(Protocol):
    def toutes(self) -> list[Tache]: ...

class Rapport:
    def __init__(self, depot: DepotTaches):
        self._depot = depot

    def surcharges(self, seuil=SEUIL):
        restantes = [t for t in self._depot.toutes() if not t.terminee]
        charges = Counter(t.assignee for t in restantes)
        return sorted(p for p, n in charges.items() if n > seuil)
```

Repository — ce que le test devient

Le fake, écrit une fois

```
class DepotEnMemoire:
    def __init__(self, taches=()):
        self._taches = list(taches)

    def toutes(self):
        return list(self._taches)
```

Dix lignes, réutilisables dans tous les tests du projet.

Ce test de borne était **impossible** avec la version SQL sans peupler une base.

Quand s'abstenir : un script qui fait deux requêtes. Le Repository se justifie quand plusieurs règles métier consomment les mêmes données.

Le métier, sans base

```
def test_seuil_strict():
    """Exactement au seuil, on n'est pas surchargé."""
    depot = DepotEnMemoire(
        [Tache(f"A{i}", assignee="ada") for i in range(SEUIL)]
        + [Tache(f"Z{i}", assignee="zoe") for i in range(SEUIL + 1)]
    )

    assert Rapport(depot).surcharges() == ["zoe"]
```

Facade — une porte d'entrée simple

Problème : pour accomplir une tâche courante, l'appelant doit connaître cinq classes et leur ordre d'assemblage.

Avant, chez chaque consommateur

```
cfg = charger_config(chemin)
depot = creer_source(cfg["source"])
regles = Rapport(depot, cfg["seuils"])
rendu = FORMATS[cfg["format"]]
notif = creer_notificateur(cfg["notif"])
resultats = regles.hebdomadaire(jour)
notif.envoyer(rendu(resultats))
```

Sept lignes recopiées dans quatre projets.

Une évolution interne les casse tous.

Attention : une facade **ne rend rien testable par elle-même**. Elle protège vos consommateurs de vos réorganisations internes. Elle se teste avec un ou deux tests de bout en bout, pas plus — les détails sont testés en dessous.

Après

```
class Rapports:
    """Point d'entrée unique de la librairie."""

    def __init__(self, cfg: Configuration):
        self._regles = Rapport(creer_source(cfg.source), cfg.seuils)
        self._rendu = FORMATS[cfg.format]
        self._notif = creer_notificateur(cfg.notif)

    def hebdomadaire(self, jour: date) -> None:
        resultats = self._regles.hebdomadaire(jour)
        self._notif.envoyer(self._rendu(resultats))
```

Dependency Injection — les trois formes

C'est moins un pattern qu'une discipline : **ne construisez pas ce que vous pouvez recevoir.**

Forme	Code	Quand
Par constructeur	<code>def __init__(self, depot: DepotTaches)</code>	dépendance utilisée par plusieurs méthodes
Par paramètre	<code>def calculer(valeurs, horloge=datetime.now)</code>	dépendance ponctuelle, avec un défaut sensé
Par assemblage	tout est construit dans <code>main()</code> / le point d'entrée	pour l'application complète

```
# Le point d'entrée est le SEUL endroit qui connaît les implémentations réelles
def main() -> None:
    depot = DepotSQLite(sqlite3.connect(DSN))
    notif = NotificateurSMTP(SMTP_HOST)
    Rappels(depot, notif).executer(date.today())
```

En Python, **aucun conteneur d'injection n'est nécessaire**. Les frameworks de DI répondent à des contraintes de langages statiques que Python n'a pas.

Template Method — figer un squelette

Problème : le même enchaînement d'étapes est recopié dans plusieurs traitements, avec deux ou trois variantes.

Le squelette

```
class Traitement(ABC):
    """Squelette commun : ne pas surcharger 'executer'."""

    def executer(self, jour: date) -> Rapport:
        brut = self.charger(jour)
        valide = self.valider(brut) # commun
        resultat = self.calculer(valide) # varie
        return self.formater(resultat) # varie

    def valider(self, brut): # commun, surchargeable
        return [t for t in brut if t.titre.strip()]

    @abstractmethod
    def charger(self, jour): ...
    @abstractmethod
    def calculer(self, taches): ...
    @abstractmethod
    def formater(self, resultat): ...
```

Le test du squelette, une fois

```
class TraitementFactice(Traitement):
    def charger(self, jour):
        return [Tache("Déployer"), Tache(" ")]
    def calculer(self, taches):
        return len(taches)
    def formater(self, r):
        return Rapport(r)

def test_la_validation_precede_le_calcul():
    """Les titres vides sont écartés avant 'calculer'."""
    assert TraitementFactice().executer(30UR).contenu == 1
```

Quand s'abstenir : deux implémentations seulement, ou des étapes qui varient trop. L'héritage rend le flot difficile à suivre — préférez alors composer avec des Strategy injectées.

Template Method ou Strategy ?

Les deux rendent un comportement variable. La différence est structurelle :

	Template Method	Strategy
Mécanisme	héritage	composition
Ce qui varie	des étapes dans un flot figé	un algorithme entier
Nombre de points de variation	plusieurs	un
Changeable à l'exécution	non	oui
Testabilité	il faut une sous-classe	on passe une fonction

```
# Souvent préférable en Python : le squelette devient une fonction
def executer(jour, charger, calculer, formater, valider=valider_par_defaut):
    return formater(calculer(valider(charger(jour))))
```

 **En pratique** : essayez d'abord la version « fonction avec des callables injectés ». Passez à la classe abstraite si l'état partagé entre étapes le justifie.

Le pattern améliore-t-il vraiment le code ?

Trois questions, à poser **avant** d'introduire un pattern :

Question	Si la réponse est non
Le problème s'est-il déjà produit ?	attendez qu'il se produise
Y a-t-il au moins deux cas réels ?	une seule implémentation ne justifie pas une abstraction

Signes de sur-design à repérer en relecture :

- une interface avec une seule implémentation, et aucun double de test ;
- une factory qui construit un seul type ;
- une hiérarchie de trois classes pour trente lignes de logique ;
- des noms qui parlent de la solution (`AbstractHandlerFactory`) plutôt que du domaine (`GrilleTarifaire`).

 **Piège** : le sur-design se défend mal en relecture, parce qu'il a l'air compétent. Le critère objectif reste le même : **combien de tests étaient impossibles avant, et sont possibles maintenant ?**

À retenir

1. Les sept patterns font tous la même chose : ils **introduisent un point de substitution**. C'est ce point qui rend le test possible.
2. En Python, la version « fonction, `dict` ou paramètre » suffit souvent. Passez aux classes quand il y a un **état** ou plusieurs méthodes.
3. Le critère d'adoption n'est pas l'élégance : c'est le nombre de tests qui deviennent possibles. Si aucun ne le devient, n'introduisez pas le pattern.

6. Design patterns, SOLID et mocking

Trois sujets, une seule idée

Le fil qui relie les trois

	Ce que ça dit	Effet sur les tests
SOLID	dépendez d'abstractions, gardez les interfaces étroites	il devient possible de substituer
Patterns	voici les formes concrètes que prend cette substitution	il devient simple de substituer
Doubles	voici quoi mettre à la place	on exerce la substitution

Autrement dit :

Le mocking n'est pas une compétence de test. C'est la **conséquence** d'une décision de conception prise bien avant, quand quelqu'un a choisi de recevoir une dépendance plutôt que de la construire.

Une meilleure conception réduit le besoin de mocks

Le même besoin — tester une règle métier qui dépend d'une base — selon la qualité de la conception :

Conception	Test
La classe ouvre sa connexion	<code>patch("module.sqlite3.connect")</code> + un <code>Mock</code> de curseur + des <code>fetchall</code> figés
La connexion est injectée	un <code>sqlite3.connect(":memory:")</code> réel
Un Repository est injecté	un fake en mémoire de dix lignes
La règle est une fonction pure	une liste et un <code>assert</code>

Le volume de code de test décroît à chaque ligne. Et surtout : **la dernière version teste la règle métier, les précédentes testent le plomberie.**

 **En pratique** : quand un test devient pénible à écrire, remontez d'une ligne dans ce tableau plutôt que de sortir un outil plus puissant.

Comment un pattern peut compliquer inutilement

Le pattern mal choisi produit exactement l'effet inverse.

Sur-design

```
class ValideurFactory:
    def creer(self, type_): ...

class AbstractValideur(ABC):
    @abstractmethod
    def valider(self, v): ...

class ValideurPositif(AbstractValideur):
    def valider(self, v): return v > 0
```

Trois classes, un fichier, une usine —
pour tester `v > 0`.

Suffisant

```
def est_positif(valeur: float) -> bool:
    return valeur > 0
```

Testable en une ligne, lisible sans
contexte, remplaçable le jour où un
deuxième cas apparaîtra.

⚠ Piège : un pattern ajouté « au cas où » crée des points de substitution que personne n'exerce. Ce sont des interfaces mortes — du coût de maintenance sans contrepartie.

Le test comme révélateur de conception

Ce que la difficulté d'écriture d'un test vous dit du code :

Symptôme dans le test	Ce que ça révèle	Correctif
Trois <code>patch</code> pour un test	trop de dépendances	découper la fonction
Un <code>Mock</code> dont on configure 5 attributs	interface trop grosse	ISP : interfaces étroites
Un <code>patch</code> sur un chemin de module	dépendance non injectée	DIP : injecter
Le test connaît des attributs privés	on teste l'implémentation	tester le comportement
40 lignes de préparation	l'objet construit ses dépendances	injection + fixtures
Le test casse à chaque refactoring	test d'interaction là où un test d'état suffirait	assertion sur le résultat

Aucune ligne de ce tableau ne se corrige dans le test. **Toutes** se corrigent dans le code de production.

À retenir

1. SOLID rend la substitution possible, les patterns la rendent simple, les doubles l'exercent. C'est une seule chaîne, pas trois sujets.
2. Quand un test devient pénible, améliorez la **conception** plutôt que l'outillage de test. Le tableau des symptômes vous dit quoi changer.
3. Un pattern qui ne rend aucun test possible ne rend service à personne.

Travaux pratiques du Jour 3

Quatre exercices indépendants — 3 h 30

Le programme

TP	Ce que vous allez faire	Durée
j3-01	Tester « les tâches en retard d'un projet » : la fonction lit l'environnement, appelle un service et consulte l'horloge. Trois versions du même test : <code>monkeypatch</code> , <code>patch</code> + autospec, puis injection — et comparaison.	45 min
j3-02	Extraire un Adapter autour d'une API de taux de change, écrire un fake, et faire remonter la panne qu'un <code>except:</code> <code>return 0.0</code> masquait.	60 min
j3-03	Introduire un Repository pour remonter les règles du rapport de la todo-list hors du SQL, puis écrire le test de contrat qui garde le fake aligné sur SQLite.	60 min
j3-04	Remplacer la cascade de <code>if</code> d'un export texte / CSV / JSON / Markdown par une Strategy , et rendre chaque format testable seul.	45 min

La question à garder en tête

Pour chaque TP, avant d'écrire le test, demandez-vous :

Est-ce que je vérifie un résultat, ou une interaction ?

Et à la fin de chaque TP :

Combien de tests étaient impossibles avant ce refactoring ?

C'est le seul critère qui justifie d'avoir introduit un pattern. Si la réponse est « zéro », le pattern était de trop — et c'est une réponse acceptable, que nous discuterons.

La *suppression d'une dépendance circulaire*, listée au programme de ce jour, est traitée demain en **j4-04** — une fois vus les outils de résolution.

Le TP **j3-01** est le plus court à comprendre mais le plus riche en discussion : gardez vos trois versions du test côte à côte pour la mise en commun.

1. Approfondir le TDD sur du code existant

Le cycle ne change pas — le point de départ, si

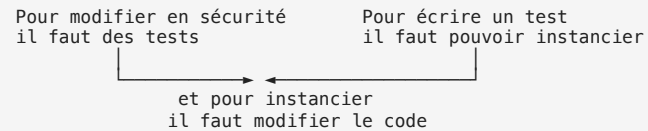
Code neuf, code existant : la même boucle, un autre problème

	Code neuf	Code existant
Point de départ	une page blanche	4 000 lignes qui tournent en production
La question	<i>que doit faire ce code ?</i>	<i>que fait-il déjà, exactement ?</i>
Le premier obstacle	choisir l'interface	réussir à instancier l'objet
Le risque principal	mal concevoir	casser sans s'en apercevoir
Le premier test	décrit une intention	capture un comportement

Le cycle Red / Green / Refactor reste identique. Ce qui change, c'est qu'avant de pouvoir écrire le premier test, il faut souvent **modifier le code** — et qu'on n'a pas encore de filet pour le faire.

C'est le paradoxe du legacy, et il a des solutions mécaniques.

Le dilemme du legacy



Trois façons d'en sortir, par ordre de risque croissant :

Sortie	Principe	Risque
Bourgeon (<i>sprout</i>)	écrire le neuf à côté, en TDD, sans toucher à l'existant	quasi nul
Enveloppe (<i>wrap</i>)	encadrer l'existant sans le modifier	faible
Couture (<i>seam</i>)	créer un point d'insertion par une modification minimale	modéré, à faire à petits pas

La couture : le plus petit changement qui rend testable

Une **couture** est un endroit où l'on peut changer le comportement du programme sans modifier le code à cet endroit.

Aucune couture

```
class Rapport:
    def __init__(self):
        self.db = connect(DSN)    # soudé
```

Impossible d'instancier sans base.

Une couture, en une ligne

```
class Rapport:
    def __init__(self, db=None):
        self.db = db or connect(DSN)
```

Le comportement en production est **identique**. Les tests peuvent désormais passer un fake.

 **En pratique** : c'est la modification la plus rentable du legacy. Elle tient en une ligne, ne change rien pour les appelants existants, et transforme un module intestable en module testable.

À terme, on supprimera le `or connect(DSN)` — mais pas au premier jour.

Bourgeon : écrire le neuf à côté

On vous demande d'ajouter une règle dans une méthode de 300 lignes. **N'écrivez pas la règle dedans.**

1. La nouvelle règle, en TDD, à part

```
def doit_etre_signale(mesure: Mesure) -> bool:
    """Nouvelle règle métier, écrite en TDD."""
    return mesure.valeur > SEUIL and mesure.validee
```

Testée à fond, isolément, sans avoir
lu les 300 lignes.

Le code de 300 lignes n'a pas été amélioré — mais il n'a pas empiré, et la
partie neuve est correcte. C'est un progrès net, obtenu en une heure.

2. Un seul appel dans l'existant

```
# ... 187 lignes plus haut ...
if doit_etre_signale(mesure):      # ← la seule ligne ajoutée
    signalements.append(mesure)
# ... 112 lignes plus bas ...
```

Le risque de régression se limite à
une ligne.

Enveloppe : encadrer sans modifier

Quand la nouvelle fonctionnalité doit s'exécuter **avant ou après** l'existant, sans s'y insérer :

```
def generer_rapport(jour):          # ← méthode existante, inchangée
    ...

def generer_rapport_avec_journal(jour):    # ← enveloppe, écrite en TDD
    """Génère le rapport et consigne l'opération."""
    debut = horloge()
    resultat = generer_rapport(jour)
    journaliser(jour, duree=horloge() - debut)
    return resultat
```

L'ancien code n'est pas relu, pas modifié, pas mis en danger.

 **Piège** : bourgeon et enveloppe laissent la dette en place. Ce sont des techniques de **survie**, à utiliser quand le délai ne permet pas mieux — pas une stratégie à long terme. Notez la dette quelque part.

Caractériser avant de modifier

Quand il faut vraiment entrer dans le code existant, on commence par **figer ce qu'il fait**, sans se demander si c'est juste.

```
def test_caracterisation_calcul_v1():  
    """Fige le comportement actuel – NON validé par le métier.  
  
    Objectif : détecter tout changement pendant le refactoring.  
    """  
    assert calculer(RELEVE_REFERENCE) == pytest.approx(72.4)
```

La technique pour trouver la valeur, quand personne ne la connaît :

```
assert calculer(RELEVE_REFERENCE) == 0          # valeur volontairement fausse
```

```
E   assert 72.4 == 0
```

Le message d'erreur **vous donne** la réponse. Vous la recopiez. Le filet est posé.

Le piège du test de caractérisation

Un test de caractérisation **fige les bugs autant que les comportements corrects**. C'est voulu — mais il faut que ce soit dit.

Dangereux

```
def test_calculer():  
    assert calculer(RELEVÉ) == 72.4
```

Six mois plus tard, quelqu'un
« corrige » ce test parce que la
valeur lui semble fausse. Il avait
raison : elle l'était. Et il vient
de supprimer la seule trace du bug.

Honnête

```
def test_caracterisation_calculer():  
    """Comportement OBSERVÉ au 2026-09-04.  
  
    Non validé par le métier. La valeur 72.4  
    inclut potentiellement le bug #4231  
    (arrondi appliqué deux fois).  
    À revoir après spécification.  
    """  
    assert calculer(RELEVÉ) == pytest.approx(72.4)
```

 **En pratique** : nommez ces tests `test_caracterisation_*` et datez la
docstring. La distinction avec un vrai test unitaire doit sauter aux yeux.

Le cas du calcul scientifique

Sur un calcul numérique, la valeur de référence est souvent inconnue, ou dépendante de la plateforme. Testez alors des **propriétés**, pas des valeurs.

Type de propriété	Exemple d'assertion
Bornes	<code>0 <= indice <= 100</code>
Monotonie	<code>f(x) <= f(y)</code> dès que <code>x <= y</code>
Conservation	<code>sum(parts) == total</code>
Symétrie	<code>distance(a, b) == distance(b, a)</code>
Invariance	permuter l'ordre d'entrée ne change pas la moyenne
Cas dégénéré connu	<code>f(0) == 0</code> , <code>f(constante) == constante</code>

La dernière ligne est particulièrement utile en refactoring de code scientifique : gardez l'ancienne implémentation, et testez que la nouvelle lui est équivalente à `1e-9` près sur mille entrées aléatoires à **graine fixe**.

Pour aller plus loin : générer les cas, pas les écrire

Tester une propriété sur cinq valeurs choisies à la main laisse passer la sixième. **Hypothesis** génère les entrées, cherche les contre-exemples, et réduit celui qu'il trouve au cas minimal.

```
from hypothesis import given, strategies as st

@given(
    montant=st.integers(min_value=0, max_value=10_000_000),
    poids=st.lists(st.integers(min_value=1, max_value=100), min_size=1, max_size=20),
)
def test_la_somme_des_parts_egale_toujours_le_montant(montant, poids):
    assert sum(repartir(montant, poids)) == montant
```

Une centaine d'exécutions par défaut, avec des cas qu'on n'aurait pas pensé à écrire : un seul bénéficiaire, vingt poids identiques, un montant de 1 centime.

 **En pratique** : c'est l'outil naturel des propriétés de la slide précédente — bornes, monotonie, conservation, symétrie. Sur du calcul scientifique, il trouve les cas dégénérés que les jeux de référence oublient.

`uv add --dev hypothesis`, et rien d'autre à installer.

Combiner les trois natures de test

Sur une base existante, les trois se complètent — dans cet ordre de construction :

- | | |
|---|---|
| 1. Un test de bout en bout
sur un jeu réel | « quelque chose a changé »
← posé en premier, coût minimal |
| ↓ | |
| 2. Des tests de caractérisation
par fonction touchée | « le changement est dans cette fonction »
← posés avant chaque refactoring |
| ↓ | |
| 3. Des tests unitaires
sur les règles extraites | « le changement est cette règle, à cette ligne »
← écrits au fur et à mesure de l'extraction |

Chaque niveau **remplace progressivement** le précédent : une fois la règle extraite et testée unitairement, le test de caractérisation correspondant peut être supprimé.

 **Piège** : garder les trois niveaux indéfiniment produit une suite lente et redondante. Les tests de caractérisation sont un **échafaudage**.

À retenir

1. Sur du legacy, le premier obstacle n'est pas d'écrire le test, c'est de **pouvoir instancier**. Une couture — un paramètre avec valeur par défaut — lève ce blocage en une ligne, sans changer la production.
2. Bourgeon et enveloppe permettent d'écrire du neuf en TDD **sans toucher** à l'existant. Le risque de régression se limite à une ligne d'appel.
3. Un test de caractérisation fige les bugs avec le reste. Nommez-le et datez-le pour que personne ne le confonde avec une validation métier.

2. Écrire le test avant le code

Six règles pour que le test reste utile dans deux ans

1. Partir du comportement, pas de la fonction

Partir du code ✗

« Je dois écrire une méthode

`calculer_majoration` dans la classe

`Tarif`. »

Le test suit la structure imaginée.

Si la structure change, il casse.

Formulation qui marche bien, en trois temps :

```
Étant donné  un colis de 35 kg
Quand       on calcule les frais en mode standard
Alors       la majoration de surpoids s'applique
```

→ `def test_surpoids_majore_les_frais_en_standard():`

Partir du comportement ✓

« Au-delà de 8 heures, les heures supplémentaires sont majorées. »

Le test énonce la règle. La structure qui la porte peut évoluer librement.

2. Commencer par le cas le plus simple

Le premier test ne doit pas être le plus représentatif. Il doit être **celui qui vous fait écrire le moins de code.**

Ordre	Test	Ce qu'il force à décider
1	<code>repartir(100, [1, 1]) == [50, 50]</code>	le nom, les paramètres, l'unité, le type de retour
2	<code>repartir(100, [3, 1]) == [75, 25]</code>	la proportionnalité
3	<code>repartir(100, [1, 1, 1])</code>	la règle d'arrondi — le vrai sujet
4	<code>repartir(0, [1, 1])</code>	le cas dégénéré
5	<code>repartir(100, [])</code>	l'erreur métier

Commencer par le cas 3 est tentant, puisque c'est le difficile. C'est aussi le meilleur moyen de mélanger quatre décisions et de ne plus savoir laquelle a causé l'échec.

3. Faire émerger l'interface, ne pas la décréter

Le test est le **premier code appelant**. Ce qui est pénible à écrire dans un test sera pénible pour vos consommateurs.

Ce que le test révèle

```
def test_frais():  
    frais = calculer(  
        2.0, 100.0, True, False, "std", None, 3  
    )
```

Sept paramètres, dont trois booléens
et un `None`. Personne ne relira cet
appel.

Ce que le test suggère

```
def test_frais():  
    frais = frais_livraison(  
        Colis(poids_kg=2.0, distance_km=100.0),  
        mode="standard",  
    )
```

 **En pratique** : quand un appel devient pénible à écrire dans un test,
ne cherchez pas à rendre le test plus court. **Changez la signature.**

4. Ajouter les cas limites un par un

Pas cinq d'un coup. Un seul, en rouge, puis vert, puis le suivant.

```
def test_serie_vide():          # rouge → implémentation → vert
    with pytest.raises(SerieVide):
        mediane([])
```

À chaque cas limite ajouté, posez-vous la question :

Est-ce que ce cas est possible chez le consommateur ?

Cas	Verdict
Liste vide	oui — une journée sans aucune vente
Valeur négative	oui — un avoir, un remboursement
Liste d'un milliard d'éléments	non — inutile de le gérer

Ajouter des cas impossibles produit du code défensif que personne n'exécutera — et qui restera non testé en pratique.

5. Garder les tests lisibles

Un test se lit **cent fois** et s'écrit une fois. C'est le seul code où l'optimisation pour la lecture prime sur tout le reste.

Compact mais opaque ✗

```
@pytest.mark.parametrize("d", DATA)
def test_all(d):
    r = process(**d["in"])
    for k, v in d["exp"].items():
        assert getattr(r, k) == v
```

Un échec affiche `d = {...}`.

Bon courage.

Trois règles concrètes : **pas de boucle**, **pas de `if`**, **pas d'héritage de test** — sauf pour un test de contrat, où c'est justement le but.

Explicite ✓

```
def test_surpoids_majore_de_12_euros():
    frais = standard(Colis(35.0, 100.0))
    assert frais == pytest.approx(31.15)
```

Un échec affiche la règle, l'entrée et l'écart.

6. Ne pas tester les détails d'implémentation

Ce qu'on teste	Verdict
La valeur retournée	✓ toujours
L'exception levée et son message	✓ toujours
L'état observable après l'appel	✓ si public
Un effet hors du système (envoi, écriture)	✓ avec un mock ou un spy
Un attribut préfixé <code>_</code>	×
L'ordre des appels internes	×
Le nombre d'itérations d'une boucle	×
Le type concret d'un objet interne	×

Le test décisif : *si je réécris entièrement l'intérieur de cette fonction en gardant le même contrat, mon test doit-il rester vert ?*

Si non, il teste l'implémentation.

À retenir

1. Le test énonce un **comportement**, pas une structure. C'est ce qui lui permet de survivre au refactoring qu'il est censé protéger.
2. Le test est votre premier consommateur : ce qui est pénible à écrire dans un test doit être corrigé dans la **signature**, pas dans le test.
3. Un cas limite ne mérite un test que s'il peut réellement se produire chez vos consommateurs.

3. Refactoring sécurisé

Changer la structure sans changer le comportement

Refactoring ou réécriture ?

	Refactoring	Réécriture
Comportement	inchangé, à chaque étape	reconstruit
Durée d'une étape	minutes	semaines
Le code compile et passe	en permanence	à la fin, on espère
Réversible	oui, à tout moment	non
Livrable intermédiaire	oui	aucun
Risque	maîtrisé	maximal

Une réécriture est un projet qui promet de retrouver dans un an les fonctionnalités d'aujourd'hui — y compris celles que personne n'a documentées, et que seuls les utilisateurs connaissent.

Le refactoring n'est pas plus lent. Il est simplement **livrable en continu**.

La règle de sécurité

1. Les tests sont verts

2. UNE transformation

3. Les tests sont verts

4. Commit

5. Retour en 2

← une seule, mécanique

Ce qui rend la méthode sûre, ce n'est pas la prudence : c'est la **taille du pas**. Quand le pas dure deux minutes, une erreur coûte deux minutes.

Signal d'alarme	Ce qu'il faut faire
« Je vais tout casser et je recolle après »	revenir au dernier commit
Rouge depuis plus de 10 minutes	<code>git stash</code> , recommencer plus petit
Trois transformations en même temps	les défaire, les refaire une par une
« Tant que j'y suis, je corrige ce bug »	non — c'est un autre commit

 **Piège** : corriger un bug pendant un refactoring rend impossible de savoir si les tests ont rougi à cause du refactoring ou de la correction.

Le catalogue utile

Sept transformations couvrent l'essentiel du travail quotidien :

Transformation	Symptôme traité
Extraire une fonction	un bloc de code avec un commentaire au-dessus
Extraire une classe	un groupe de champs et de méthodes qui vont ensemble
Introduire un objet paramètre	les mêmes 3 arguments partout
Remplacer une conditionnelle par un dispatch	une cascade de <code>elif</code>
Supprimer une duplication	le même bloc à trois endroits
Rendre une fonction pure	une fonction qui lit ou écrit un état global
Clarifier les erreurs	un <code>return None</code> , un code de retour <code>-1</code> , un <code>except: pass</code>

Les trois premières sont réalisables **automatiquement** par votre éditeur (VS Code, PyCharm) — donc sans risque de faute de frappe.

Extraire une fonction — le geste de base

Avant

```
def traiter(montants):  
    ...  
    # normaliser les unités  
    normalises = []  
    for m in montants:  
        if m.unite == "centimes":  
            normalises.append(m.valeur / 100)  
        else:  
            normalises.append(m.valeur)  
    ...
```

Après

```
def en_euros(montants) -> list[float]:  
    """Convertit tous les montants en euros."""  
    return [  
        m.valeur / 100 if m.unite == "centimes" else m.valeur  
        for m in montants  
    ]  
  
def traiter(mesures):  
    ...  
    normalisees = en_microgrammes(mesures)  
    ...
```

 **Le signal** : un commentaire qui décrit ce que fait le bloc suivant est presque toujours un **nom de fonction en attente**. Le commentaire disparaît, le nom le remplace, et le bloc devient testable seul.

Introduire un objet paramètre

Avant

```
def frais(poids, distance, mode): ...  
def delai(poids, distance, mode): ...  
def suivi(poids, distance): ...  
def assurance(poids, distance, valeur): ...
```

Les mêmes deux valeurs voyagent partout, sans jamais être validées ensemble.

Trois bénéfices immédiats : la validation est faite **une fois**, les signatures raccourcissent, et les tests construisent un objet nommé plutôt que d'aligner des nombres anonymes.

Après

```
@dataclass(frozen=True)  
class Colis:  
    poids_kg: float  
    distance_km: float  
  
    def __post_init__(self):  
        if self.poids_kg < 0:  
            raise ColisInvalide(...)  
  
def frais(colis: Colis, mode: str): ...  
def delai(colis: Colis, mode: str): ...
```

Remplacer une conditionnelle par un dispatch

C'est le refactoring qui transforme un `elif` interminable en code ouvert à l'extension — et chaque branche devient testable seule.

```
# Avant : rouvrir la fonction à chaque nouveau cas
if type_ == "csv": ...
elif type_ == "json": ...
elif type_ == "xml": ...

# Après : ajouter une entrée, ne jamais rouvrir le dispatch
LECTEURS = {"csv": lire_csv, "json": lire_json, "xml": lire_xml}

def lire(chemin: Path, type_: str) -> list[dict]:
    try:
        return LECTEURS[type_](chemin)
    except KeyError:
        raise FormatInconnu(type_) from None
```

 **Piège** : ne faites cette transformation que s'il y a **au moins trois branches** et qu'une quatrième est prévisible. Pour deux branches stables, le `if` est plus lisible.

Rendre une fonction pure

C'est le refactoring qui rapporte le plus en testabilité, pour le moins d'effort.

Avant

```
SEUIL = 50.0
_journal = []

def est_en_alerte(valeur):
    global SEUIL
    _journal.append(valeur)    # effet
    return valeur > SEUIL     # état global
```

Le résultat dépend d'un état invisible,
et l'appel modifie le monde.

Après

```
def est_en_alerte(valeur: float, seuil: float = 50.0) -> bool:
    """Vrai si la valeur dépasse strictement le seuil."""
    return valeur > seuil
```

Testable en une ligne, pour n'importe
quel seuil, dans n'importe quel ordre.

La journalisation remonte chez
l'appelant, où elle est visible.

Introduire un pattern pendant un refactoring

Un pattern ne s'introduit pas d'un bloc. Il **apparaît** au terme d'une suite de petites transformations, chacune validée par les tests :

```
1. Extraire une fonction par branche du `elif`      ← tests verts
2. Mettre les fonctions dans un dict                ← tests verts
3. Remplacer le `elif` par un accès au dict        ← tests verts
   ↓
   c'est un Strategy
```

```
1. Ajouter un paramètre `depot=None` (couture)      ← tests verts
2. Extraire les requêtes SQL dans une classe        ← tests verts
3. Déclarer un Protocol décrivant ses méthodes    ← tests verts
4. Écrire un fake en mémoire                       ← nouveaux tests possibles
   ↓
   c'est un Repository
```

 **En pratique** : si vous ne pouvez pas nommer le problème que le pattern résout **avant** de commencer, ne commencez pas. Le nom du pattern se met à la fin, dans le message de commit — pas au début, dans un plan d'architecture.

Ce que le refactoring ne doit jamais faire

Interdit pendant un refactoring	Pourquoi
Corriger un bug	on ne saura plus ce qui a fait rougir
Ajouter une fonctionnalité	idem, et le commit devient irrelisible
Changer une signature publique	ce n'est plus du refactoring, c'est une rupture d'API
Modifier un test pour le faire passer	c'est l'aveu que le comportement a changé
Reformater tout le fichier	le diff devient illisible en revue

La règle absolue : si un test rougit pendant un refactoring, **le refactoring est faux**. Ne modifiez pas le test — annulez la transformation.

C'est le seul moment du développement où cette règle est sans exception.

À retenir

1. Refactoring = comportement inchangé à **chaque étape**, avec un livrable possible en permanence. Ce n'est pas une réécriture menée prudemment.
2. La sécurité vient de la **taille du pas**, pas de la prudence. Deux minutes par transformation, un commit à chaque vert.
3. Un test qui rougit pendant un refactoring condamne le refactoring, jamais le test.

4. Traiter les méthodes longues

Démonter 300 lignes sans rien casser

Pourquoi 300 lignes coûtent cher

Ce n'est pas une question de goût. Trois coûts mesurables :

Coût	Mécanisme
Combinatoire	12 <code>if</code> imbriqués = jusqu'à 4 096 chemins. Vos tests en couvrent 5.
Mémoire de travail	on retient 5 à 7 éléments. Une méthode longue en manipule 40.
Portée des variables	une variable définie ligne 40 et relue ligne 280 : impossible de savoir ce qui l'a modifiée entre les deux

Et un quatrième, propre aux librairies :

Une méthode longue **ne peut pas être testée par morceaux**. Il faut fournir toutes ses dépendances pour vérifier une seule de ses règles — ce qui revient à monter un environnement de production pour tester une multiplication.

Le seuil qui compte

Pas un nombre de lignes : le moment où **vous devez faire défiler** pour comprendre le début.

Trois signaux plus fiables qu'un compteur :

Signal	Ce qu'il révèle
Des commentaires en bannière (<code># --- calcul ---</code>)	l'auteur a déjà identifié les blocs à extraire
Des variables temporaires réutilisées (<code>tmp, res, d</code>)	plusieurs traitements distincts partagent un espace
Des paramètres booléens	la méthode fait deux choses différentes selon l'appel

 **En pratique** : les commentaires en bannière sont un cadeau. Quelqu'un a fait le travail d'analyse ; il n'a simplement pas fait l'extraction.

La méthode, en six étapes

- | | |
|---------------------------------|---|
| 1. Caractériser | un test de bout en bout sur un jeu réel |
| ↓ | → « si je casse quelque chose, je le saurai » |
| 2. Cartographier | annoter les blocs : entrées, sorties, effets |
| ↓ | |
| 3. Extraire les fonctions pures | les blocs SANS effet de bord d'abord |
| ↓ | → chacun devient testable immédiatement |
| 4. Tester le noyau | les règles métier, maintenant accessibles |
| ↓ | |
| 5. Isoler les I/O | lecture au début, écriture à la fin |
| ↓ | |
| 6. Réduire l'orchestration | la méthode d'origine ne fait plus qu'appeler |

À chaque étape : tests verts, commit. Jamais deux étapes à la fois.

Étape 2 — Cartographier avant de couper

Pour chaque bloc, trois questions, notées dans la marge :

Question	Pourquoi elle compte
Qu'entre-t-il ?	ce seront les paramètres
Qu'en sort-il ?	ce sera le type de retour
Que modifie-t-il en dehors ?	s'il n'y a rien : bloc pur , extraction sans risque

```
def generer(self, jour):
    # bloc A - lit self.conn                → I/O, à isoler en 5
    # bloc B - entrée: lignes / sortie: mesures → PUR, à extraire en 3
    # bloc C - entrée: mesures / sortie: indice → PUR, à extraire en 3
    # bloc D - écrit un fichier              → I/O, à isoler en 5
    # bloc E - envoie un mail                → effet externe, à injecter
```

Le bloc C est celui qui porte le risque métier. C'est le premier à extraire et le premier à tester — pas le bloc A, qui est le plus visible.

Étape 3 — Extraire les fonctions pures d'abord

Avant — bloc C, ligne 74 à 100

```
def generer(self, jour):
    ...
    notes = []
    for p in SEUILS:
        if ligne.get(p) is not None:
            notes.append(ligne[p] / SEUILS[p])
    if notes:
        ligne["indice"] = round(
            min(100, max(notes) * 50), 1)
    else:
        ligne["indice"] = 0
    ...
```

Après

```
def calculer_indice(
    moyennes: dict[str, float | None],
    seuils: dict[str, float],
) -> float | None:
    """Indice global, ou None si aucune mesure."""
    notes = [
        moyennes[p] / seuils[p]
        for p in seuils
        if moyennes.get(p) is not None
    ]
    if not notes:
        return None # ← décision métier explicitée
    return round(min(100, max(notes) * 50), 1)
```

L'extraction a rendu visible une **décision métier cachée** : « pas de mesure »

valait 0, c'est-à-dire la meilleure note possible. Ce n'était pas un choix.

Étape 5 — Isoler les I/O aux extrémités

La forme cible d'un traitement, quelle que soit sa taille d'origine :

```
def generer(self, jour: date, sortie: Path) -> Rapport:
    """Orchestration : lire, calculer, écrire. Rien d'autre."""
    mesures = self._depot.du_jour(jour)      # ← une lecture, au début
    resultats = analyser(mesures, self._seuils) # ← tout le métier, pur
    ecrire_rapport(sortie, resultats)         # ← une écriture, à la fin
    return Rapport(jour, len(resultats))
```

```
lecture  → [ ===== métier pur ===== ] → écriture
1 ligne      testable sans rien du tout      1 ligne
```

 **En pratique** : visez cette forme même quand vous ne l'atteindrez pas complètement. Chaque ligne de calcul qui remonte dans la zone centrale est une ligne devenue testable.

Introduire un objet de configuration

Les méthodes longues accumulent les paramètres. Au-delà de quatre, regroupez.

Avant

```
def generer(self, jour=None, format="csv",
            envoyer=True, verbose=False,
            seuil=None, destinataires=None,
            couverture=3, strict=False):
```

Huit paramètres, quatre booléens.

Tout appel est illisible :

```
generer(j, "csv", False, True, ...)
```

Le test devient explicite : `OptionsRapport(envoyer=False, seuil=25.0)` .

Et le jour où une option s'ajoute, **aucune signature ne change**.

Après

```
@dataclass(frozen=True)
class OptionsRapport:
    format: str = "csv"
    envoyer: bool = True
    seuil: float | None = None
    destinataires: tuple[str, ...] = ()

def generer(self, jour: date,
            options: OptionsRapport = OptionsRapport()):
```

Vérifier la non-régression à chaque étape

Le filet posé à l'étape 1 doit être **exécuté après chaque transformation** — pas à la fin.

```
# La boucle, littéralement :
uv run pytest tests/caracterisation -q && git commit -am "extrait calculer_indice"
```

Trois protections complémentaires :

Protection	Détecte
Le test de caractérisation	tout changement de résultat
<code>git diff</code> avant commit	les modifications non intentionnelles
Comparer ancienne et nouvelle version	les écarts sur des entrées variées

```
def test_equivalence_ancienne_nouvelle():
    """Le temps du refactoring, on garde les deux et on les compare."""
    for releve in JEUX_DE_TEST:
        assert nouvelle(releve) == pytest.approx(ancienne(releve))
```

La dernière technique est particulièrement efficace sur du calcul scientifique.

À retenir

1. Extrayez les blocs **purs en premier** : ils deviennent testables immédiatement et sans risque, et ce sont eux qui portent le risque métier.
2. Un commentaire en bannière est un nom de fonction en attente. Le travail d'analyse a déjà été fait par quelqu'un.
3. L'extraction révèle des **décisions métier cachées** — c'est souvent le bénéfice principal, avant même la testabilité.

5. Dépendances circulaires et architecture

Deux modules qui s'importent sont un module mal découpé

Le symptôme

```
# taches.py
from affichage import en_ligne

# affichage.py
from taches import Tache
```

```
ImportError: cannot import name 'Tache' from partially initialized module
'taches' (most likely due to a circular import)
```

Les contournements qu'on voit dans le code réel :

Contournement	Ce qu'il cache
<code>import</code> au milieu d'une fonction	le cycle est toujours là, il est juste différé
<code>if TYPE_CHECKING:</code>	le cycle n'existe qu'au typage — acceptable
Tout fusionner dans un gros module	le cycle devient invisible et permanent
Réordonner les imports jusqu'à ce que ça marche	fragile : le prochain import cassera tout

Ce qu'un cycle signifie vraiment

Deux modules qui s'importent mutuellement **ne peuvent pas être compris séparément**, ni testés séparément, ni réutilisés séparément.

Ce sont donc, en pratique, **un seul module** — qui a été coupé au mauvais endroit.

Conséquences concrètes sur les tests :

- importer `taches` charge `affichage`, qui charge le moteur de rendu, qui charge la configuration... : le test le plus simple traîne tout le paquet ;
- un test d'un module fait échouer l'autre ;
- impossible d'extraire un sous-ensemble du package pour un autre projet.

Les quatre causes fréquentes

Cause	Exemple typique
Le module fourre-tout	<code>utils.py</code> importe la moitié du projet, et tout le monde l'importe
Objets métier ↔ services	<code>Tache</code> appelle <code>ServiceAffichage</code> pour se formater
Le module de constantes bavard	<code>config.py</code> importe des classes pour typer ses valeurs
Les imports de confort	<code>__init__.py</code> réexporte tout, créant des cycles indirects

Le cas le plus fréquent en pratique : un objet métier qui « sait s'afficher », « sait se sauvegarder » et « sait s'envoyer ». Il dépend alors du rendu, de la persistance et du réseau — qui dépendent tous de lui.

Détecter les cycles

```
# ruff – signale les imports dans les fonctions, symptôme fréquent
uv run ruff check --select PLC0415 .

# import-linter – déclare et vérifie une architecture en couches
uv pip install import-linter && uv run lint-imports
```

```
# .importlinter
[importlinter]
root_package = gestion

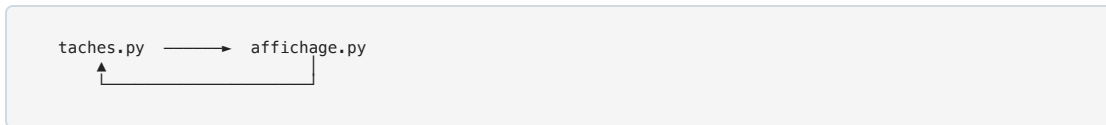
[importlinter:contract:couches]
name = Architecture en couches
type = layers
layers =
    gestion.service      # le plus haut
    gestion.projets | gestion.affichage
    gestion.modele       # le plus bas – ne dépend de rien
```

 **En pratique** : ce fichier vaut une documentation d'architecture, avec un avantage décisif — il **échoue en CI** quand quelqu'un enfreint la règle. Nous le brancherons demain dans la pipeline.

Résolution 1 — Extraire un module commun

C'est la solution qui marche dans 80 % des cas.

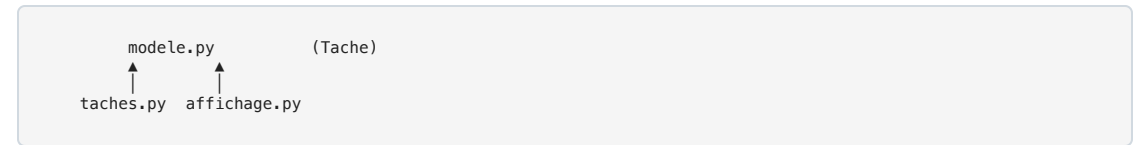
Avant — cycle



`affichage` a besoin de `Tache` .

`taches` a besoin de `en_ligne` .

Après — arbre



Les deux dépendent d'un troisième module qui, lui, ne dépend de rien.

La règle qui évite les cycles par construction : les objets de valeur

(`dataclass`, exceptions, constantes) vivent dans un module de base qui n'importe **rien** du projet. Tout le reste peut en dépendre.

Résolution 2 — Inverser la dépendance

Quand le module bas niveau a besoin d'appeler le haut niveau.

Avant

```
# rappels.py (bas niveau)
from notifications import envoyer_rappel

class Rappels:
    def verifier(self, taches, jour):
        for t in taches:
            if t.est_en_retard(jour):
                envoyer_rappel(t)
```

rappels dépend de notifications .

Après

```
# rappels.py — ne dépend plus de rien
class Notificateur(Protocol):
    def rappeler(self, tache: Tache) -> None: ...

class Rappels:
    def __init__(self, notif: Notificateur):
        self._notif = notif
```

Le Protocol est déclaré **du côté qui l'utilise**. C'est ce qui inverse le sens de la flèche.

Le point d'entrée assemble : `Rappels(NotificateurSMTP(...))` .

Résolution 3 — Retirer le comportement de l'objet métier

Avant — l'objet sait tout faire

```
# modele.py
from rendu import en_csv
from depot import sauver

@dataclass
class Tache:
    titre: str

    def to_csv(self): return en_csv(self)
    def sauver(self): sauver(self)
```

`modele` dépend de tout le projet.

Après — l'objet ne sait que se décrire

```
# modele.py - n'importe rien
@dataclass(frozen=True)
class Tache:
    titre: str

# rendu.py
def en_csv(tache: Tache) -> str: ...

# depot.py
def sauver(tache: Tache) -> None: ...
```

Le sens des flèches s'inverse : ce sont les services qui connaissent le modèle, jamais l'inverse.

⚠ **Piège** : « c'est de la POO, l'objet doit porter son comportement ».

Il porte son comportement **métier** (`est_en_retard`, `est_a_surveiller`),
pas sa sérialisation ni sa persistance.

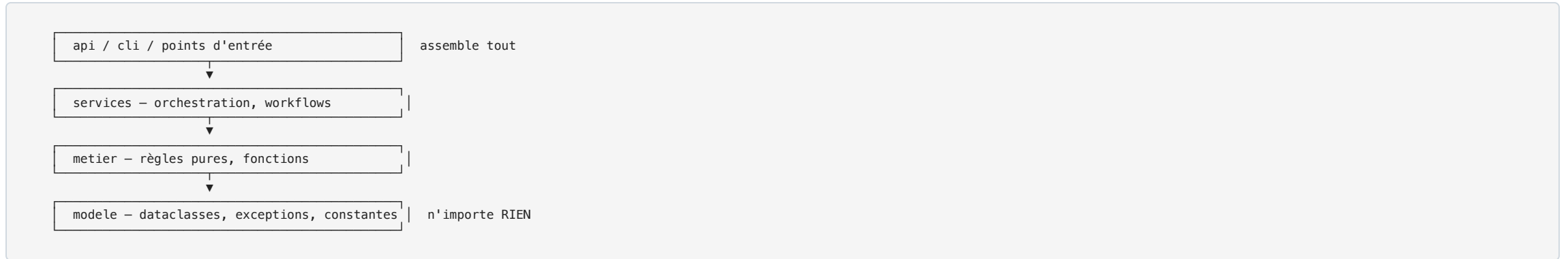
Récapitulatif des techniques

Situation	Technique	Effort
Deux modules partagent des types	extraire un module commun	faible
Le bas niveau appelle le haut niveau	inverser par un Protocol	moyen
Un objet métier fait du rendu ou de l'I/O	sortir le comportement	moyen
Deux modules s'appellent en alternance	service applicatif au-dessus des deux	moyen
Un couplage ponctuel et rare	passer la dépendance en paramètre	très faible
Notification entre modules sans lien	événements — un module publie, l'autre écoute	élevé

 **En pratique** : essayez les deux premières lignes avant tout le reste.

Les événements sont une solution puissante et souvent surdimensionnée : ils remplacent un cycle visible par un couplage invisible.

La cible : un graphe sans cycle



Les flèches ne vont que **vers le bas**. Les adaptateurs (base, API, fichiers) se branchent au niveau `services`, jamais dans `metier` ni `modele`.

À retenir

1. Un cycle d'import n'est pas un problème technique : c'est un découpage fait au mauvais endroit. Les contournements le masquent sans le régler.
2. La solution la plus fréquente est la plus simple : **extraire un module de base** que tout le monde importe et qui n'importe rien.
3. `import-linter` transforme votre schéma d'architecture en test exécutable. C'est la seule documentation d'architecture qui reste vraie.

6. Stratégie de reprise d'une base existante

Par où commencer quand tout est à faire

La question de lundi matin

Vous héritez d'une librairie de 12 000 lignes, sans tests, utilisée par quatre projets. Vous avez, disons, deux jours par mois à y consacrer.

Les deux réponses qui échouent :

Réponse	Pourquoi elle échoue
« On réécrit tout proprement »	18 mois, aucun livrable, et les projets consommateurs continuent d'évoluer pendant ce temps
« On vise 80 % de couverture »	6 semaines de tests sur le code facile — c'est-à-dire celui qui ne cassait jamais

La réponse qui fonctionne tient en une phrase :

Testez ce que vous vous apprêtez à modifier, et rien d'autre.

Le reste viendra quand il faudra y toucher — et pas avant.

Cartographier les zones critiques

Une demi-journée, en équipe, sur trois sources d'information :

Source	Question	Où la trouver
L'historique Git	quels fichiers changent le plus ?	<code>git log</code>
Les incidents	où sont apparus les bugs ?	tickets, mails, mémoire de l'équipe
Les consommateurs	quelles fonctions sont appelées par d'autres projets ?	<code>grep</code> dans les projets clients

```
# Les 15 fichiers les plus modifiés sur deux ans
git log --since="2 years ago" --name-only --pretty=format: \
  | grep '\.py$' | sort | uniq -c | sort -rn | head -15
```

Un fichier souvent modifié **et** cité dans les incidents **et** appelé par trois projets : c'est là que vous commencez. Pas ailleurs.

La matrice de priorité

	Rarement modifié	Souvent modifié
Conséquence forte	② tests de caractérisation, sans refactoring	① commencer ici
Conséquence faible	④ ne rien faire	③ tester au fil de l'eau

Ce que chaque case veut dire concrètement :

- ① filet complet, puis refactoring, puis tests unitaires sur les règles ;
- ② quelques tests de caractérisation, pour la seule non-régression ;
- ③ règle d'équipe : on ajoute les tests quand on touche au fichier ;
- ④ rien. Ce code coûte moins cher à ignorer qu'à tester.

 **Piège** : la case ④ met mal à l'aise. Elle est pourtant ce qui rend le plan réaliste — et donc ce qui permet aux cases ① et ② d'aboutir.

La règle du boy-scout, en version testable

Vous laissez le code un peu plus propre que vous ne l'avez trouvé.

Traduite en règle d'équipe applicable :

Quand vous...	Vous ajoutez...
corrigez un bug	le test qui échouait avant la correction
ajoutez une fonctionnalité	ses tests, écrits avant (bourgeon)
lisez du code pour le comprendre	le test qui documente ce que vous venez de comprendre
touchez une fonction non testée	un test de caractérisation, même minimal

L'effet est cumulatif et se concentre **naturellement** sur le code qui bouge — c'est-à-dire exactement celui qui casse.

Aucune de ces quatre lignes ne demande de budget dédié. Elles s'appliquent pendant le travail déjà planifié.

La couverture : ce qu'elle sait et ne sait pas

Elle sait	Elle ne sait pas
quelles lignes n'ont jamais été exécutées	si les assertions vérifient quoi que ce soit
si un module entier est à 0 %	si les cas limites sont couverts
si votre nouveau code est testé (couverture du diff)	si les tests sont justes

```
def diviser(a, b):  
    return a / b  
  
def test_diviser():  
    diviser(10, 2)    # 100 % de couverture, zéro assertion
```

 **En pratique** : sur du legacy, l'indicateur utile n'est pas la couverture globale — elle démarrera à 4 % et humiliera tout le monde. C'est la **couverture des lignes modifiées** dans la merge request. Nous la mettrons en place demain.

Un plan réaliste sur six mois

Mois	Objectif	Effort
1	Cartographie, pipeline CI qui lance <code>pytest</code> , même sur 3 tests	2 j
1-2	Test de bout en bout sur les 2 traitements les plus critiques	3 j
2-3	Caractérisation + extraction des règles métier de la zone ①	5 j
3-4	Règle d'équipe : aucun correctif sans test de non-régression	0 j
4-6	Seuil de couverture sur le diff en merge request	1 j

Ce qui rend ce plan tenable :

- il produit un résultat visible dès le premier mois ;
- il ne demande jamais d'arrêter les évolutions fonctionnelles ;
- il ne fixe **aucun objectif de couverture globale**.

Les indicateurs qui valent la peine d'être suivis

Indicateur	Ce qu'il dit	Piège s'il devient un objectif
Couverture du diff	le code neuf est-il testé ?	peut se contourner par des tests vides
Nombre de bugs revenus	les non-régressions font-elles leur travail ?	—
Durée de la suite	reste-t-elle lançable en local ?	—
Tests instables (<i>flaky</i>)	y a-t-il de l'état partagé ?	—
Couverture globale	peu de choses sur du legacy	beaucoup de tests sans valeur

La loi de Goodhart, version test : dès qu'une métrique devient un objectif, elle cesse d'être une bonne métrique. Une équipe à qui l'on impose 80 % de couverture atteindra 80 % — et vous n'aurez rien gagné.

À retenir

1. Ne testez pas la base existante « en général ». Testez **ce que vous vous apprêtez à modifier** — le reste attendra son tour légitimement.
2. La matrice conséquence × fréquence donne l'ordre de travail, et le quadrant « ne rien faire » est ce qui rend le plan réaliste.
3. Sur du legacy, l'indicateur utile est la couverture du **diff**, pas la couverture globale.

Travaux pratiques du Jour 4

Quatre exercices indépendants — 4 h 15

Le programme

TP	Ce que vous allez faire	Durée
j4-01	Ajouter un historique avec annulation à une calculatrice de 115 lignes en TDD , par bourgeon — sans réécrire le moteur.	60 min
j4-02	Mettre sous test un générateur de bulletin scolaire de 190 lignes par un test d'équivalence, puis l'extraire bloc par bloc. Le TP le plus long, et le plus proche du quotidien.	90 min
j4-03	Faire émerger Strategy, injection et Factory dans un module de rappels par une suite de petites transformations, chacune validée par les tests.	60 min
j4-04	Diagnostiquer et casser le cycle <code>taches ↔ projets ↔ affichage</code> d'une todo-list, puis verrouiller l'architecture avec <code>import-linter</code> .	45 min

La discipline de la journée

Tests verts → une transformation → tests verts → commit.

Sur les TPs 02 et 03, engagez-vous vraiment à committer à chaque étape. Vous mesurerez la différence quand une transformation tournera mal — et il y en aura une.

🎯 Si vous êtes rouge depuis plus de 10 minutes, ne cherchez pas la cause : revenez au dernier commit et recommencez avec un pas plus petit. C'est contre-intuitif, et c'est plus rapide.

Sur `j4-02`, résistez à la tentation de corriger les bugs que vous allez trouver. Notez-les dans un fichier `BUGS.md` — nous en discuterons en fin de séance. Un refactoring qui corrige des bugs en même temps n'est plus vérifiable.

1. Rôle de la CI/CD dans la qualité logicielle

Ce qui change quand la machine exécute les tests

Ce que la CI change vraiment

Vos tests existent déjà. Ce que la pipeline ajoute n'est pas technique :

Sans CI	Avec CI
« J'ai lancé les tests »	On voit qu'ils ont été lancés
Sur le poste de celui qui a écrit le code	Sur un environnement neutre et reproductible
Avec les dépendances installées il y a six mois	Avec celles déclarées dans <code>pyproject.toml</code>
Quand on y pense	À chaque poussée, sans exception
Le relecteur croit sur parole	Le relecteur voit un état vert ou rouge

La CI ne rend pas les tests meilleurs. Elle rend leur exécution **non négociable** — et transforme une intention d'équipe en fait vérifiable.

Test local et test en pipeline : les écarts

Un test qui passe chez vous et échoue en CI révèle presque toujours une **dépendance implicite** de votre poste :

Écart	Ce qu'il révèle
Un paquet installé chez vous, absent du <code>pyproject.toml</code>	dépendance non déclarée — vos utilisateurs auront le même problème
Un fichier présent chez vous, absent du dépôt	donnée de test non versionnée
Une variable d'environnement locale	configuration non documentée
Un ordre d'exécution différent	état partagé entre tests
Un chemin absolu, un séparateur <code>\</code> ou <code>/</code>	code non portable
Un fuseau horaire ou une locale	calcul dépendant de l'environnement

 **En pratique** : ces échecs sont désagréables et **précieux**. Chacun est un bug que vos consommateurs auraient rencontré à l'installation.

Le coût du délai de détection

Le même défaut, selon le moment où il est trouvé :

à l'écriture	→ 30 secondes	le contexte est en tête
en CI, sur la MR	→ 10 minutes	il faut relire son diff
en revue	→ 1 heure	deux personnes mobilisées
après fusion	→ 1 demi-journée	bisect, correctif, nouvelle MR
chez un consommateur	→ 1 semaine	diagnostic à distance, version à publier
en production	→ ?	+ le coût des résultats faux diffusés

Chaque ligne coûte environ **dix fois** la précédente. C'est tout l'argument économique de la CI, et il ne dépend pas de la taille de l'équipe.

Quand la pipeline doit bloquer

Bloquer trop peu ne sert à rien ; bloquer trop pousse l'équipe à contourner.

Contrôle	Bloque ?	Pourquoi
Tests unitaires en échec	oui	c'est la raison d'être de la pipeline
Erreur de lint (<code>ruff check</code>)	oui	déterministe, corrigeable en une commande
Formatage non conforme	oui	zéro discussion possible en revue
Typage <code>mypy</code> sur du code neuf	oui, progressivement	commencez par les modules récents
Couverture du diff sous le seuil	oui	garde-fou sur le code neuf
Couverture globale sous un seuil	non	pousse à écrire des tests sans valeur
Tests d'intégration lents	selon	dans un stage séparé, non bloquant au début
Avertissements de sécurité	selon la criticité	à trier, pas à ignorer en bloc

Une politique qui tient dans le temps

Trois principes, à décider **en équipe** et à écrire quelque part :

1. Ce qui bloque doit être réparable en moins de dix minutes.

Sinon, l'équipe apprend à contourner — et vous perdez tout le bénéfice.

2. Un contrôle nouveau n'est jamais bloquant le premier jour.

On l'ajoute en avertissement, on résorbe le passif, puis on bloque.

3. Une pipeline rouge est traitée avant tout autre travail.

Une pipeline rouge tolérée pendant trois jours cesse d'être un signal.

Le troisième point est le plus difficile — et celui qui détermine si la CI sert à quelque chose au bout de six mois.

Tests rapides et tests lents

Une suite qui dure vingt minutes n'est plus lancée en local. C'est un problème d'équipe avant d'être un problème technique.

À chaque poussée	lint + format + unitaires	< 2 min
Sur la merge request	+ intégration + couverture	< 10 min
Chaque nuit	+ tests lents, matrice de versions, service réel	illimité

C'est le découpage préparé au Jour 2 avec les marqueurs :

```
pytest -m "not integration and not lent" # boucle rapide
pytest                                  # tout, sur la MR
```

Les pièges classiques

Piège	Conséquence	Correctif
Tests instables (<i>flaky</i>)	l'équipe relance jusqu'au vert, puis ignore les rouges	traquer l'état partagé, jamais « relancer »
Pipeline de 40 minutes	plus personne n'attend, on fusionne à l'aveugle	découper en stages, mettre en cache
Seuil de couverture trop haut d'un coup	tests vides écrits pour la métrique	seuil sur le diff , relevé progressivement
Secrets en clair dans le <code>.gitlab-ci.yml</code>	fuite dans l'historique Git	variables masquées et protégées
Un seul job qui fait tout	on ne sait pas ce qui a échoué	un job par nature de contrôle
Job vert alors que la commande a échoué	fausse confiance	vérifier les codes de retour, pas les logs

⚠ **Piège** : le dernier est le plus dangereux, parce qu'il est invisible.

Nous en avons rencontré un exemple réel en préparant ce support.

À retenir

1. La CI ne rend pas les tests meilleurs : elle rend leur exécution **non négociable** et visible par toute l'équipe.
2. Un test qui passe en local et casse en CI a trouvé une dépendance implicite de votre poste — c'est un bug de votre package, pas de la pipeline.
3. Ce qui bloque doit être réparable en dix minutes, et une pipeline rouge se traite avant tout autre travail. Sinon la CI devient du bruit.

2. GitLab CI/CD pour un projet Python

Du `.gitlab-ci.yml` minimal à la pipeline complète

Le vocabulaire, en une slide

Terme	Ce que c'est
Pipeline	l'ensemble déclenché par une poussée ou une merge request
Stage	une étape ; les stages s'exécutent en séquence
Job	une unité de travail ; les jobs d'un même stage s'exécutent en parallèle
Runner	la machine qui exécute les jobs
Image	le conteneur Docker dans lequel le job tourne
Artifact	un fichier produit par un job, conservé et transmis
Cache	des fichiers réutilisés d'une pipeline à l'autre, pour la vitesse

Artifact ou cache ? L'artifact est un **résultat** qu'on veut consulter (rapport de tests, paquet). Le cache est un **moyen** d'aller plus vite (dépendances). Perdre un cache est sans conséquence ; perdre un artifact, non.

La pipeline minimale qui marche

```
# .gitlab-ci.yml
image: python:3.12-slim

test:
  script:
    - pip install -e ".[dev]"
    - pytest
```

Six lignes. C'est déjà mieux que rien, et c'est le bon point de départ pour une équipe qui n'a pas encore de CI.

 **En pratique** : mettez ceci en place **cette semaine**, même si votre suite ne compte que trois tests. Une pipeline qui existe s'améliore ; une pipeline parfaite qu'on prévoit d'écrire n'existe jamais.

Les slides suivantes ajoutent, dans l'ordre : les stages, le cache, les rapports, la matrice de versions.

Structurer en stages

```
stages:
  - qualite
  - test
  - packaging

format:
  stage: qualite
  script:
    - ruff format --check .

lint:
  stage: qualite
  script:
    - ruff check .

types:
  stage: qualite
  script:
    - mypy src

tests:
  stage: test
  script:
    - pytest -m "not lent"
```

Les trois jobs de `qualite` tournent **en parallèle** ; `test` ne démarre que s'ils sont tous verts. Un échec de lint n'attend pas 4 minutes de tests.

On voit parfois un stage `installation` séparé. Avec `uv sync --frozen` en `before_script` et un cache, l'installation est refaite dans chaque job en quelques secondes — plus simple qu'un stage qui devrait transmettre un environnement complet par artifact.

uv dans la pipeline

```
image: python:3.12-slim

variables:
  UV_CACHE_DIR: .uv-cache
  UV_LINK_MODE: copy      # évite les avertissements de lien dur en conteneur

.avec_uv: &avec_uv
  before_script:
    - pip install --no-cache-dir uv
    - uv sync --frozen      # installe EXACTEMENT le uv.lock du dépôt
  cache:
    key:
      files: [uv.lock]      # nouvelle clé quand les dépendances changent
      paths: [.uv-cache]
  tests:
    <<: *avec_uv
    stage: test
    script:
      - uv run pytest
```

--frozen est le point important : il échoue si `uv.lock` ne correspond plus à `pyproject.toml` . Sans lui, la CI peut installer des versions différentes des vôtres — et vous ne testez plus la même chose.

Le cache des dépendances

Sans cache, chaque job réinstalle tout. Sur un projet avec `numpy`, `pandas` et `scipy`, cela représente **deux à trois minutes par job**.

```
cache:
  key:
    files:
      - uv.lock          # la clé change quand le verrou change
    paths:
      - .uv-cache
  policy: pull-push      # pull seul pour les jobs qui n'installent rien
```

Situation	Effet
<code>uv.lock</code> inchangé	cache réutilisé — installation en quelques secondes
<code>uv.lock</code> modifié	nouvelle clé, cache reconstruit une fois
Cache absent ou corrompu	le job réinstalle, sans échouer

 **Piège** : ne mettez **jamais** le `.venv` en cache entre des images différentes. Les chemins absolus et les binaires compilés ne sont pas portables. Mettez en cache le **répertoire de téléchargement**, pas l'environnement.

Variables et secrets

```
variables:
  PYTHONDONTWRITEBYTECODE: "1"
  PIP_DISABLE_PIP_VERSION_CHECK: "1"
  UV_CACHE_DIR: .uv-cache
```

Ce sont des variables **de configuration**, versionnées : aucun secret ici.

Les secrets se déclarent dans l'interface GitLab
(*Settings* → *CI/CD* → *Variables*), avec deux options qui comptent :

Option	Effet
Masked	la valeur est remplacée par [MASKED] dans les journaux
Protected	la variable n'est exposée qu'aux branches protégées

 **Piège** : un secret écrit dans le `.gitlab-ci.yml` reste dans l'historique Git **même après suppression**. Le retirer du fichier ne suffit pas : il faut le révoquer.

Contrôler ce qui déclenche quoi avec **rules**

```
tests:
  stage: test
  script: [uv run pytest -m "not lent"]
  rules:
    - if: $CI_PIPELINE_SOURCE == "merge_request_event"
    - if: $CI_COMMIT_BRANCH == $CI_DEFAULT_BRANCH

tests_lents:
  stage: test
  script: [uv run pytest -m lent]
  rules:
    - if: $CI_PIPELINE_SOURCE == "schedule"      # la nuit
    - if: $CI_COMMIT_BRANCH == $CI_DEFAULT_BRANCH
      when: manual                               # ou à la demande
    allow_failure: true
```

Variable GitLab	Contient
<code>\$CI_PIPELINE_SOURCE</code>	<code>push</code> , <code>merge_request_event</code> , <code>schedule</code> , <code>web</code>
<code>\$CI_COMMIT_BRANCH</code>	la branche courante
<code>\$CI_DEFAULT_BRANCH</code>	<code>main</code> , en général
<code>\$CI_MERGE_REQUEST_IID</code>	le numéro de la MR

Tester plusieurs versions de Python

Pour une librairie interne consommée par des projets qui n'ont pas tous la même version de Python, c'est le contrôle le plus utile de toute la pipeline.

```
tests:
  stage: test
  image: python:$VERSION_PYTHON-slim
  parallel:
    matrix:
      - VERSION_PYTHON: ["3.10", "3.11", "3.12", "3.13"]
  script:
    - pip install --no-cache-dir uv
    - uv sync --frozen
    - uv run pytest
```

Quatre jobs en parallèle, un par version. Vous découvrez immédiatement qu'une syntaxe `match` ne passe pas en 3.9, ou qu'un `StrEnum` exige 3.11.

 **En pratique** : la matrice doit refléter les versions **réellement** utilisées par vos consommateurs — pas les versions supportées en amont.

Le stage **packaging**

```
build:
  stage: packaging
  script:
    - uv build                # produit dist/*.whl et dist/*.tar.gz
    - uv run twine check dist/* # vérifie les métadonnées
  artifacts:
    paths: [dist/]
    expire_in: 1 week
  rules:
    - if: $CI_COMMIT_TAG      # uniquement sur une version taguée
```

Deux contrôles que ce job apporte, et que les tests ne donnent pas :

- le paquet **se construit** — un `pyproject.toml` cassé est détecté ;
- le paquet **contient ce qu'il faut** — c'est ici qu'on découvre un fichier de données oublié, grâce au layout `src/` vu au Jour 2.

Une pipeline complète, commentée

```
stages: [qualite, test, rapport]

image: python:3.12-slim
variables:
  UV_CACHE_DIR: .uv-cache
  UV_LINK_MODE: copy

default:
  before_script:
    - pip install --no-cache-dir uv
    - uv sync --frozen
  cache:
    key: {files: [uv.lock]}
    paths: [.uv-cache]

format:      {stage: qualite, script: [uv run ruff format --check .]}
lint:        {stage: qualite, script: [uv run ruff check .]}
types:       {stage: qualite, script: [uv run mypy src]}
architecture: {stage: qualite, script: [uv run lint-imports]}

tests:
  stage: test
  script:
    - uv run pytest --junitxml=rapport.xml --cov=src --cov-report=xml
  artifacts:
    when: always
    reports:
      junit: rapport.xml
      coverage_report: {coverage_format: cobertura, path: coverage.xml}
```

Débogage : les erreurs les plus fréquentes

Symptôme	Cause habituelle
<code>ModuleNotFoundError</code> sur votre propre package	pas d'installation ; il faut <code>uv sync</code> OU <code>pip install -e .</code>
Les tests passent en local, pas en CI	dépendance non déclarée, ou état partagé entre tests
<code>Permission denied</code> sur le cache	<code>UV_LINK_MODE: copy</code> manquant
Job vert alors que la commande a échoué	commandes chaînées par <code>;</code> au lieu de <code>&&</code>
<code>uv.lock</code> en conflit à chaque merge	verrou non régénéré après modification des dépendances
Pipeline très lente	pas de cache, ou un seul job qui fait tout

 **Le quatrième** est le plus insidieux. GitLab n'échoue que si la **dernière** commande d'un `script` renvoie un code non nul — sauf si vous les séparez par des lignes distinctes, ce que fait la syntaxe YAML en liste. Utilisez toujours une commande par ligne.

Valider son `.gitlab-ci.yml` avant de pousser

Trois moyens, du plus rapide au plus fidèle :

```
# 1. Syntaxe YAML seule — instantané
python -c "import yaml, pathlib; yaml.safe_load(pathlib.Path('.gitlab-ci.yml').read_text())"

# 2. Validation par GitLab — sans pousser
#   Interface : CI/CD → Editor → onglet « Validate »
#   Ou l'API : POST /api/v4/projects/:id/ci/lint

# 3. Exécuter localement les mêmes commandes que les jobs
uv run ruff check . && uv run mypy src && uv run pytest
```

 **En pratique** : le troisième moyen est le plus utile au quotidien.

Une pipeline ne doit contenir **aucune commande** que vous ne puissiez lancer sur votre poste — sinon vous déboguez à l'aveugle, une poussée à la fois.

À retenir

1. Commencez par six lignes qui lancent `pytest`. Une pipeline qui existe s'améliore ; celle qu'on prévoit d'écrire parfaitement n'arrive jamais.
2. `uv sync --frozen` garantit que la CI installe exactement ce que décrit `uv.lock` — sans quoi vous ne testez pas la même chose que vos consommateurs installeront.
3. Toute commande de la pipeline doit être exécutable sur votre poste. C'est la seule façon de déboguer autrement qu'en poussant des commits.

3. Rapports de tests

Passer des journaux bruts à une information exploitable

Journal brut ou rapport structuré

Le journal du job

```
===== FAILURES =====
___ test_seuil_inclusif ___
  def test_seuil_inclusif():
>     assert est_eligible(...) is True
E     assert False is True
=== 1 failed, 632 passed in 0.9s ===
```

Il faut ouvrir le job, dérouler
2 000 lignes, et chercher.

Le rapport structuré apporte trois choses que le journal ne donne pas : la **visibilité** sans ouvrir le job, la distinction **nouvel échec / échec existant**, et l'historique d'un test dans le temps.

Le rapport JUnit dans la MR

GitLab affiche directement, en tête
de la merge request :

✖ 1 test en échec

test_seuil_inclusif

nouvel échec

Un clic donne la trace complète.

Produire un rapport JUnit

```
pytest --junitxml=rapport.xml
```

```
tests:
  stage: test
  script:
    - uv run pytest --junitxml=rapport.xml
  artifacts:
    when: always          # ← indispensable : sinon rien en cas d'échec
    reports:
      junit: rapport.xml
    expire_in: 30 days
```

⚠ **Le piège classique** : sans `when: always`, GitLab ne récupère les artifacts que si le job réussit. Or c'est **précisément quand il échoue** que vous avez besoin du rapport. Cette ligne est la plus importante de la slide.

Enrichir le rapport avec des propriétés utiles :

```
pytest --junitxml=rapport.xml -o junit_family=xunit2 -o junit_logging=all
```

La couverture dans la merge request

```
pytest --cov=src --cov-report=xml --cov-report=term
```

```
tests:
  script:
    - uv run pytest --cov=src --cov-report=xml --cov-report=term
  coverage: '/TOTAL.*\s+(\d+)%$/' # extrait le pourcentage du journal
  artifacts:
    when: always
    reports:
      coverage_report:
        coverage_format: cobertura
        path: coverage.xml
```

Deux affichages distincts en résultent :

Où	Quoi
Badge de la MR	le pourcentage global, et son évolution
Diff de la MR	chaque ligne ajoutée est marquée couverte ou non

Le second est le seul vraiment utile : il montre **si le code de cette MR** est testé, indépendamment du passif du projet.

Conserver et transmettre les artifacts

```
tests:
  artifacts:
    when: always
    expire_in: 30 days
    paths:
      - rapport.xml
      - htmlcov/          # rapport HTML de couverture, navigable
    reports:
      junit: rapport.xml
  analyse:
    stage: rapport
    needs: [tests]        # récupère automatiquement les artifacts de `tests`
    script:
      - python outils/comparer_couverture.py coverage.xml
```

Réglage	Effet
expire_in	durée de conservation — au-delà, GitLab supprime
paths	fichiers téléchargeables depuis l'interface
reports	fichiers interprétés par GitLab (JUnit, couverture)
needs	dépendance explicite entre jobs, et transfert d'artifacts

Lire un échec de pipeline efficacement

L'ordre qui fait gagner du temps :

- | | |
|--------------------------------|--|
| 1. L'onglet « Tests » de la MR | quel test, et est-il nouveau ? |
| ↓ | |
| 2. Le nom du test | quelle règle métier est en cause ? |
| ↓ | |
| 3. L'assertion | quelle valeur, quel écart ? |
| ↓ | |
| 4. Reproduire en local | <code>pytest tests/...:test_... -x</code> |
| ↓ | |
| 5. Le journal complet du job | seulement si les 4 précédents ne suffisent pas |

 **En pratique** : si vous devez systématiquement aller jusqu'à l'étape 5, vos noms de tests ou vos assertions sont trop pauvres. C'est un défaut de **vos tests**, que la CI rend simplement visible.

C'est le retour direct des conventions du Jour 2 : un test = un comportement, un nom qui décrit ce comportement, une assertion sur une valeur.

Ce qu'il ne faut pas attendre d'un rapport

Attente	Réalité
« Le rapport nous dira si le code est bon »	il dit si vos tests passent
« 100 % de couverture = pas de bug »	la couverture ne juge aucune assertion
« Le badge motivera l'équipe »	il motive surtout à écrire des tests faciles
« On archivera tous les rapports »	<code>expire_in</code> existe pour une raison : le stockage

Ce que le rapport apporte réellement, et qui est déjà beaucoup :

- une **preuve** partagée que la suite a tourné, sur un environnement neutre ;
- la distinction entre un échec **nouveau** et un échec **connu** ;
- une trace consultable **sans relancer** quoi que ce soit ;
- un point d'appui factuel en revue, à la place d'une discussion d'opinion.

À retenir

1. `when: always` sur les artifacts. Sans cette ligne, vous perdez le rapport exactement quand vous en avez besoin.
2. La couverture qui compte est celle du **diff**, affichée ligne à ligne dans la merge request — pas le pourcentage global.
3. Si lire un échec exige d'ouvrir le journal complet, le problème est dans le nommage de vos tests, pas dans l'outil.

4. Contrôles complémentaires

Ce qu'on automatise à côté des tests

Les cinq contrôles, et ce qu'ils attrapent

Contrôle	Attrape	Coût
<code>ruff format</code>	débats de mise en forme en revue	quelques secondes
<code>ruff check</code>	imports inutilisés, variables mortes, pièges courants	quelques secondes
<code>mypy</code>	incohérences de types, <code>None</code> non géré	dizaines de secondes
<code>pytest-cov</code>	code neuf jamais exécuté	+20 % sur la suite
<code>import-linter</code>	cycles et violations d'architecture	quelques secondes

Aucun ne remplace un test. Tous attrapent des défauts qu'un test ne verrait qu'indirectement — et pour un coût très inférieur.

 **L'ordre d'adoption qui fonctionne** : formatage, puis lint, puis architecture, puis typage, puis couverture. Du moins discutable au plus discutable.

ruff — formatage et lint

```
# pyproject.toml
[tool.ruff]
line-length = 100
target-version = "py312"
extend-exclude = ["**/migrations"]

[tool.ruff.lint]
select = ["E", "F", "I", "UP", "B", "SIM"]
```

Famille	Ce qu'elle apporte
E, F	erreurs et avertissements de base (ex- <code>pycodestyle</code> , <code>pyflakes</code>)
I	tri automatique des imports (ex- <code>isort</code>)
UP	modernisation de la syntaxe selon la version cible
B	pièges réels — dont l'argument par défaut mutable

```
uv run ruff format .           # applique
uv run ruff format --check .   # vérifie, pour la CI
uv run ruff check --fix .      # corrige ce qui est corrigeable
```

Un seul outil remplace `black`, `isort`, `flake8` et une partie de `pylint` — pour un temps d'exécution de l'ordre de la centaine de millisecondes.

mypy — l'adoption progressive

Activer `strict` sur une base existante produit des milliers d'erreurs et une équipe démotivée. L'adoption se fait **module par module**.

```
[tool.mypy]
python_version = "3.12"
ignore_missing_imports = true      # les dépendances non typées ne bloquent pas

# Le socle, écrit récemment : exigeant
[[tool.mypy.overrides]]
module = ["mesures.modele", "mesures.calculs"]
strict = true

# Le legacy : on vérifie ce qui est déjà annoté, sans plus
[[tool.mypy.overrides]]
module = ["mesures.ancien_moteur"]
ignore_errors = true
```

 **En pratique** : commencez par les modules du bas de l'architecture —

`modele`, `calculs`. Ce sont les mieux découpés, les plus faciles à typer, et ceux dont tout le reste dépend.

pytest-cov — mesurer sans se mentir

```
[tool.coverage.run]
source = ["src"]
branch = true           # couvre aussi les branches, pas que les lignes

[tool.coverage.report]
exclude_also = [
    "if TYPE_CHECKING:",
    "raise NotImplementedError",
    "if __name__ == '__main__':",
]
```

```
uv run pytest --cov=src --cov-report=term-missing
```

Name	Stmts	Miss	Branch	BrPart	Cover	Missing
src/mesures/calculs.py	42	2	14	1	95%	58, 71->74

La colonne **Missing** est la seule vraiment exploitable : elle donne les lignes et les branches à regarder. Le pourcentage, lui, ne dit rien d'actionnable.

Le seuil de couverture : sur le diff, pas sur le projet

Seuil global ✗

```
pytest --cov=src --cov-fail-under=80
```

Sur du legacy à 12 %, la pipeline est rouge dès le premier jour. L'équipe désactive le contrôle — ou écrit des tests sans assertion.

🎯 **En pratique** : `--cov-fail-under` reste utile avec un seuil réglé **juste en dessous** de la valeur actuelle. Il n'impose pas de progresser, il empêche de **régresser** — c'est un cliquet, pas un objectif.

Seuil sur le diff ✓

Dans GitLab : *Settings* → *Merge requests* → *Merge checks*, ou via l'affichage ligne à ligne du diff.

On n'exige rien du passé. On exige que le **code neuf** soit testé.

pre-commit — attraper avant de pousser

```
# .pre-commit-config.yaml
repos:
- repo: https://github.com/astral-sh/ruff-pre-commit
  rev: v0.16.5
  hooks:
    - id: ruff-format
    - id: ruff
      args: [--fix]
- repo: https://github.com/pre-commit/pre-commit-hooks
  rev: v5.0.0
  hooks:
    - id: trailing-whitespace
    - id: end-of-file-fixer
    - id: check-yaml
    - id: check-added-large-files
```

```
uv run pre-commit install      # une fois par poste
```

Le formatage et le lint sont corrigés **avant** le commit : la pipeline ne rougit plus jamais pour une virgule.

 **Piège** : `pre-commit` ne remplace pas la CI. Il s'installe poste par poste et peut être contourné (`--no-verify`). Les mêmes contrôles doivent rester dans la pipeline.

Choisir des indicateurs utiles

Indicateur utile	Ce qu'il pilote
Le code neuf est-il testé ?	couverture du diff
Un bug corrigé peut-il revenir ?	présence d'un test de non-régression dans la MR
La suite reste-t-elle lançable en local ?	durée des tests unitaires
L'architecture tient-elle ?	<code>lint-imports</code> en CI

Métrique cosmétique	Ce qu'elle produit
Couverture globale imposée	des tests sans assertion
Nombre de tests	des tests dupliqués
Nombre de lignes par fonction, en dur	des fonctions découpées arbitrairement

Chaque ligne du second tableau a été observée dans des équipes réelles, à qui l'on avait fixé la métrique comme objectif.

À retenir

1. Adoptez dans cet ordre : formatage, lint, architecture, typage, couverture. Chaque contrôle n'est rendu bloquant qu'après résorption du passif.
2. `mypy` s'active **module par module**, en commençant par le bas de l'architecture — jamais en `strict` global sur du legacy.
3. Le seuil de couverture est un **cliquet** contre la régression, réglé juste sous la valeur actuelle. Ce n'est pas un objectif à atteindre.

5. Tests et workflow d'équipe

Les règles qui survivent à la formation

Quand écrire un test

Situation	Test à écrire	Quand
Nouvelle règle métier	unitaire, en TDD	avant le code
Correction de bug	non-régression	avant la correction
Refactoring	caractérisation	avant de toucher au code
Nouvelle intégration externe	test d'intégration	pendant, pour apprendre le comportement réel
Code lu et compris difficilement	un test qui documente	pendant la lecture
Fonction d'affichage, log, confort	aucun	—

Les trois premières lignes disent **avant**. Ce n'est pas une préférence méthodologique : un test écrit après n'a jamais été vu rouge, donc rien ne prouve qu'il détecte quoi que ce soit.

Ajouter un test lors d'une correction de bug

La séquence, telle qu'elle devrait figurer dans le guide de contribution :

```
1. Reproduire le bug par un test      → rouge
2. Committer le test seul             → « test: reproduit #4231 »
3. Corriger                          → vert
4. Committer la correction            → « fix: corrige #4231 »
5. Élargir aux cas voisins           → la famille du bug
```

L'étape 2 est celle qu'on saute. Elle a pourtant deux vertus :

- le relecteur voit **la reproduction** séparée de **la correction** ;
- l'historique contient un commit où le test échoue — la preuve définitive qu'il détecte bien le bug.

 **En pratique** : sur une base sans tests, c'est la règle la plus rentable à instaurer. Elle ne demande aucun budget dédié et concentre l'effort exactement là où le code casse déjà.

Relire une merge request sous l'angle qualité

Questions à poser, dans cet ordre — les trois premières prennent deux minutes :

#	Question	Signal
1	La pipeline est-elle verte ?	sinon, on ne relit pas encore
2	Y a-t-il un test pour ce qui change ?	sinon, pourquoi ?
3	Les noms de tests décrivent-ils un comportement ?	<code>test_cas_2</code> → à renommer
4	Les cas limites sont-ils couverts ?	vide, borne, erreur attendue
5	Le test casserait-il si le bug revenait ?	le test décisif
6	Le contrat public change-t-il ?	signature, exception, valeur de retour
7	Y a-t-il des mocks superflus ?	signe de couplage non traité

La question 5 est la plus utile et la moins posée. Un test qui reste vert quand on réintroduit le bug ne protège rien. En cas de doute, demandez à l'auteur de le montrer.

Ne pas casser une librairie utilisée par d'autres

Une modification est **cassante** dès qu'un projet consommateur doit changer son code pour continuer à fonctionner :

Changement	Cassant ?
Ajouter une fonction	non
Ajouter un paramètre avec valeur par défaut	non
Ajouter un paramètre sans valeur par défaut	oui
Renommer une fonction, un paramètre, une classe publique	oui
Changer le type de retour	oui
Lever une nouvelle exception dans un cas qui marchait	oui

⚠ **La dernière ligne** est la plus délicate dans une librairie scientifique : une correction légitime modifie des résultats déjà publiés. Elle se traite comme une rupture — annonce, version majeure, note explicite.

Déprécier plutôt que supprimer

```
import warnings

def calculer_indice_v1(mesures):
    """Ancienne formule d'indice.

    .. deprecated:: 2.3
       Utiliser :func:`calculer_indice`. Suppression prévue en 3.0.
    """
    warnings.warn(
        "calculer_indice_v1 est déprécié depuis la 2.3, "
        "utiliser calculer_indice ; suppression en 3.0",
        DeprecationWarning,
        stacklevel=2,
    )
    return calculer_indice(mesures, formule="v1")
```

Et le test qui garantit que l'avertissement est bien émis :

```
def test_l_ancienne_api_avertit():
    with pytest.warns(DeprecationWarning, match="3.0"):
        calculer_indice_v1(MESURES)
```

Trois éléments obligatoires dans le message : **depuis quand**, **par quoi remplacer**, **quand ce sera supprimé**.

Documentation minimale des tests

Ce qui mérite d'être écrit, et ce qui ne le mérite pas :

À documenter	Où
Pourquoi ce cas limite existe	docstring du test
Le numéro de ticket d'un bug	docstring du test de non-régression
Qu'un test est de caractérisation	docstring, en toutes lettres, avec la date
Comment lancer la suite	<code>README</code> du projet, cinq lignes

À ne pas documenter
Ce que fait le test — son nom doit suffire
Chaque assertion, commentée ligne à ligne
...

La règle : un commentaire dans un test explique un **pourquoi**. Si vous devez expliquer le **quoi**, renommez le test.

Les standards d'équipe qui tiennent

Cinq règles, tenant sur une page du guide de contribution :

1. Aucune correction de bug sans le test qui échouait avant.
2. Le code neuf est testé — vérifié par la couverture du diff.
3. Une pipeline rouge se traite avant tout autre travail.
4. Un test instable est supprimé ou réparé dans la semaine — jamais relancé.
5. Une rupture d'API publique s'annonce et se déprécie avant de se supprimer.

Ce qui rend ces règles applicables : **quatre sur cinq sont vérifiables automatiquement**. La seule qui repose sur la discipline est la troisième — et c'est celle qui déterminera si tout le reste sert à quelque chose.

À retenir

1. Le test s'écrit **avant** dans les trois cas qui comptent : nouvelle règle, correction de bug, refactoring. Sinon il n'a jamais été vu rouge.
2. En revue, la question décisive est : *ce test casserait-il si le bug revenait ?*
3. Dans une librairie partagée, corriger un calcul faux est une rupture pour vos consommateurs. Elle s'annonce.

Travaux pratiques du Jour 5

Trois exercices et un bilan — 3 h

Le programme

TP	Ce que vous allez faire	Durée
j5-01	Écrire un <code>.gitlab-ci.yml</code> complet pour un vrai package, <code>calculatrice</code> : stages, cache <code>uv</code> , matrice de versions. Validé par un simulateur local qui exécute réellement les jobs.	60 min
j5-02	Sur le package <code>todolist</code> , produire un rapport JUnit et un rapport de couverture, trouver le bug que cachent les 28 % non couverts, et régler le seuil en cliquet plutôt qu'en objectif.	45 min
j5-03	Recevoir une merge request rouge sur le package <code>convertisseur</code> : lire le rapport, diagnostiquer, corriger, valider.	45 min
Bilan	En groupe : chacun repart avec trois règles applicables à sa librairie dès lundi, et une zone de code par laquelle commencer.	30 min

Une pipeline s'exécute sans GitLab

Vous n'avez pas de runner sous la main. Ce n'est pas un obstacle :

```
uv run python outils/simuler_pipeline.py
```

Ce simulateur lit votre `.gitlab-ci.yml`, en extrait les stages et les jobs, et **exécute réellement leurs commandes** dans l'ordre, en respectant les codes de retour.

 **Le principe qui compte** : toute commande de votre pipeline doit être exécutable sur votre poste. Si le simulateur ne peut pas la lancer, votre collègue ne pourra pas la déboguer non plus.

Sur `j5-03`, résistez à l'envie d'ouvrir le code source en premier. Commencez par le rapport de tests — c'est la compétence que le TP fait travailler.

Conclusion

Ce qu'on emporte, et par quoi commencer

Le fil des cinq jours

Jour	La question	La réponse retenue
1	Pourquoi ce code est-il dur à tester ?	il construit ses dépendances et mélange calcul et I/O
2	Que tester, et comment ?	le comportement, en priorité là où l'erreur est silencieuse
3	Comment tester ce qui dépend de l'extérieur ?	déplacer la frontière avant de sortir un mock
4	Comment améliorer sans casser ?	petits pas, sous filet, sans corriger de bug en même temps
5	Comment rendre tout cela systématique ?	une pipeline qui rend l'exécution non négociable

Une seule idée les traverse :

Un code difficile à tester n'a pas un problème de tests.

Il a un problème de conception — et le test est ce qui le révèle.

Les cinq gestes les plus rentables

Par ordre de rapport bénéfice / effort, tels qu'observés sur les TPs :

1. Remplacer une dépendance construite par une dépendance **reçue**.

Une ligne. Transforme un module intestable en module testable.

2. Extraire la **fonction pure** au cœur d'un traitement.

Elle porte le risque métier et se teste sans rien monter.

3. Transformer chaque bug corrigé en **test de non-régression**.

Cinq minutes, et le cas est déjà connu.

4. Remplacer un `return None` silencieux par une **exception métier**.

Fait échouer bruyamment ce qui produisait des résultats faux.

5. Mettre en place une **pipeline de six lignes**.

Rend visible ce que l'équipe fait déjà.

Un plan à trente jours pour l'équipe

Semaine	Action	Effort
1	Pipeline minimale : <code>pytest</code> à chaque poussée, même sur trois tests	½ j
1	<code>ruff format</code> + <code>ruff check</code> en CI, non bloquants d'abord	½ j
2	Cartographier les zones critiques (<code>git log</code> + incidents + consommateurs)	½ j
2	Règle d'équipe : aucune correction de bug sans son test	0 j
3	Test de bout en bout sur les deux traitements les plus critiques	2 j
4	Extraire et tester les règles métier de la zone la plus risquée	2 j
4	Rapport JUnit et couverture du diff dans les merge requests	½ j

Ce plan ne demande **jamais** d'arrêter les évolutions fonctionnelles, et ne fixe **aucun objectif de couverture globale**.

Trois pièges à éviter en rentrant

- ⚠ **Appliquer les patterns partout.** Un pattern qui ne rend aucun test possible n'apporte rien. Le critère est là, et nulle part ailleurs.
- ⚠ **Viser un pourcentage de couverture.** Vous l'atteindrez, et vous n'aurez rien gagné. Visez la couverture du **code neuf**.
- ⚠ **Vouloir tout reprendre d'un coup.** Testez ce que vous vous apprêtez à modifier. Le reste attendra son tour, légitimement.

Et une chose à faire dès lundi : reprenez **le code de votre équipe** auquel vous pensiez au premier jour. Posez-lui la question du chapitre 5 du Jour 1 — *de quoi ai-je besoin pour tester ceci ?* — et écrivez le premier test.

Pour aller plus loin

Sujet	Référence
Refactoring et catalogue de transformations	<i>Refactoring</i> , Martin Fowler
Code legacy, coutures, caractérisation	<i>Working Effectively with Legacy Code</i> , Michael Feathers
Doubles de test, état vs interaction	<i>Growing Object-Oriented Software, Guided by Tests</i> , Freeman & Pryce
Frontières et architecture testable	<i>Clean Architecture</i> , Robert C. Martin
Outillage Python	documentation de <code>pytest</code> , <code>ruff</code> , <code>uv</code> , <code>import-linter</code>

Le support complet, les 19 TPs et leurs corrigés restent à votre disposition.

Merci

Questions, retours, et bon courage pour lundi